华为认证 Cloud 系列教程

华为认证云计算容器高级工程师

HCIP-Cloud Computing-Container V1.0

实验指导手册

版本:1.0



华为技术有限公司

版权所有 © 华为技术有限公司 2019。 保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标声明


和其他华为商标均为华为技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

您购买的产品、服务或特性等应受华为公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，华为公司对本文档内容不做任何明示或暗示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

华为技术有限公司

地址： 深圳市龙岗区坂田华为总部办公楼 邮编：518129

网址： http://e.huawei.com

华为认证体系介绍

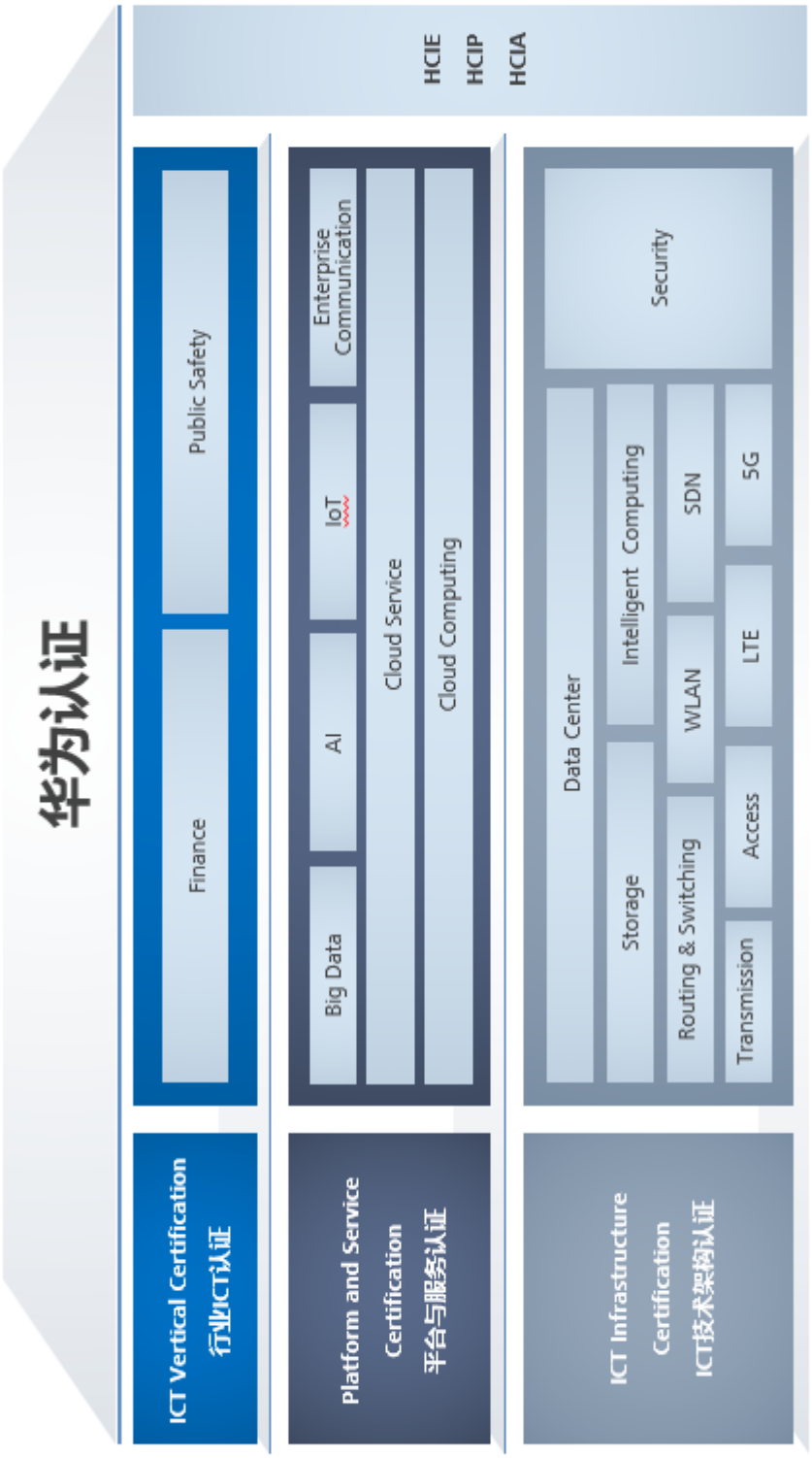
基于“平台+AI+生态”战略，围绕“云-管-端”协同的新ICT技术架构，华为公司打造了业界唯一覆盖ICT全技术领域的认证体系，包含ICT技术架构认证、平台与服务认证和行业ICT认证三类认证。

根据ICT从业者的学习和进阶需求，华为认证分为工程师级别、高级工程师级别和专家级别三个认证等级。

华为认证覆盖ICT全领域，符合ICT融合的技术趋势，致力于提供领先的人才培养体系和认证标准，培养数字化时代的新型ICT人才，构建良性的ICT人才生态。

HCIP-Cloud Computing-Container V1.0认证主要面向华为产品的用户、合作伙伴工程师、ISV工程师、内部工程师、高校学生以及ICT从业人员等。HCIP-Cloud Computing-Container认证在内容上涵盖容器技术概述、容器镜像、容器网络和存储、容器底层实现技术、PaaS概述、Kubernetes架构介绍、Deployment管理与使用、Pod管理与使用、Label与Label Selector、Service服务发现、DaemonSet与Job、Pod健康检查、Kubernetes网络和存储、ConfigMap与Secret、StatefulSet管理与使用、Kubernetes服务质量和资源调度、Kubernetes Dashboard、Helm包管理工具、RBAC权限控制、Kubernetes日志和监控。

华为认证协助您打开行业之窗，开启改变之门，屹立在云计算世界的潮头浪尖！


华为认证
HCIE
HCIP
HCIA
ICT Vertical Certification
行业ICT认证
Finance
Public Safety
Platform and Service
Certification
平台与服务认证
Big Data
AI
IoT
Enterprise Communication
Cloud Service
Cloud Computing
ICT Infrastructure
Certification
ICT技术架构认证
Data Center
Storage
Intelligent Computing
Routing & Switching
WLAN
SDN
Transmission
Access
LTE
5G
Security

前 言

简介

本书为 HCIP-Cloud Computing-Container V1.0 认证培训教程，适用于准备参加 HCIP-Cloud Computing-Container V1.0 认证考试的学员或者希望学习开源 Container 和 Kubernetes 技术的读者。

内容描述

本实验指导书匹配 HCIP-Cloud Computing-Container V1.0 教材。

本实验指导书共包含 2 大部分：Container 实验和 Kubernetes 实验。其中 Container 部分又分为 5 个模块，Kubernetes 部分包括 18 个模块。

其中每个模块又分为“实验”和“实训任务”两部分。实验内容为 step by step 操作；实训任务要求根据理论和实验中所学知识，由学员独立完成。

- Container 部分包含 5 个实验：容器基础操作、容器镜像、容器网络、容器存储、namespace 和 cgroups。
- Kubernetes 部分包含 18 个实验：Kubernetes 组件、Deployment 实验、Pod 实验、Label 与 Label Selector、Service 实验、DeamonSet 与 Job、健康检查、Kubernetes 网络、Kubernete 存储、ConfigMap 和 Secret、StatefulSet、Qos、Kubernetes 资源调度、Kubernetes Dashboard、Helm 包管理工具、RBAC 权限控制、Kubernetes 日志管理、Prometheus。

读者知识背景

本课程为华为认证高级课程，为了更好地掌握本书内容，阅读本书的读者应首先具备以下基本条件：

- 具有良好 IT 技能，了解服务器、存储、网络、虚拟化等方向的技术要点；
- 具有云计算相关从业经验，了解云计算发展趋势；
- 具有较系统的 Linux 操作系统知识和操作经验。

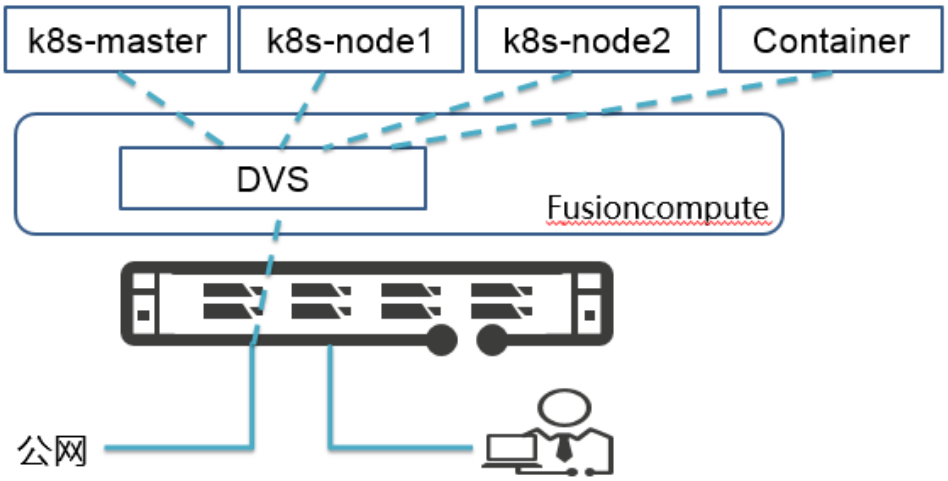
实验环境说明

组网说明

HCIP-Cloud Computing-Container V1.0 实验环境包括 Container 环境和 Kubernetes 环境两部分内容。该两部分实验环境均搭建在 FusionCompute 系统上的虚拟机中。

本课程的实验环境共需 4 台 VM：1 台用于搭建 Container 实验环境，3 台用于搭建 Kubernetes 实验环境。

- Container 环境：1 台 VM。
- Kubernetes 环境：1 台 VM 作为 k8s-master，2 台 VM 作为 node（分别为 k8s-node1 和 k8s-node2）。



HCIP-Cloud Computing-Container V1.0 实验拓扑图

注意事项：

1. Kubernetes 部分实验中，Master 节点用于 Cli 接入，默认情况下使用 Putty 登录 Master 节点进行实验操作。如果实验步骤中不指明在哪个节点输入命令，默认指代 Master 节点。

设备介绍

- 为了满足HCIP-Cloud Computing-Contaienr V1.0实验需要，建议每套实验环境采用以下配置：

设备名称	设备型号	数量
服务器	2288H V5	1

交换机	S5700	1

- 2288H V5服务器配置规格建议如下，详见本课程配套的“设备清单“。

设备名称	CPU	内存	硬盘	网卡
2288H V5	2 路 12-core	192G	2*SSD（960G）+3*SAS（1.2T）	2*GE+2*10GE

- 2288H V5服务器RAID配置说明：

两块 SSD 配置为 RAID1，用于安装 FusionCompute 系统。

软件和工具

本实验过程中会使用以下工具：

软件包或工具	用途
Putty	SSH 登录。
WinSCP	FTP 传输文件。

环境准备

请根据课程配套发布的 HCIP-Cloud Computing-Container V1.0 实验环境搭建指南，搭建实验环境。

常用 Linux 命令

命令	作用	举例
cd	切换目录	cd /mnt
ls	查看文件或目录信息	ls /tmp
pwd	查看当前位置的绝对路径	pwd
vim、vi	编辑文件	vim ~/.bash_profile
cp	拷贝文件或目录	cp /etc/passwd /tmp
mv	移动文件或目录	mv /tmp/passwd /tmp/psw.txt
cat	显示文件内容	cat /etc/passwd
mkdir	创建目录	mkdir /hcip

touch	创建文件或更新文件时间戳	touch ~/hcip.sh
chmod	修改文件或目录权限	chmod u+x ~/hcip.sh
chown	修改文件或目录的uid或gid	chown huawei:huawei ~/hcip.sh
df	显示分区使用情况	df -h

目录

前　言 3
简介 3
内容描述 3
读者知识背景 3
实验环境说明 4
1 Container 实验 12
1.1 实验介绍 12
1.1.1 关于本实验 12
1.1.2 实验目的 12
1.2 容器基础操作 12
1.2.1 运行第一个容器 12
1.2.2 docker attach 进入容器 13
1.2.3 docker exec 进入容器 14
1.2.4 实训任务 15
1.3 容器镜像 16
1.3.1 docker commit 构建镜像 16
1.3.2 Dockerfile 构建镜像 17
1.3.3 搭建私有 Registry 18
1.3.4 实训任务 20
1.4 容器网络（可选） 21
1.4.1 Bridge 网络 21
1.5 容器存储 24
1.5.1 volume 24
1.5.2 bind mount 25
1.5.3 volume container 27
1.6 Namespace 和 Cgroups 28
1.6.1 Namespace 28

1.6.2 CPU 资源限制 29
1.6.3 内存资源限制 31
1.6.4 实训任务 31
2 Kubernetes 实验 32
2.1 实验介绍 32
2.1.1 关于本实验 32
2.1.2 实验目的 32
2.2 Kubernetes 组件 32
2.2.1 配置步骤 32
2.3 Deployment 实验 34
2.3.1 运行第一个 Deployment 34
2.3.2 使用 kubectl 命令行工具 35
2.3.3 创建 Deployment 36
2.3.4 弹性伸缩 Deployment 38
2.3.5 滚动升级 deployment 38
2.3.6 实训任务 40
2.4 Pod 实验 41
2.4.1 使用 Pod 41
2.4.2 运行一次性 Pod 44
2.4.3 实训任务 45
2.5 Label 与 Label Selector 45
2.5.1 标签 45
2.5.2 标签选择器 46
2.5.3 使用标签选择器实现调度 47
2.5.4 实训任务 49
2.6 Service 服务发现 50
2.6.1 使用 Service 50
2.6.2 使用 DNS 52
2.6.3 使用 Headless Service 53
2.6.4 实训任务 55
2.7 DeamonSet 与 Job 55

2.7.1 使用 DaemonSet 55
2.7.2 使用 Job 56
2.7.3 使用 CronJob 57
2.7.4 实训任务 58
2.8 健康检查 59
2.8.1 使用存活探针 59
2.8.2 使用 readiness 探针 62
2.8.3 实训任务 65
2.9 Kubernetes 网络 65
2.9.1 Node 与 Pod 间通信 65
2.9.2 Pod 和 Pod 间通信 66
2.9.3 集群外访问实验（NodePort 方式） 67
2.10 Kubernetes 存储 68
2.10.1 使用 emptyDir 68
2.10.2 使用 emptyDir 的容量限制功能 71
2.10.3 hostPath 72
2.10.4 PV 和 PVC 73
2.10.5 使用 StorageClass 方式关联 PV 和 PVC 77
2.10.6 实训任务 78
2.11 Configmap 和 Secret 79
2.11.1 创建 configmap 79
2.11.2 使用 configmap 82
2.11.3 使用 Secret 84
2.11.4 使用 secret 85
2.11.5 实训任务 87
2.12 StatefulSet（可选） 88
2.12.1 创建 StatefulSet 88
2.12.2 StatefulSet 常见操作 91
2.12.3 实训任务 93
2.13 QoS 94
2.13.1 配置 Pod 的资源配额 94

2.13.2 测试内存限制 96
2.13.3 测试节点资源限额 97
2.14 Kubernetes 资源调度 98
2.14.1 nodeselector：根据节点选择器调度 98
2.14.2 Pod 资源亲和调度 101
2.14.3 创建 Kubernetes 调度优先级 PriotityClass 103
2.14.4 实训任务 104
2.15 Kubernetes Dashboard 104
2.15.1 登录 Dashboard 104
2.15.2 “CREATE AN APP”方式部署 Deployment 105
2.15.3 “CREATE FROM TEXT INPUT”方式部署 Pod 109
2.15.4 “CREATE FROM FILE”方式部署 Deployment 112
2.16 Helm 包管理工具 115
2.16.1 自定义创建 Chart 115
2.16.2 将自定义的应用发布到本地 Repository 119
2.16.3 helm 升级和回退一个应用 120
2.16.4 搭建 helm 私有仓库 121
2.16.5 实训任务 122
2.17 RBAC 权限控制 123
2.17.1 创建 User Account 123
2.17.2 使用 User Account 设定 kube-config 配置文件 124
2.17.3 使用内置的 ClusterRole 绑定新建账户 125
2.17.4 创建 Service Account 等相关对象 126
2.17.5 创建 Pod 引用该 Service Account 128
2.17.6 实训任务 130
2.18 Kubernetes 日志管理（可选） 130
2.18.1 使用 sidecar 输出日志 130
2.18.2 EFK 的简单使用 133
2.19 Prometheus（可选） 139
2.19.1 Prometheus 的简单使用 139

1 Container 实验

1.1 实验介绍

1.1.1 关于本实验

本实验通过 Docker 容器生命周期管理、容器镜像、容器网络、容器存储、namespace 和 cgroup 等 5 个模块的实验练习和实训，让学员全面掌握容器的操作维护技能。

1.1.2 实验目的

- 掌握 Docker 容器基础操作。
- 掌握容器镜像的制作方法、搭建私有 Registry 的方法。
- 掌握容器间通信的原理。
- 掌握容器存储管理方式，了解容器数据共享方法。
- 掌握容器使用资源限制方法。

1.2 容器基础操作

1.2.1 运行第一个容器

步骤 1 创建一个名为 huawei1 的 httpd 容器。

```
[root@localhost ~]# docker create --name huawei1 httpd
```

```
[root@localhost ~]# docker create --name huawei1 httpd
2252def20ce276fed397069bf9ab85afd4af6a8854c5828d0d8e076eb18b2122
```

步骤 2 查看该容器信息。

```
[root@localhost ~]# docker ps -a
```

```
[root@localhost ~]# docker ps -a
CONTAINER ID  IMAGE  COMMAND              CREATED          STATUS   PORT  NAMES
2252def20ce2  httpd  "httpd-foreground"  14 minutes ago  Created        huawei1
```

查询到容器 ID 为 2252def20ce2，容器名称为 huawei1，容器状态为已创建 Created。

步骤 3 启动容器 huawei1。

```
[root@localhost ~]# docker start huawei1
```

或者

```
[root@localhost ~]# docker start 2252def20ce2
```

```
[root@localhost ~]# docker start 2252def20ce2
2252def20ce2
```

步骤 4 再次查看容器的 huawei1 信息，状态为 UP。

```
[root@localhost ~]# docker container ls
```

```
[root@localhost ~]# docker container ls
CONTAINER ID   IMAGE   COMMAND              CREATED         STATUS         PORT     NAMES
2252def20ce2   httpd   "httpd-foreground"   15 minutes ago   UP 2 minutes   80/tcp   huawei1
```

步骤 5 停止容器 huawei1，并查看到该容器状态为 Exited。

```
[root@localhost ~]# docker stop huawei1
```

```
[root@localhost ~]# docker stop huawei1
huawei1
```

```
[root@localhost ~]# docker ps -a
```

```
[root@localhost ~]# docker ps -a
CONTAINER ID   IMAGE   COMMAND              CREATED         STATUS                     NAMES
2252def20ce2   httpd   "httpd-foreground"   15 minutes ago   Exited (0) 2 minutes ago   huawei1
```

步骤 6 删除容器 huawei1。

```
[root@localhost ~]# docker rm huawei1
```

```
[root@localhost ~]# docker rm huawei1
huawei1
```

1.2.2 docker attach 进入容器

步骤 1 长期运行一个 centos 容器。

```
[root@localhost ~]# docker run -d centos /bin/bash -c "while true; do sleep 1;
echo Huawei; done"
```

回显：

```
9c4252c22352b666d7e8da56d538da452a33a48864173677754130fd8e1260cb
```

步骤 2 使用 docker attach 命令直接进入这个 centos 容器终端。查看到每隔 1 秒钟输出字符“Huawei”。

```
[root@localhost ~]# docker attach 9c4252c22352
```

```
[root@localhost ~]# docker attach 9c4252c22352
Huawei
Huawei
Huawei
```

步骤 3 打开另一个宿主机终端，暂停这个容器。

```
[root@localhost ~]# docker pause 9c4252c22352
[root@localhost ~]# docker pause 9c4252c22352
9c4252c22352
```

步骤 4 在 docker attach 进入容器的终端上，按“Enter”键，发现字符串“Huawei”已停止输出。

```
Huawei
Huawei
Huawei
```

步骤 5 查看该 centos 容器的状态，状态为 Paused。

```
[root@localhost ~]# docker ps -a | grep 9c4252c22352
[root@localhost ~]# docker ps -a | grep 9c4252c22352
9c4252c22352  centos  "/bin/bash -c 'while…"  9 minutes ago  Up 9 minutes (Paused)  zen_feynman
```

步骤 6 恢复该 centos 容器的运行状态。

```
[root@localhost ~]# docker unpause 9c4252c22352
[root@localhost ~]# docker unpause 9c4252c22352
9c4252c22352
```

步骤 7 再次在 docker attach 进入容器的终端上查看，已恢复字符串“Huawei”的输出。

步骤 8 强制停止容器进程。

```
[root@localhost ~]# docker kill 9c4252c22352
[root@localhost ~]# docker kill 9c4252c22352
9c4252c22352
```

1.2.3 docker exec 进入容器

步骤 1 在后台运行一个名为“httpd1”的 httpd 容器，并将其服务端口 80 映射到宿主机 8080 端口。

```
[root@localhost ~]# docker run --name httpd1 -d -p 8080:80 httpd
[root@localhost ~]# docker run --name httpd1 -d -p 8080:80 httpd
bf68eba8a1ec98f578e129e9211b4603a668c466e824e4637491310ea622de8a
```

步骤 2 访问容器 httpd1。

```
[root@localhost ~]# curl 127.0.0.1:8080
```

```
[root@localhost ~]# curl 127.0.0.1:8080
<html><body><h1>It works!</h1></body></html>
```

步骤 3 进入容器 httpd1。

```
[root@localhost ~]# docker exec -it httpd1 bash
```

```
[root@localhost ~]# docker exec -it httpd1 bash
root@bf68eba8a1ec:/usr/local/apache2# ls
bin  build  cgi-bin  conf  error  htdocs  icons  include  logs  modules
root@bf68eba8a1ec:/usr/local/apache2#
```

步骤 4 修改 httpd1 容器中静态文件内容，修改完成后输入“exit”退出。

```
echo "update to httpd" > index.html
```

```
root@bf68eba8a1ec:/usr/local/apache2# cd htdocs
root@bf68eba8a1ec:/usr/local/apache2/htdocs# ls
index.html
root@bf68eba8a1ec:/usr/local/apache2/htdocs# echo "update to httpd" > index.html
root@bf68eba8a1ec:/usr/local/apache2/htdocs# exit
exit
```

步骤 5 再次访问容器 httpd1。容器依旧可正常访问，说明“exit”退出并不会导致容器进程结束。

```
[root@localhost ~]# curl 127.0.0.1:8080
```

```
[root@localhost ~]# curl 127.0.0.1:8080
update to httpd
```

步骤 6 为方便后续实验，将本小节中的容器删除。

```
docker kill
docker rm
```

1.2.4 实训任务

步骤 1 启动一个 httpd 容器并命名为 httpd10。

步骤 2 进入容器并在容器中执行命令，使之每隔 10s 输出字符串“Huawei”。要求，不使用 docker attach 和 docker exec 命令。

步骤 3 在宿主机浏览器中使用 8081 端口访问该容器，且要求浏览器中显示的内容是“Hello World”。

步骤 4 在宿主机上使用命令查看该容器中持续输出的字符串“Huawei”。

步骤 5 任务完成后，删除容器 httpd10。

1.3 容器镜像

1.3.1 docker commit 构建镜像

步骤 1 在后台运行一个 nginx 容器，取名为 nginx1，并映射到宿主机 80 端口。

```
[root@localhost ~]# docker run --name nginx1 -d -p 80:80 nginx
[root@localhost ~]# docker run --name nginx1 -d -p 80:80 nginx
7fe26d09e985a70d860481f5c86bd00dcc0a926b21c6a368ed8c27435f916716
```

步骤 2 在操作机上使用浏览器访问容器 nginx1。



步骤 3 进入容器修改 nginx1 的 index.html 文件内容。

```
[root@localhost ~]# docker exec -it 7fe26d09e985 bash
[root@localhost ~]# docker exec -it 7fe26d09e985 bash
root@7fe26d09e985:/# echo "update the nginx index" > /usr/share/nginx/html/index.html
root@7fe26d09e985:/# exit
exit
[root@localhost ~]#
```

步骤 4 验证 nginx1 容器内容修改成功。

```
[root@localhost ~]# curl 127.0.0.1:80
[root@localhost ~]# curl 127.0.0.1:80
update the nginx index
```

步骤 5 将容器 nginx1 保存为镜像，新镜像命名为 nginx:v1.1。

```
[root@localhost ~]# docker commit \
[root@localhost ~]# docker commit \
> --author "Michael@huawei.com" \
> --message "update the index" \
> nginx1 \
> nginx:v1.1
sha256:f31a5d13dae18c0ba28c00e2034e04e7e8dc307395cd9c6edd5342ce50d98955
```

步骤 6 比较 nginx:v1.1 和 nginx:latest 两个镜像，发现 nginx:v1.1 多了一个镜像层。

```
[root@localhost ~]# docker history nginx:v1.1
```

```
[root@localhost ~]# docker history nginx:v1.1
IMAGE          CREATED             CREATED BY                                      SIZE      COMMENT
f31a5d13dae1   5 minutes ago       nginx -g daemon off;                            95B       update the index
e445ab08b2be   2 weeks ago         /bin/sh -c #(nop)  CMD ["nginx" "-g" "daemon…   0B
<missing>      2 weeks ago         /bin/sh -c #(nop)  STOPSIGNAL SIGTERM           0B
<missing>      2 weeks ago         /bin/sh -c #(nop)  EXPOSE 80                    0B
<missing>      2 weeks ago         /bin/sh -c ln -sf /dev/stdout /var/log/nginx…   22B
<missing>      2 weeks ago         /bin/sh -c set -x     && addgroup --system -…   56.6MB
<missing>      2 weeks ago         /bin/sh -c #(nop)  ENV PKG_RELEASE=1~buster     0B
<missing>      2 weeks ago         /bin/sh -c #(nop)  ENV NJS_VERSION=0.3.3        0B
<missing>      2 weeks ago         /bin/sh -c #(nop)  ENV NGINX_VERSION=1.17.2     0B
<missing>      2 weeks ago         /bin/sh -c #(nop)  LABEL maintainer=NGINX Do…   0B
<missing>      4 weeks ago         /bin/sh -c #(nop)  CMD ["bash"]                 0B
<missing>      4 weeks ago         /bin/sh -c #(nop) ADD file:71ac26257198ecf6a…   69.2MB
```

```
[root@localhost ~]# docker history nginx
```

```
[root@localhost ~]# docker history nginx
IMAGE          CREATED             CREATED BY                                      SIZE      COMMENT
e445ab08b2be   2 weeks ago         /bin/sh -c #(nop)  CMD ["nginx" "-g" "daemon…   0B
<missing>      2 weeks ago         /bin/sh -c #(nop)  STOPSIGNAL SIGTERM           0B
<missing>      2 weeks ago         /bin/sh -c #(nop)  EXPOSE 80                    0B
<missing>      2 weeks ago         /bin/sh -c ln -sf /dev/stdout /var/log/nginx…   22B
<missing>      2 weeks ago         /bin/sh -c set -x     && addgroup --system -…   56.6MB
<missing>      2 weeks ago         /bin/sh -c #(nop)  ENV PKG_RELEASE=1~buster     0B
<missing>      2 weeks ago         /bin/sh -c #(nop)  ENV NJS_VERSION=0.3.3        0B
<missing>      2 weeks ago         /bin/sh -c #(nop)  ENV NGINX_VERSION=1.17.2     0B
<missing>      2 weeks ago         /bin/sh -c #(nop)  LABEL maintainer=NGINX Do…   0B
<missing>      4 weeks ago         /bin/sh -c #(nop)  CMD ["bash"]                 0B
<missing>      4 weeks ago         /bin/sh -c #(nop) ADD file:71ac26257198ecf6a…   69.2MB
```

步骤 7 以镜像 nginx:v1.1 运行一个容器。

```
[root@localhost ~]# docker run --name nginx2 -d -p 81:80 nginx:v1.1
```

```
[root@localhost ~]# docker run --name nginx2 -d -p 81:80 nginx:v1.1
624331ff508aaf0afaf45fde2b4c2ab9eac1062d25113e520854983fb3c0ac53
```

步骤 8 为方便后续实验，将本小节中的容器删除。

```
docker kill
docker rm
```

1.3.2 Dockerfile 构建镜像

步骤 1 在/root 目录下创建一个 dockerfile 文件夹

```
[root@localhost ~]# mkdir dockerfile
```

```
[root@localhost ~]# mkdir dockerfile
[root@localhost ~]#
```

步骤 2 创建一个名为 dockerfile1 的 dockerfile 文件。

```
[root@localhost dockerfile]# touch dockerfile1
```

```
[root@localhost ~]# cd dockerfile/
[root@localhost dockerfile]# touch dockerfile1
```

步骤 3 使用 vi 编辑器编辑 dockerfile1，输入如下内容。

```
[root@localhost dockerfile]# vi dockerfile1
```

```
[root@localhost dockerfile]# vi dockerfile1

FROM httpd
MAINTAINER Allen@Huawei.com
RUN echo "dockefile test" > /usr/local/apache2/htdocs/index.html
```

步骤 4 构建镜像，镜像命名为 httpd:v11

```
[root@localhost dockerfile]# docker build -t httpd:v11 -f dockerfile1
/root/dockerfile
```

```
[root@localhost dockerfile]# docker build -t httpd:v11 -f dockerfile1 /root/dockerfile
Sending build context to Docker daemon  2.048kB
Step 1/3 : FROM httpd
 ---> ee39f68eb241
Step 2/3 : MAINTAINER Allen@Huawei.com
 ---> Running in cd3cfd1b4d4b
Removing intermediate container cd3cfd1b4d4b
 ---> 7119054970a9
Step 3/3 : RUN echo "dockefile test" > /usr/local/apache2/htdocs/index.html
 ---> Running in 6c116e3df88c
Removing intermediate container 6c116e3df88c
 ---> 2b69b19cfcdb
Successfully built 2b69b19cfcdb
Successfully tagged httpd:v11
```

步骤 5 查看创建的镜像。

```
[root@localhost ~]# docker images
```

```
[root@localhost ~]# docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
httpd               v11                 2b69b19cfcdb        2 minutes ago       154MB
```

步骤 6 以镜像 httpd:v11 运行一个容器。

```
[root@localhost ~]# docker run -d -p 8081:80 httpd:v11
```

```
[root@localhost ~]# docker run -d -p 8081:80 httpd:v11
fcaf3f1b0bd0c0b95fab40d01e2b0da011ea209c95d40327cbd20ce992e1dca1
```

步骤 7 验证容器内容。

```
[root@localhost ~]# curl 127.0.0.1:8081
```

```
[root@localhost ~]# curl 127.0.0.1:8081
dockefile test
```

步骤 8 为方便后续实验，将本小节中的容器删除。

```
docker kill
docker rm
```

1.3.3 搭建私有 Registry

步骤 1 在/root 目录下创建一个 myregistry 文件夹，作为私有 Registry 的存储空间。

```
[root@localhost ~]# mkdir myregistry
```

```
[root@localhost ~]# mkdir myregistry
[root@localhost ~]#
```

步骤 2 运行一个 registry 容器，并将主机 1000 端口映射到到其服务端口 5000。同时挂载步骤 1 中创建的文件夹作为 image 存储空间。

```
[root@localhost ~]# docker run -d -p 1000:5000 -v
/root/myregistry:/var/lib/registry registry
[root@localhost ~]# docker run -d -p 1000:5000 -v /root/myregistry:/var/lib/registry registry
708e71f1d900a4ca1b5e5bb7b327fe75f61cee20411f2da53e07431163db2205
```

步骤 3 将 httpd:v11 更改成 Registry 要求的格式：

[Registry-host]:[port]/[username]/[repository:tag]

```
[root@localhost ~]# docker tag httpd:v11 127.0.0.1:1000/michael/httpd:v11
[root@localhost ~]# docker tag httpd:v11 127.0.0.1:1000/michael/httpd:v11
[root@localhost ~]# docker images
REPOSITORY                    TAG                 IMAGE ID            CREATED             SIZE
127.0.0.1:1000/michael/httpd  v11                 2b69b19cfcdb        33 minutes ago      154MB
httpd                         v11                 2b69b19cfcdb        33 minutes ago      154MB
```

步骤 4 将镜像 michael/httpd:v11 上传至 Registry。

```
[root@localhost ~]# docker push 127.0.0.1:1000/michael/httpd:v11
[root@localhost ~]# docker push 127.0.0.1:1000/michael/httpd:v11
The push refers to repository [127.0.0.1:1000/michael/httpd]
fa3e0ddac404: Pushed
635721fc6973: Pushed
bea448567d6c: Pushed
bfaa5f9c3b51: Pushed
9d542ac296cc: Pushed
d8a33133e477: Pushed
v11: digest: sha256:29ac807152e3439c7b82a074dbb7dd2ab8c29798c9fd8b0ddd6d7559bfd10922 size: 1574
```

步骤 5 查看 Registry 中的镜像。

```
[root@localhost ~]# curl 127.0.0.1:1000/v2/_catalog
[root@localhost ~]# curl 127.0.0.1:1000/v2/_catalog
{"repositories":["michael/httpd"]}
```

步骤 6 在/root/myregistry 目录中查看相应的镜像。

```
[root@localhost sha256]# pwd
/root/myregistry/docker/registry/v2/repositories/michael/httpd/_layers/sha256
[root@localhost sha256]# ll
total 0
drwxr-xr-x. 2 root root 18 Aug 13 14:24 2b69b19cfcdb9cbfd8e16df18d386a7321c187dcd51b3032b1cd7a6e60e6922b
drwxr-xr-x. 2 root root 18 Aug 13 14:24 622a9dd8cfed1101585ed2d6e7df1d6f09b328d5944821c0e06f41d47883de50
drwxr-xr-x. 2 root root 18 Aug 13 14:24 98f47fcaa52fe04baffe4e3018677123529f15e268869314e221af7c254f56b3
drwxr-xr-x. 2 root root 18 Aug 13 14:24 b083c5fd185bd3ca1156b8695d8b8952d1741c765efcec28e34b443ef4153833
drwxr-xr-x. 2 root root 18 Aug 13 14:24 bb435d0316e98077ed6b053b2fa9476d3ea2123831ac06b4d8445bf257659747
drwxr-xr-x. 2 root root 18 Aug 13 14:24 bf5100a89e7815158a8afa82b6dfe3bd78e6761f6cc7b48333ff8eeaffbb2ce4
drwxr-xr-x. 2 root root 18 Aug 13 14:24 f5d23c7fed465a9eb762fc4c3ccod551a05914aba42492ceb972497db4df38bf
```

步骤 7 删除宿主机上的 httpd:v11 和 michael/httpd:v11 镜像。

```
[root@localhost ~]# docker rmi httpd:v11
```

```
[root@localhost ~]# docker rmi httpd:v11
Untagged: httpd:v11
```

```
[root@localhost ~]# docker rmi 127.0.0.1:1000/michael/httpd:v11
```

```
[root@localhost ~]# docker rmi 127.0.0.1:1000/michael/httpd:v11
Untagged: 127.0.0.1:1000/michael/httpd:v11
Untagged: 127.0.0.1:1000/michael/httpd@sha256:29ac807152e3439c7b82a074dbb7dd2ab8c29798c9fd8b0ddd6d7559bfd10922
Deleted: sha256:2b69b19cfcdb9cbfd8e16df18d386a7321c187dcd51b3032b1cd7a6e60e6922b
Deleted: sha256:0f403d50d27c06bf7c9bac451446d4c30e819ebfc7d24221a35b82889abc5ff8
Deleted: sha256:7119054970a9a3bc41ff499638242f3ef264bab7cb076363ee3b325dc91e4c7e
```

步骤 8 从 Registry 下载镜像 michael/httpd:v11。

```
[root@localhost ~]# docker pull 127.0.0.1:1000/michael/httpd:v11
```

```
[root@localhost ~]# docker pull 127.0.0.1:1000/michael/httpd:v11
v11: Pulling from michael/httpd
f5d23c7fed46: Already exists
b083c5fd185b: Already exists
bf5100a89e78: Already exists
98f47fcaa52f: Already exists
622a9dd8cfed: Already exists
bb435d0316e9: Pull complete
Digest: sha256:29ac807152e3439c7b82a074dbb7dd2ab8c29798c9fd8b0ddd6d7559bfd10922
Status: Downloaded newer image for 127.0.0.1:1000/michael/httpd:v11
```

1.3.4 实训任务

步骤 1 搭建一个私有 Registry。

步骤 2 创建一个 index.html 文件。

1、在宿主机的/root/df 目录下创建一个 index.html 文件；

2、Index.html 文件内容为“This is a webserver”；

步骤 3 使用 dockerfile 构建一个镜像：

1、以 httpd 作为 base image；

2、命名为 httpd:v12；

3、注明镜像作者；

4、将步骤 2 中创建的 index.html 文件添加到新镜像的对应目录下。

要求：以 httpd:v12 镜像运行的容器，web 访问页面输出为“This is a webserver”。

步骤 4 将 httpd:v12 其上传到私有 Registry。

步骤 5 删除宿主机上的 httpd:v12 镜像。

步骤 6 从 Registry 中下载 httpd:v12 镜像，然后运行一个 httpd:v12 容器，命名为 httpd12。

步骤 7 验证容器中的 web 服务可正常访问。

步骤 8　任务完成后，删除容器 httpd12。

1.4 容器网络（可选）

1.4.1 Bridge 网络

步骤 1　创建一个名为 net1 的 Bridge 网络。

```
[root@localhost ~]# docker network create --driver bridge net1
```

```
[root@localhost ~]# docker network create --driver bridge net1
93fb485fc2471ed14cc5292ddf3650f5eb277fe31a2c468b87052ad639a2135c
```

步骤 2　查看 net1 网桥，subnet 已自动配置为 172.18.0.0/16。

```
[root@localhost ~]# docker network inspect net1
```

```
[root@localhost ~]# docker network inspect net1
[
    {
        "Name": "net1",
        "Id": "93fb485fc2471ed14cc5292ddf3650f5eb277fe31a2c468b87052ad639a2135c",
        "Created": "2019-08-13T23:23:55.111499596-04:00",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.18.0.0/16",
                    "Gateway": "172.18.0.1"
                }
            ]
        },
```

步骤 3　创建一个名为 net2 的 bridge 网桥，并指定 subnet=172.10.10.0/24。

```
[root@localhost ~]# docker network create --driver bridge --subnet
172.10.10.0/24 --gateway 172.10.10.1 net2
```

```
[root@localhost ~]# docker network create --driver bridge --subnet 172.10.10.0/24 --gateway 172.10.10.1 net2
471e136c28316c9e232f503fdb113908fdf23954d76410e535e57ee69510ccc2
```

步骤 4　启动名为 centos1 的容器，并加入 net1 网络。

```
[root@localhost ~]# docker run --name centos1 -dit --network=net1 centos
```

```
[root@localhost ~]# docker run --name centos1 -dit --network=net1 centos
be7ab6026d73ca4417eb9ef325e2a7d388e8881314cb3bd4e8e562dfef9ec48c
```

步骤 5　启动名为 centos2 的容器，并加入 net2 网络。

```
[root@localhost ~]# docker run --name centos2 -dit --network=net2 centos
```

```
[root@localhost ~]# docker run --name centos2 -dit --network=net2 centos
88056a697e0856129d67a64251dabc66388e6e09453a1303b37e7d6a3d2ac580
```

步骤 6　启动名为 centos3 的容器，并加入 net2 网络，同时指定该容器 IP=172.10.10.10。

```
[root@localhost ~]# docker run --name centos3 -dit --network=net2 --ip
172.10.10.10 centos
[root@localhost ~]# docker run --name centos3 -dit --network=net2 --ip 172.10.10.10 centos
7d66df3d3acbfe3be565c5421f2e86bf68c29889298caba35c9dbe07a02b6135
```

步骤 7　分别查看 3 个 centos 容器的 IP 地址信息。其中 centos2 和 centos3 位于同一网段，centos1 与前两者位于不同网段。

```
docker inspect centos1
```

```
"Networks": {
    "net1": {
        "IPAMConfig": null,
        "Links": null,
        "Aliases": [
            "be7ab6026d73"
        ],
        "NetworkID": "93fb485fc2471ed14cc5292ddf3650f5eb277fe31a2c468b87052ad639a2135c",
        "EndpointID": "7943c6017295cae634bb5ede204802f3eb356741250f6be81bd35dc4215e9d37",
        "Gateway": "172.18.0.1",
        "IPAddress": "172.18.0.2",
        "IPPrefixLen": 16,
        "IPv6Gateway": "",
        "GlobalIPv6Address": "",
        "GlobalIPv6PrefixLen": 0,
        "MacAddress": "02:42:ac:12:00:02",
        "DriverOpts": null
    }
```

```
docker inspect centos2
```

```
"Networks": {
    "net2": {
        "IPAMConfig": null,
        "Links": null,
        "Aliases": [
            "88056a697e08"
        ],
        "NetworkID": "471e136c28316c9e232f503fdb113908fdf23954d76410e535e57ee69510ccc2",
        "EndpointID": "92d12ded719afdc0bc28d4dd0c0192ca3d12328a011e446a0945f1fb1501012f",
        "Gateway": "172.10.10.1",
        "IPAddress": "172.10.10.2",
        "IPPrefixLen": 24,
        "IPv6Gateway": "",
        "GlobalIPv6Address": "",
        "GlobalIPv6PrefixLen": 0,
        "MacAddress": "02:42:ac:0a:0a:02",
        "DriverOpts": null
    }
```

```
docker inspect centos3
```

```
"Networks": {
    "net2": {
        "IPAMConfig": {
            "IPv4Address": "172.10.10.10"
        },
        "Links": null,
        "Aliases": [
            "7d66df3d3acb"
        ],
        "NetworkID": "471e136c28316c9e232f503fdb113908fdf23954d76410e535e57ee69510ccc2",
        "EndpointID": "f47a1c7a32a018d27ab63b0c0fd64c65dc03731843931f992b56cf97ef43200e",
        "Gateway": "172.10.10.1",
        "IPAddress": "172.10.10.10",
        "IPPrefixLen": 24,
        "IPv6Gateway": "",
        "GlobalIPv6Address": "",
        "GlobalIPv6PrefixLen": 0,
        "MacAddress": "02:42:ac:0a:0a:0a",
        "DriverOpts": null
    }
```

步骤 8　进入容器 centos3，验证网络连通性。centos3 与 centos2 可以通信，但与 centos1 不能通信。

```
[root@localhost ~]# docker exec -it centos3 bash
```

```
[root@localhost ~]# docker exec -it centos3 bash
[root@7d66df3d3acb /]# ping 172.10.10.2
PING 172.10.10.2 (172.10.10.2) 56(84) bytes of data.
64 bytes from 172.10.10.2: icmp_seq=1 ttl=64 time=0.172 ms
64 bytes from 172.10.10.2: icmp_seq=2 ttl=64 time=0.105 ms
64 bytes from 172.10.10.2: icmp_seq=3 ttl=64 time=0.105 ms
^C
--- 172.10.10.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 1999ms
rtt min/avg/max/mdev = 0.105/0.127/0.172/0.032 ms
[root@7d66df3d3acb /]# ping 172.18.0.2
PING 172.18.0.2 (172.18.0.2) 56(84) bytes of data.
^C
--- 172.18.0.2 ping statistics ---
168 packets transmitted, 0 received, 100% packet loss, time 167000ms

[root@7d66df3d3acb /]# exit
```

步骤 9　为 centos1 添加一块网卡，并加入 net2 网络。

```
[root@localhost ~]# docker network connect net2 centos1
```

```
[root@localhost ~]# docker network connect net2 centos1
```

步骤 10　进入 centos1 容器测试连通性。

```
[root@localhost ~]# docker exec -it centos1 bash
[root@be7ab6026d73 /]# ping 172.10.10.2
PING 172.10.10.2 (172.10.10.2) 56(84) bytes of data.
64 bytes from 172.10.10.2: icmp_seq=1 ttl=64 time=0.126 ms
64 bytes from 172.10.10.2: icmp_seq=2 ttl=64 time=0.100 ms
64 bytes from 172.10.10.2: icmp_seq=3 ttl=64 time=0.109 ms
64 bytes from 172.10.10.2: icmp_seq=4 ttl=64 time=0.107 ms
^C
--- 172.10.10.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 2999ms
rtt min/avg/max/mdev = 0.100/0.110/0.126/0.014 ms
[root@be7ab6026d73 /]# ping 172.10.10.10
PING 172.10.10.10 (172.10.10.10) 56(84) bytes of data.
64 bytes from 172.10.10.10: icmp_seq=1 ttl=64 time=0.152 ms
64 bytes from 172.10.10.10: icmp_seq=2 ttl=64 time=0.123 ms
64 bytes from 172.10.10.10: icmp_seq=3 ttl=64 time=0.138 ms
64 bytes from 172.10.10.10: icmp_seq=4 ttl=64 time=0.132 ms
^C
--- 172.10.10.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 2999ms
rtt min/avg/max/mdev = 0.123/0.136/0.152/0.013 ms
```

步骤 11　为方便后续实验，删除本小节中的容器。

```
docker kill
docker rm
```

1.5 容器存储

1.5.1 volume

步骤 1　创建一个卷，并挂载个一个 httpd 容器。

```
[root@localhost ~]# docker run -d -p 8080:80 -v /usr/local/apache2/htdocs httpd
[root@localhost ~]# docker run -d -p 8080:80 -v /usr/local/apache2/htdocs httpd
0314232b0a05b4c2abf7affbdc16ac0aa651b825c1073b5f5d08ca468657030d
```

步骤 2　查看 volume 信息。

```
[root@localhost ~]# docker volume ls
[root@localhost ~]# docker volume ls
DRIVER              VOLUME NAME
local               3d5603513b758599b8695834235b3dc174379552dda002944bbbf013db2d05fc
```

步骤 3　查看容器的 volume 挂载信息，同时得到 volume 路径。Type=volume。

```
docker inspect 0314232b0a05
```

```
"Mounts": [
    {
        "Type": "volume",
        "Name": "3d5603513b758599b8695834235b3dc174379552dda002944bbbf013db2d05fc",
        "Source": "/var/lib/docker/volumes/3d5603513b758599b8695834235b3dc174379552dda002944bbbf013db2d05fc/_data",
        "Destination": "/usr/local/apache2/htdocs",
        "Driver": "local",
        "Mode": "",
        "RW": true,
        "Propagation": ""
    }
],
```

步骤 4　查看 volume 中的数据。

```
[root@localhost ~]# cd /var/lib/docker/volumes/3d5603513b758599b8695834235b3dc174379552dda002944bbbf013db2d05fc/_data
[root@localhost _data]# ls
index.html
[root@localhost _data]# cat index.html
<html><body><h1>It works!</h1></body></html>
```

步骤 5　查看容器中相应的数据。结果：容器中的数据=volume 中数据。

```
curl 127.0.0.1:8080
```

```
[root@localhost ~]# curl 127.0.0.1:8080
<html><body><h1>It works!</h1></body></html>
```

步骤 6　进入容器，更新 index.html 文件内容。

```
echo "update the index" > index.html
```

```
[root@localhost ~]# docker exec -it 0314232b0a05 bash
root@0314232b0a05:/usr/local/apache2# cd htdocs
root@0314232b0a05:/usr/local/apache2/htdocs# echo "update the index" > index.html
```

步骤 7　再次查看 volume 中的内容，已同步跟新。

```
[root@localhost ~]# cat /var/lib/docker/volumes/3d5603513b758599b8695834235b3dc174379552dda002944bbbf013db2d05fc/_data/index.html
update the index
```

由此，实现容器和 Host 的数据共享。

步骤 8　强制删除容器。

```
[root@localhost ~]# docker rm -f 0314232b0a05
```

```
[root@localhost ~]# docker rm -f 0314232b0a05
0314232b0a05
```

步骤 9　查看 volume 中的数据，依旧存在。

```
[root@localhost ~]# cat /var/lib/docker/volumes/3d5603513b758599b8695834235b3dc174379552dda002944bbbf013db2d05fc/_data/index.html
update the index
```

1.5.2 bind mount

步骤 1　在宿主机上创建/root/htdocs 目录，以只读的方式挂载给一个名为 httpd1 的 httpd 容器，映射端口 8081。

```
[root@localhost ~]# docker run --name httpd1 -d -p 8081:80 -v
/root/htdocs:/usr/local/apache2/htdocs:ro httpd
```

```
[root@localhost ~]# docker run --name httpd1 -d -p 8081:80 -v /root/htdocs:/usr/local/apache2/htdocs:ro httpd
ee885d10fea155fe31311262219030b20b4eea6e2f2f2ca28fe0b1ad5ff4c91a
```

步骤 2　查看容器挂载信息。Type=bind。

```
[root@localhost ~]# docker inspect httpd1
```

```
"Mounts": [
    {
        "Type": "bind",
        "Source": "/root/htdocs",
        "Destination": "/usr/local/apache2/htdocs",
        "Mode": "ro",
        "RW": false,
        "Propagation": "rprivate"
    }
],
```

步骤 3　在宿主机上更新 index.html 文件数据，发现 httpd1 容器中的数据也一起更新。

```
[root@localhost ~]# echo "bind mount" > /root/htdocs/index.html
```

```
[root@localhost ~]# echo "bind mount" > /root/htdocs/index.html
[root@localhost ~]# curl 127.0.0.1:8081
bind mount
```

由此，实现容器和 Host 的数据共享。

步骤 4　进入 httpd1 容器中更新 index.html 文件数据，提示 Read-only。

```
[root@localhost ~]# docker exec -it httpd1 bash
```

```
root@ee885d10fea1:/usr/local/apache2# echo "read only ?" > /usr/local/apache2/htdocs/index.html
bash: /usr/local/apache2/htdocs/index.html: Read-only file system
```

步骤 5　将宿主机/root/htdocs 挂载给名为 httpd2 的 httpd 容器，映射端口 8082，不设置 ro。

```
[root@localhost ~]# docker run --name httpd2 -d -p 8082:80 -v
/root/htdocs:/usr/local/apache2/htdocs httpd
```

```
[root@localhost ~]# docker run --name httpd2 -d -p 8082:80 -v /root/htdocs:/usr/local/apache2/htdocs httpd
8c4caa765582a3ccbdb7380a397986436458def89a805b46a6bed6b336f689c4
```

步骤 6　进入 httpd2 容器中更新 index.html 文件数据。

```
[root@localhost ~]# docker exec -it httpd2 bash
```

```
[root@localhost ~]# docker exec -it httpd2 bash
root@8c4caa765582:/usr/local/apache2# echo "i can do this" > /usr/local/apache2/htdocs/index.html
root@8c4caa765582:/usr/local/apache2# exit
```

步骤 7　分别查看宿主机、httpd1 容器、httpd2 容器中数据。三者数据一致。

```
[root@localhost ~]# cat /root/htdocs/index.html
i can do this
[root@localhost ~]# curl 127.0.0.1:8081
i can do this
[root@localhost ~]# curl 127.0.0.1:8082
i can do this
```

由此实现容器间的数据共享。

步骤 8　为方便后续实验，删除本小节中的容器。

```
docker kill
docker rm
```

1.5.3 volume container

步骤 1　创建一个名为 myvolume1 的空卷。

```
[root@localhost ~]# docker volume create myvolume1
```

```
[root@localhost ~]# docker volume create myvolume1
myvolume1
[root@localhost ~]# docker volume ls
DRIVER              VOLUME NAME
local               myvolume1
```

步骤 2　将 myvolume1 挂载一个名为 vc 的容器。

```
[root@localhost ~]# docker create --name vc -v
myvolume1:/usr/local/apache2/htdocs httpd
```

```
[root@localhost ~]# docker create --name vc -v myvolume1:/usr/local/apache2/htdocs httpd
8a084c7dd8032165beee5df8c831a7bb31569a44eab459d558457d004c40ab3f
```

步骤 3　运行两个 httpd 容器，都引用容器 vc 中的数据，分别命名为 httpd3 和 httpd4。

```
[root@localhost ~]# docker run --name httpd3 -d -p 1003:80 --volumes-from vc
httpd
[root@localhost ~]# docker run --name httpd4 -d -p 1004:80 --volumes-from vc
httpd
```

```
[root@localhost ~]# docker run --name httpd3 -d -p 1003:80 --volumes-from vc httpd
4c7cb65680aba437393f9130201375b21b72965688366ac1843a58c356e894fe
[root@localhost ~]# docker run --name httpd4 -d -p 1004:80 --volumes-from vc httpd
49c258d3fb68c26094d82462d2bfbf576a0999069f1cfb61c4806b1fa67eae61
```

步骤 4　进入 httpd3 中，更新 index.html 数据。

```
[root@localhost ~]# docker exec -it httpd3 bash
```

```
[root@localhost ~]# docker exec -it httpd3 bash
root@4c7cb65680ab:/usr/local/apache2# echo "Sharing with httpd4" > /usr/local/apache2/htdocs/index.html
root@4c7cb65680ab:/usr/local/apache2# exit
exit
```

步骤 5　查看 httpd4 容器中数据，已同步跟新。

```
[root@localhost ~]# curl 127.0.0.1:1004
```

```
[root@localhost ~]# curl 127.0.0.1:1004
Sharing with httpd4
```

步骤 6　为方便后续实验，删除本小节中的容器。

```
docker kill
docker rm
```

1.6 Namespace 和 Cgroups

1.6.1 Namespace

步骤 1　运行一个 centos 容器，并设置容器的 hostname=hwhost。

```
docker run -h hwhost -it centos
```

```
[root@localhost ~]# docker run -h hwhost -it centos
[root@hwhost /]#
```

步骤 2　在容器中添加一个用户 hwusera。

```
useradd hwusera
```

```
[root@hwhost /]# useradd hwusera
[root@hwhost /]#
[root@hwhost /]# su hwusera
[hwusera@hwhost /]$
```

步骤 3　打开另一个宿主机终端，设置宿主机 hostname=HUAWEI。

```
hostname HUAWEI
```

```
[root@localhost /]# hostname HUAWEI
[root@localhost /]# hostname
HUAWEI
[root@localhost /]# su hwusera
su: user hwusera does not exist
```

通过以上可知，容器进程的 user 空间和 hostname 空间分别由 user namespace 和 UTS namespace 隔离，独立于宿主机。

步骤 4　在容器中查看 PID，/bin/bash 的 PID=1。

```
ps axf
```

```
[hwusera@hwhost /]$ ps axf
   PID TTY      STAT   TIME COMMAND
     1 pts/0    Ss     0:00 /bin/bash
    63 pts/0    S      0:00 su hwusera
    64 pts/0    S      0:00  \_ bash
    79 pts/0    R+     0:00      \_ ps axf
```

步骤 5　在宿主机上查看该容器的 PID，/bin/bash 的 PID=141944。

```
ps axf
```

相关回显

```
175586 ?        Ssl    6:22  \_ docker-containerd --config /var/run/docker/containerd/containerd.toml
141930 ?        Sl     0:00      \_ docker-containerd-shim -namespace moby -workdir /var/lib/docker/containerd/daemon/io.containerd.runtime.v1.linux/moby/09dec586d
141944 pts/0    Ss     0:00          \_ /bin/bash
143754 pts/0    S      0:00              \_ su hwusera
143755 pts/0    S+     0:00                  \_ bash
```

通过以上可知，容器进程的 PID 空间由 PID namespace 隔离，独立于宿主机。

步骤 6　为方便后续实验，删除本小节中的容器。

```
docker kill
docker rm
```

1.6.2 CPU 资源限制

步骤 1　启动一个名为 huawei1 的压力测试容器，CPU 权重设置为 512。

```
[root@localhost ~]# docker run --name huawei1 -it --cpu-shares 512
progrium/stress --cpu 2
```

```
[root@localhost ~]# docker run --name huawei1 -it --cpu-shares 512 progrium/stress --cpu 2
stress: info: [1] dispatching hogs: 2 cpu, 0 io, 0 vm, 0 hdd
stress: dbug: [1] using backoff sleep of 6000us
stress: dbug: [1] --> hogcpu worker 2 [6] forked
stress: dbug: [1] using backoff sleep of 3000us
stress: dbug: [1] --> hogcpu worker 1 [7] forked
```

步骤 2　打开第二个宿主机终端，TOP 查看 CPU 使用率，已将近 100%。

```
top
```

```
top - 21:11:48 up 17:50,  4 users,  load average: 3.02, 3.90, 3.94
Tasks: 126 total,   3 running, 123 sleeping,   0 stopped,   0 zombie
%Cpu(s):100.0 us,  0.0 sy,  0.0 ni,  0.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
KiB Mem :  7970316 total,  7140620 free,   250616 used,   579080 buff/cache
KiB Swap:        0 total,        0 free,        0 used.  7326644 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU %MEM     TIME+ COMMAND
50250 root      20   0    7304     96      0 R  99.7  0.0  10:12.43 stress
50251 root      20   0    7304     96      0 R  99.7  0.0  10:09.74 stress
 1323 root      20   0  475828  30456  12420 S   0.3  0.4   2:40.24 docker-containe
    1 root      20   0  128044   6680   4172 S   0.0  0.1   0:31.62 systemd
```

虽然设置了权重值，但 CPU 已经到 100%。

步骤 3　打开第三个宿主机终端，启动第二个压力测试容器 huawei2，权重指设置为 2048。

```
[root@localhost ~]# docker run --name huawei2 -it --cpu-shares 2048
progrium/stress --cpu 2
```

```
[root@localhost ~]# docker run --name huawei2 -it --cpu-shares 2048 progrium/stress --cpu 2
stress: info: [1] dispatching hogs: 2 cpu, 0 io, 0 vm, 0 hdd
stress: dbug: [1] using backoff sleep of 6000us
stress: dbug: [1] --> hogcpu worker 2 [6] forked
stress: dbug: [1] using backoff sleep of 3000us
stress: dbug: [1] --> hogcpu worker 1 [7] forked
```

步骤 4　再次查看 CPU 使用率。huawei2 的 CPU 占用将近是 huawei1 的 4 倍。

```
top
```

```
top - 21:14:40 up 17:53,  4 users,  load average: 4.19, 3.85, 3.90
Tasks: 131 total,   5 running, 126 sleeping,   0 stopped,   0 zombie
%Cpu(s): 99.7 us,  0.3 sy,  0.0 ni,  0.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
KiB Mem :  7970316 total,  7120644 free,   265972 used,   583700 buff/cache
KiB Swap:        0 total,        0 free,        0 used.  7306840 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU %MEM     TIME+ COMMAND
52494 root      20   0    7304     96      0 R  80.4  0.0   1:14.39 stress
52493 root      20   0    7304     96      0 R  78.1  0.0   1:14.89 stress
50251 root      20   0    7304     96      0 R  19.3  0.0  11:44.91 stress
50250 root      20   0    7304     96      0 R  18.9  0.0  11:48.21 stress
 1323 root      20   0  475828  30456  12420 S   0.7  0.4   2:40.59 docker-containe
    1 root      20   0  128044   6680   4172 S   0.3  0.1   0:31.72 systemd
```

步骤 5　查看 CONTAINER ID。

```
docker ps
```

```
[root@localhost docker]# docker ps
CONTAINER ID        IMAGE               COMMAND                  CREATED             STATUS              PORTS               NAMES
d2fc304f3563        progrium/stress     "/usr/bin/stress --v…"   27 seconds ago      Up 25 seconds                           huawei2
6597fd9b1507        progrium/stress     "/usr/bin/stress --v…"   5 minutes ago       Up 5 minutes                            huawei1
```

步骤 6　根据 CONTAINER ID，查找 cgroup 下相应的 CPU 配置文件。此处以 huawei2 为例。

```
[root@localhost d2fc304f3563573562a7ccd3dd4d4bcb23f7c9dfede0e352427b9ff5808b12e0]# pwd
/sys/fs/cgroup/cpu/docker/d2fc304f3563573562a7ccd3dd4d4bcb23f7c9dfede0e352427b9ff5808b12e0
[root@localhost d2fc304f3563573562a7ccd3dd4d4bcb23f7c9dfede0e352427b9ff5808b12e0]# ll
total 0
-rw-r--r--. 1 root root 0 Aug 13 21:22 cgroup.clone_children
--w--w--w-. 1 root root 0 Aug 13 21:22 cgroup.event_control
-rw-r--r--. 1 root root 0 Aug 13 21:22 cgroup.procs
-r--r--r--. 1 root root 0 Aug 13 21:22 cpuacct.stat
-rw-r--r--. 1 root root 0 Aug 13 21:22 cpuacct.usage
-r--r--r--. 1 root root 0 Aug 13 21:22 cpuacct.usage_percpu
-rw-r--r--. 1 root root 0 Aug 13 21:22 cpu.cfs_period_us
-rw-r--r--. 1 root root 0 Aug 13 21:22 cpu.cfs_quota_us
-rw-r--r--. 1 root root 0 Aug 13 21:22 cpu.rt_period_us
-rw-r--r--. 1 root root 0 Aug 13 21:22 cpu.rt_runtime_us
-rw-r--r--. 1 root root 0 Aug 13 21:22 cpu.shares
-r--r--r--. 1 root root 0 Aug 13 21:22 cpu.stat
-rw-r--r--. 1 root root 0 Aug 13 21:22 notify_on_release
-rw-r--r--. 1 root root 0 Aug 13 21:22 tasks
```

步骤 7　查看 cpu.shares 值和 tasks 值。cpu.shares=2048，tasks 值即 huawei2 在宿主机上的 PID。

```
cat cpu.shares
cat tasks
```

```
[root@localhost d2fc304f3563573562a7ccd3dd4d4bcb23f7c9dfede0e352427b9ff5808b12e0]# cat cpu.shares
2048
[root@localhost d2fc304f3563573562a7ccd3dd4d4bcb23f7c9dfede0e352427b9ff5808b12e0]# cat tasks
217401
217429
```

步骤 8　为方便后续实验，删除本小节中的容器。

```
docker kill
docker rm
```

1.6.3 内存资源限制

步骤 1 在后台启动一个 centos 容器，并限制其最多使用 400M 内存和 100M swap。

```
docker run -m 400M --memory-swap=500M -dit centos /bin/bash
```

```
[root@localhost ~]# docker run -m 400M --memory-swap=500M -dit centos /bin/bash
515da3dcef3ced12e714f66b3224f1f73588ab0b8eaa630166e8e6b955608911
```

步骤 2 找到该容器在 cgroup 下内存配置目录。

```
cd /sys/fs/cgroup/memory/docker/
```

```
[root@localhost ~]# cd /sys/fs/cgroup/memory/docker/
[root@localhost docker]# ll
total 0
drwxr-xr-x. 2 root root 0 Aug 12 22:10 507dd38c109086aa6a0c384bfbed722ff7e8e9fa1a0e9559ed4d2aee36268bbc
drwxr-xr-x. 2 root root 0 Aug 12 22:42 515da3dcef3ced12e714f66b3224f1f73588ab0b8eaa630166e8e6b955608911
drwxr-xr-x. 2 root root 0 Aug 12 22:25 5d88c92d5de9d601f77a69ebf219530d6127d78ba9fd73bb3c8dc1a62a52e9d2
drwxr-xr-x. 2 root root 0 Aug 12 22:25 b0a337e7370b203ec28cdd7c3ad49c133d15a3dd8a2a81d4041af196f6f1b2a1
drwxr-xr-x. 2 root root 0 Aug 12 22:25 c46a346e516ec03f11786a639efafef50ef5c7ee642ac628dca7f788568df504
-rw-r--r--. 1 root root 0 Jul 10 09:14 cgroup.clone_children
--w--w--w-. 1 root root 0 Jul 10 09:14 cgroup.event_control
```

步骤 3 进入该容器的内存配置目录，查看内存配置文件。查看到内存使用限制为 400M，内存和 SWAP 资源为 500M。

```
cat memory.limit_in_bytes
```

```
[root@localhost docker]# cd 515da3dcef3ced12e714f66b3224f1f73588ab0b8eaa630166e8e6b955608911
[root@localhost 515da3dcef3ced12e714f66b3224f1f73588ab0b8eaa630166e8e6b955608911]# cat memory.limit_in_bytes
419430400
[root@localhost 515da3dcef3ced12e714f66b3224f1f73588ab0b8eaa630166e8e6b955608911]#
```

```
cat memory.memsw.limit_in_bytes
```

```
[root@localhost 515da3dcef3ced12e714f66b3224f1f73588ab0b8eaa630166e8e6b955608911]# cat memory.memsw.limit_in_bytes
524288000
```

步骤 4 为方便后续实验，删除本小节中的容器。

```
docker kill
docker rm
```

1.6.4 实训任务

步骤 1 后台启动一个压力测试容器，使其占用 100%的 CPU 资源。

步骤 2 进入该容器的 cgroup 目录中的 CPU 子系统，配置其最大只能使用 20%的 CPU 资源。

步骤 3 在不修改前 3 个步骤中所涉及的任何配置参数情况下，设法使该容器的 CPU 占用率再次达到 100%。

2 Kubernetes 实验

2.1 实验介绍

2.1.1 关于本实验

本实验通过 18 个模块的 Kubernetes 实验练习和实训任务演练，让学员系统性学习 Kubernetes 平台，掌握其运维操作、故障排查技能。

2.1.2 实验目的

- 描述 Kubernetes 架构和组件，掌握其基本操作
- 掌握 Kubernetes 常用对象的功能作用和操作配置
- 掌握 Kubernetes 平台管理运维、故障处理能力

2.2 Kubernetes 组件

2.2.1 配置步骤

步骤 1 查看节点状态：

```
[root@k8s-master /]# kubectl get node
```

```
NAME         STATUS   ROLES    AGE   VERSION
k8s-master   Ready    master   9d    v1.14.3
k8s-node1    Ready    <none>   9d    v1.14.3
k8s-node2    Ready    <none>   9d    v1.14.3
```

步骤 2 查看 Docker 运行状态

登录 k8s-master 节点和 node 节点，使用如下命令查看 docker 状态：

```
[root@k8s-node1]# systemctl status docker
```

如 docker 运行状态正常，回显如下：

```
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; vendor
preset: disabled)
   Active: active (running) since Sun 2019-04-28 18:00:36 CST; 2 weeks 3
days ago
```

步骤 3　在所有节点查看 kubelet 运行状态

使用如下命令：

```
[root@k8s-node1]# systemctl status kubelet
```

```
● kubelet.service - kubelet: The Kubernetes Node Agent
   Loaded: loaded (/usr/lib/systemd/system/kubelet.service; enabled; vendor
preset: disabled)
  Drop-In: /usr/lib/systemd/system/kubelet.service.d
           └─10-kubeadm.conf
   Active: active (running) since Sun 2019-04-28 17:15:03 CST; 2 weeks 3
days ago
```

步骤 4　在 Master 节点查看封装到 Pod 中的组件的状态。

```
[root@k8s-master]# kubectl get pods --field-selector spec.nodeName=k8s-master -
-namespace=kube-system
```

回显可以看到：

```
NAME                                 READY   STATUS    RESTARTS   AGE
coredns-fb8b8dccf-j4kn5              1/1     Running   3          18d
coredns-fb8b8dccf-mx6pw              1/1     Running   3          18d
etcd-k8s-master                      1/1     Running   2          18d
kube-apiserver-k8s-master            1/1     Running   3          18d
kube-controller-manager-k8s-master   1/1     Running   2          18d
kube-flannel-ds-amd64-lfslh          1/1     Running   3          18d
kube-proxy-dt69s                     1/1     Running   2          18d
kube-scheduler-k8s-master            1/1     Running   2          18d
kubernetes-dashboard-5f798cc594      1/1     Running   3          18d
```

步骤 5　查看 Node 节点上封装到 Pod 中的系统组件：

```
[root@k8s-master]# kubectl get pods --field-selector spec.nodeName=k8s-node2 --
namespace=kube-system
```

回显如下

```
NAME                          READY   STATUS    RESTARTS   AGE
kube-flannel-ds-amd64-7tn6g   1/1     Running   2          18d
kube-proxy-q8xmm              1/1     Running   2          18d
```

步骤 6　查看现有 namespace

```
[root@k8s-master]kubectl get namespace
```

或者简写命令

```
[root@k8s-master]kubectl get ns
```

或查看完整信息

```
[root@k8s-master]# kubectl get namespace -o wide
```

```
NAME              STATUS   AGE
default           Active   9d
kube-node-lease   Active   9d
kube-public       Active   9d
kube-system       Active   9d
```

步骤 7　创建 namespace

```
[root@k8s-master /]# kubectl create namespace new-namespace
```

```
namespace/new-namespace created
```

步骤 8　再次查看 namespace 清单

```
[root@k8s-master]kubectl get namespace
```

可以看到 namespace 中的可以看到已创建的 namespace

```
NAME              STATUS   AGE
default           Active   9d
kube-node-lease   Active   9d
kube-public       Active   9d
kube-system       Active   9d
new-namespace     Active   21m
```

步骤 9　删除刚刚创建的 namespace

```
[root@k8s-master]kubectl delete namespace new-namespace
```

2.3 Deployment 实验

2.3.1 运行第一个 Deployment

步骤 1　输入命令创建一个 Deployment

```
[root@k8s-master /]# kubectl create deployment mydep  --image=nginx
```

步骤 2　查看 Deployment

```
[root@k8s-master /]# kubectl get deployment
```

或者

```
[root@k8s-master /]# kubectl get deploy
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
mydep   1/1     1            1           65s
```

2.3.2 使用 kubectl 命令行工具

步骤 1　使用 kubectl get 命令。

get all 可以查看所有对象。

```
[root@k8s-master /]# kubectl get all
```

查看指定 namespace 的对象：

```
[root@k8s-master /]# kubectl get all --namespace=kube-system
```

或者

```
[root@k8s-master /]# kubectl get all -n kube-system
```

以交互模式观察对象的变化：

```
[root@k8s-master /]# kubectl get deployment --watch
```

或者

```
[root@k8s-master /]# kubectl get deployment -w
```

按 ctrl+c 退出交互模式

查看更为完整的对象信息：

```
[root@k8s-master /]# kubectl get deployment -o wide
```

步骤 2　使用 kubectl help 命令

使用如下命令查看 kubectl 中可用的[COMMAND]

```
[root@k8s-master /]# kubectl --help
```

使用如下命令查看 kubectl 中 get 命令的详细说明，使用样例：

```
[root@k8s-master /]# kubectl get --help
```

步骤 3　使用 kubectl describe 命令查看资源对象的详细信息

```
[root@k8s-master /]# kubectl describe deployment mydep
```

回显示例

```
Name:                   mydep
```

```
Namespace:              default
CreationTimestamp:      Fri, 28 Jun 2019 14:17:51 +0800
Labels:                 app=mydep
Annotations:            deployment.kubernetes.io/revision: 1
Selector:               app=mydep
Replicas:               1 desired | 1 updated | 1 total | 1 available | 0
unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=mydep
  Containers:
   nginx:
    Image:        nginx
    Port:         <none>
    Host Port:    <none>
    Environment:  <none>
    Mounts:       <none>
  Volumes:        <none>
Conditions:
  Type          Status  Reason
  ----          ------  ------
  Available     True    MinimumReplicasAvailable
  Progressing   True    NewReplicaSetAvailable
OldReplicaSets:  <none>
NewReplicaSet:   mydep-65bbdb4c9f (1/1 replicas created)
Events:
  Type    Reason             Age   From                   Message
  ----    ------             ----  ----                   -------
  Normal  ScalingReplicaSet  34m   deployment-controller  Scaled up replica
set mydep-65bbdb4c9f to 1
```

2.3.3 创建 Deployment

步骤 1　在 master 节点创建/labfile/deployfile 目录，用于保存配置文件。后续创建 deployment 的 yaml 文件保存在此处。

```
[root@k8s-master /]# cd /
[root@k8s-master /]# mkdir labfile
[root@k8s-master /]# cd labfile/
[root@k8s-master labfile]# mkdir deployfile
[root@k8s-master labfile]# cd deployfile/
```

步骤 2　创建 deployment 文件

```
[root@k8s-master deployfile]# vim nginx-deployment.yaml
```

文件内容如下

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.7.9
        ports:
        - containerPort: 80
```

步骤 3 部署该 nginx-deployment

```
[root@k8s-master deployfile]# kubectl apply -f nginx-deployment.yaml
```

步骤 4 查看创建结果

```
[root@k8s-master deployfile]# kubectl describe deployment
[root@k8s-master deployfile]# kubectl get deployment
```

步骤 5 查看 replicaset

```
[root@k8s-master deployfile]# kubectl get replicaset
NAME                          DESIRED   CURRENT   READY   AGE
nginx-deployment-6dd86d77d    3         3         3       33m
```

步骤 6 查看 Pod（等待约 2min，才会显示 Running 状态）

```
[root@k8s-master deployfile]# kubectl get pod
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-6dd86d77d-cl2wh    1/1     Running   0          34m
nginx-deployment-6dd86d77d-llnb8    1/1     Running   0          34m
nginx-deployment-6dd86d77d-sjmmd    1/1     Running   0          34m
```

2.3.4 弹性伸缩 Deployment

步骤 1 编辑之前创建的 nginx-deployment.yaml 文件，将副本数量“replicas”改为 5。

```
[root@k8s-master deployfile]# vim nginx-deployment.yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 5
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.7.9
        ports:
        - containerPort: 80
```

步骤 2 应用变更后的文件 yaml 文件。

```
[root@k8s-master deployfile]# kubectl apply -f nginx-deployment.yaml
```

步骤 3 查看 pod 状态。

```
[root@k8s-master deployfile]# kubectl get pod
```

确认 Pod 数量是否正确。

2.3.5 滚动升级 deployment

步骤 1 复制 nginx-deployment.yaml 文件为两个新版本。

```
[root@k8s-master deployfile]# cp nginx-deployment.yaml nginx-deployment-v2.yaml
[root@k8s-master deployfile]# cp nginx-deployment.yaml nginx-deployment-v3.yaml
```

步骤 2 修改 v2 和 v3 的 yaml 文件中镜像版本为 1.8.1 和 1.9.1

```
[root@k8s-master deployfile]# vim nginx-deployment-v2.yaml
```

```
……
```

```
spec:
      containers:
      - name: nginx
        image: nginx:1.8.1
......
[root@k8s-master deployfile]# vim nginx-deployment-v3.yaml
……
spec:
      containers:
      - name: nginx
        image: nginx:1.9.1
……
```

步骤 3　进行滚动更新

```
[root@k8s-master deployfile]# kubectl apply -f nginx-deployment-v2.yaml --
record
```

使用 get 命令查看更新状态（等待 10min 左右，能看到如图状态）

```
[root@k8s-master deployfile]# kubectl get deployment -o wide
```

可以看到镜像版本已变为 1.8.1

```
NAME               READY   UP-TO-DATE   AVAILABLE   AGE   CONTAINERS   IMAGES         SELECTOR
nginx-deployment   5/5     5            5           94m   nginx        nginx:1.8.1    app=nginx
```

查看 replicaset，可以发现出现了一个新的 replicaset，里面有 5 个 Pod，而原有的 replicaset 中没有 pod 存在。

```
[root@k8s-master deployfile]# kubectl get replicaset
```

```
NAME                          DEIRED   CURRENT   READY   AGE
nginx-deployment-59988f74c7   0        0         0       17m
nginx-deployment-784b7cc96d   5        5         5       5m53s
```

查看 deployment 更新事件：

```
[root@k8s-master deployfile]# kubectl describe deployment
```

```
Events:
  Type    Reason             Age                    From                   Message
  ----    ------             ----                   ----                   -------
  Normal  ScalingReplicaSet  20m (x2 over 67m)      deployment-controller  Scaled up replica set nginx-deployment-6dd86d77d to 5
  Normal  ScalingReplicaSet  20m                    deployment-controller  Scaled up replica set nginx-deployment-59988f74c7 to 2
  Normal  ScalingReplicaSet  20m                    deployment-controller  Scaled down replica set nginx-deployment-6dd86d77d to 4
  Normal  ScalingReplicaSet  20m                    deployment-controller  Scaled up replica set nginx-deployment-59988f74c7 to 3
  Normal  ScalingReplicaSet  17m (x2 over 63m)      deployment-controller  Scaled down replica set nginx-deployment-6dd86d77d to 3
  Normal  ScalingReplicaSet  17m                    deployment-controller  Scaled up replica set nginx-deployment-59988f74c7 to 4
  Normal  ScalingReplicaSet  17m                    deployment-controller  Scaled down replica set nginx-deployment-6dd86d77d to 2
  Normal  ScalingReplicaSet  17m                    deployment-controller  Scaled up replica set nginx-deployment-59988f74c7 to 5
  Normal  ScalingReplicaSet  17m                    deployment-controller  Scaled down replica set nginx-deployment-6dd86d77d to 1
  Normal  ScalingReplicaSet  8m44s (x10 over 17m)  deployment-controller  (combined from similar events): Scaled down replica set
```

更新到 v3 版本

```
[root@k8s-master deployfile]# kubectl apply -f nginx-deployment-v3.yaml --
record
```

步骤 4　查看 deployment 的更新记录

```
[root@k8s-master deployfile]# kubectl rollout history deployment nginx-
deployment
deployment.extensions/nginx-deployment
REVISION  CHANGE-CAUSE
1         <none>
2         kubectl apply --filename=nginx-deployment-v2.yaml --record=true
3         kubectl apply --filename=nginx-deployment-v3.yaml --record=true
```

步骤 5　查看历史版本 2 的详细信息

```
[root@k8s-master deployfile]# kubectl rollout history deployment nginx-
deployment --revision=2
deployment.extensions/nginx-deployment with revision #2
Pod Template:
  Labels:       app=nginx
        pod-template-hash=59988f74c7
  Annotations:  kubernetes.io/change-cause: kubectl apply --filename=nginx-
deployment-v2.yaml --record=true
  Containers:
   nginx:
    Image:      nginx:1.8.1
    Port:       80/TCP
    Host Port:  0/TCP
    Environment:        <none>
    Mounts:     <none>
  Volumes:      <none>
```

步骤 6　回滚到历史版本 2

```
[root@k8s-master deployfile]# kubectl rollout undo deployment nginx-deployment
--to-revision=2
```

查看当前 deployment 版本信息

```
[root@k8s-master deployfile]# kubectl get deployment -o wide
NAME               READY   UP-TO-DATE   AVAILABLE   AGE    CONTAINERS   IMAGES         SELECTOR
nginx-deployment   5/5     5            5           128m   nginx        nginx:1.8.1    app=nginx
```

步骤 7　删除 deployment

```
[root@k8s-master podfile]# kubectl delete deployment nginx-deployment
```

2.3.6 实训任务

步骤 1　使用 kubectl 命令查看 kube-system 中所有 deployment 的信息，包括每个 deployment 中存在几个 Pod。

步骤 2　通过 yaml 文件创建一个 deployment，有如下要求

1. 使用 httpd：2.4
2. 4 副本

步骤 3　将上一步创建的 deployment 删除一个 pod，变成 3 副本 deployment

步骤 4　升级该 deployment 的镜像版本至 latest。

步骤 5　寻找该 deployment 中各个 pod 运行节点，deployment 创建的时间戳（Creation Timestamp）。

2.4 Pod 实验

2.4.1 使用 Pod

步骤 1　创建 podfile 目录

```
[root@k8s-master]# mkdir /labfile/podfile
[root@k8s-master]# cd /labfile/podfile
```

步骤 2　编写 mypod.yaml

```
[root@k8s-master podfile]# vim mypod.yaml
```

```
kind: Pod
apiVersion: v1
metadata:
  name: mypod
spec:
  containers:
    - name: mypod
      image: busybox
      args:
      - /bin/sh
      - -c
      - sleep 30000
```

步骤 3　创建 pod

```
[root@k8s-master podfile]# kubectl apply -f mypod.yaml
pod/mypod created
```

步骤 4　查看 pod

```
[root@k8s-master podfile]# kubectl get pod
NAME    READY   STATUS    RESTARTS   AGE
mypod   1/1     Running   0          72s
```

步骤 5　查看 yaml 输出格式的 pod 信息

```
[root@k8s-master podfile]# kubectl get pod mypod -o yaml
apiVersion: v1
kind: Pod
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |

{"apiVersion":"v1","kind":"Pod","metadata":{"annotations":{},"name":"mypod","na
mespace":"default"},"spec":{"containers":[{"args":["/bin/sh","-c","sleep
30000"],"image":"busybox","name":"mypod"}]}}
  creationTimestamp: "2019-06-28T11:01:30Z"
  name: mypod
  namespace: default
  resourceVersion: "1258931"
  selfLink: /api/v1/namespaces/default/pods/mypod
  uid: 15241cde-9994-11e9-ac91-000c290a92ce
spec:
  containers:
  - args:
    - /bin/sh
    - -c
    - sleep 30000
    image: busybox
    imagePullPolicy: Always
    name: mypod
……
```

可以看到，回显的信息比我们创建的 yaml 文件丰富许多，因为许多我们未在 yaml 中定义的参数系统使用默认值配置。

步骤 6　使用 explain 命令

如果我们对 yaml 中的某些参数的值意味什么意思，该如何填写没有把握，可以使用 explain 命令。

如果希望了解关于 pod 中 apiVersion 的相关信息：

```
[root@k8s-master podfile]# kubectl explain pod.apiVersion
KIND:     Pod
VERSION:  v1
```

```
FIELD:    apiVersion <string>

DESCRIPTION:
    APIVersion defines the versioned schema of this representation of an
    object. Servers should convert recognized schemas to the latest internal
    value, and may reject unrecognized values. More info:
    https://git.k8s.io/community/contributors/devel/api-
conventions.md#resources
```

如果希望了解关于 Pod 中，spec 里，container 下 imagePullPolicy 的定义：

```
[root@k8s-master podfile]# kubectl explain pod.spec.containers.imagePullPolicy
KIND:     Pod
VERSION:  v1

FIELD:    imagePullPolicy <string>

DESCRIPTION:
    Image pull policy. One of Always, Never, IfNotPresent. Defaults to Always
    if :latest tag is specified, or IfNotPresent otherwise. Cannot be
updated.
    More info:
    https://kubernetes.io/docs/concepts/containers/images#updating-images
```

步骤 7　进入 mypod 容器的命令行交互界面

```
[root@k8s-master podfile]# kubectl exec -it mypod /bin/sh
/ #
```

使用 ls 命令查看容器内目录

```
/ # ls
bin   dev   etc   home  proc  root  sys   tmp   usr   var
```

步骤 8　退出容器

在容器内输入 exit 可以退出容器

```
/ # exit
```

步骤 9　查看容器运行位置

```
[root@k8s-master podfile]# kubectl get pod -o wide
NAME    READY   STATUS    RESTARTS   AGE   IP           NODE        NOMINATED
mypod   1/1     Running   0          15m   10.244.2.74  k8s-node1   <none>
```

步骤 10 可以看到容器运行位置位于 k8s-node1，切换到主机 k8s-node1 中，使用 docker 命令查看运行在该主机上的容器。通过容器的名称找到 mypod 中的容器。

```
[root@k8s-node1 hostpathdir]# docker ps
CONTAINER ID        IMAGE                  COMMAND                  CREATED
STATUS              PORTS               NAMES
b9de9f770a08        busybox                "/bin/sh -c 'sleep 3…"   16 minutes
ago      Up 16 minutes
k8s_mypod_mypod_default_15241cde-9994-11e9-ac91-000c290a92ce_0
11edb1e081a8        k8s.gcr.io/pause:3.1   "/pause"                 16 minutes
ago      Up 16 minutes
k8s_POD_mypod_default_15241cde-9994-11e9-ac91-000c290a92ce_0
```

2.4.2 运行一次性 Pod

步骤 1 创建 helloworld.yaml 文件

```
[root@k8s-master podfile]# vim helloworld.yaml
```

```
kind: Pod
apiVersion: v1
metadata:
  name: helloworld
spec:
  restartPolicy: Never
  containers:
    - name: helloworld
      image: hello-world
```

步骤 2 从 helloworld.yaml 创建 Pod

```
[root@k8s-master podfile]# kubectl apply -f helloworld.yaml
```

步骤 3 查看 pod 状态

```
[root@k8s-master podfile]# kubectl get pod
```

```
NAME         READY   STATUS      RESTARTS   AGE
helloworld   0/1     Completed   0          2s
```

步骤 4 查看 pod 输出信息

```
[root@k8s-master podfile]# kubectl logs helloworld
```

出现如下信息表示成功

```
Hello from Docker!
```

```
This message shows that your installation appears to be working correctly.
```

步骤 5　删除本次实验创建的 pod

2.4.3 实训任务

步骤 1　创建一个 Pod，包含两个容器。

1. 容器 1 为 busybox 进行
2. 容器 2 为 httpd 镜像

步骤 2　进入 busybox 容器命令行界面。在 home 界面下使用如下命令写入一个文件

```
]# cat > test.txt << EOF
> i am busybox
> EOF
```

2.5 Label 与 Label Selector

2.5.1 标签

步骤 1　创建目录用于保存本次实验文件

```
[root@k8s-master labfile]# mkdir /labfile/labelfile
[root@k8s-master labfile]# cd /labfile/labelfile
```

步骤 2　创建 labelpod.yaml 并使用多个标签

```
[root@k8s-master labelfile]# vim labelpod.yaml
```

```
kind: Pod
apiVersion: v1
metadata:
  name: labelpod
  labels:
    app: busybox
    version: new
spec:
  containers:
    - name: labelpod
```

```
    image: busybox
    args:
    - /bin/sh
    - -c
    - sleep 30000
```

步骤 3　创建 Pod

```
[root@k8s-master labelfile]# kubectl apply -f labelpod.yaml
```

步骤 4　查看 pod 的 label

```
[root@k8s-master labelfile]# kubectl get pod --show-labels
NAME       READY   STATUS    RESTARTS   AGE     LABELS
labelpod   1/1     Running   0          2m19s   app=busybox,version=new
```

步骤 5　为 labelpod 添加一个新的 label

```
[root@k8s-master labelfile]# kubectl label pod labelpod time=2019
```

再次查看 pod 的 label

```
[root@k8s-master labelfile]# kubectl get pod --show-labels
NAME       READY   STATUS    RESTARTS   AGE   LABELS
labelpod   1/1     Running   0          24m   app=busybox,time=2019,version=new
```

2.5.2 标签选择器

步骤 1　创建 labelpod2.yaml

```
[root@k8s-master labelfile]# vim labelpod2.yaml
```

```
kind: Pod
apiVersion: v1
metadata:
  name: labelpod2
  labels:
    app: httpd
    version: new
spec:
  containers:
    - name: httpd
      image: httpd
```

步骤 2　创建并查看新创建的 labelpod2

```
[root@k8s-master labelfile]# kubectl apply -f labelpod2.yaml
[root@k8s-master labelfile]# kubectl get pod --show-labels
```

```
NAME        READY   STATUS    RESTARTS   AGE   LABELS
labelpod    1/1     Running   0          31m   app=busybox,time=2019,version=new
labelpod2   1/1     Running   0          19s   app=httpd,version=new
```

步骤 3　使用基于等值的标签选择器

```
[root@k8s-master labelfile]# kubectl get pod -l app=httpd 或者
[root@k8s-master labelfile]# kubectl get pod -l app==httpd
NAME        READY   STATUS    RESTARTS   AGE
labelpod2   1/1     Running   0          5m5s
```

步骤 4　使用基于不等值的标签选择器

```
[root@k8s-master labelfile]# kubectl get pod -l app!=httpd
NAME       READY   STATUS    RESTARTS   AGE
labelpod   1/1     Running   0          74m
```

步骤 5　查看各 pod 针对某标签键的值

```
[root@k8s-master labelfile]# kubectl get pod -L app
NAME        READY   STATUS    RESTARTS   AGE   APP
labelpod    1/1     Running   0          95m   busybox
labelpod2   1/1     Running   0          64m   httpd
```

2.5.3 使用标签选择器实现调度

步骤 1　将节点 2 打上标签

```
[root@k8s-master labelfile]# kubectl label node k8s-node2 env=test
node/k8s-node2 labeled
```

步骤 2　查看节点上的标签

```
[root@k8s-master labelfile]# kubectl get node -L env
NAME         STATUS   ROLES    AGE   VERSION   ENV
k8s-master   Ready    master   10d   v1.14.3
k8s-node1    Ready    <none>   10d   v1.14.3
k8s-node2    Ready    <none>   10d   v1.14.3   test
```

步骤 3　使用 nodeselector 实现调度，创建 yaml 文件

```
[root@k8s-master labelfile]# vim nsdeploy.yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.7.9
        ports:
        - containerPort: 80
      nodeSelector:
        env: test
```

步骤 4　创建 deployment

```
[root@k8s-master labelfile]# kubectl apply -f nsdeploy.yaml
```

```
deployment.apps/nginx-deployment created
```

步骤 5　查看 deployment 中 Pod 的位置

```
[root@k8s-master labelfile]# kubectl get pod -o wide
```

```
NAME                                READY   STATUS    RESTARTS   AGE     IP            NODE
nginx-deployment-dfdf84446-45bs5    1/1     Running   0          6m49s   10.244.1.49   k8s-node2
nginx-deployment-dfdf84446-bts48    1/1     Running   0          6m49s   10.244.1.48   k8s-node2
nginx-deployment-dfdf84446-16fg5    1/1     Running   0          6m49s   10.244.1.50   k8s-node2
```

步骤 6　使用 node affinity 调度，创建一个新的 yaml 文件 nadeploy2.yaml

```
[root@k8s-master labelfile]# vim nadeploy2.yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: httpd-deployment
```

```
  labels:
    app: httpd
spec:
  replicas: 3
  selector:
    matchLabels:
      app: httpd
  template:
    metadata:
      labels:
        app: httpd
    spec:
      containers:
      - name: httpd
        image: httpd
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
            - matchExpressions:
              - key: env
                operator: In
                values:
                - test
```

步骤 7　创建 deployment

```
[root@k8s-master labelfile]# kubectl apply -f nadeploy2.yaml
```

步骤 8　查看新建的 deployment 中 pod 的位置，可以看到，3 个 pod 全在 node2 上

```
[root@k8s-master labelfile]# kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
httpd-deployment-6d49f8d56d-8n6vr	1/1	Running	0	8m41s	10.244.1.51	k8s-node2
httpd-deployment-6d49f8d56d-tg69j	1/1	Running	0	8m41s	10.244.1.53	k8s-node2
httpd-deployment-6d49f8d56d-wzgrh	1/1	Running	0	8m41s	10.244.1.52	k8s-node2
nginx-deployment-dfdf84446-45bs5	1/1	Running	0	28m	10.244.1.49	k8s-node2
nginx-deployment-dfdf84446-bts48	1/1	Running	0	28m	10.244.1.48	k8s-node2
nginx-deployment-dfdf84446-16fg5	1/1	Running	0	28m	10.244.1.50	k8s-node2

2.5.4 实训任务

步骤 1　创建一个 deployment，要求如下

1. 使用镜像为 nginx，5 副本
2. deployment 中的 pod 不能出现在 node1 上

步骤 2　寻找一种方式可以搜索出 kubernetes 系统上提供 core-dns,kubeproxy 以及 dashboard 服务的 pod

步骤 3　使用标签和标签选择器，使用一条命令批量删除 node2 节点上的 nginx-deployment pod。（deployment 会自动恢复 pod，只需要批量删除一次 pod 即可）

2.6 Service 服务发现

2.6.1 使用 Service

步骤 1　创建 Service 实验使用的目录

```
[root@k8s-master labelfile]# mkdir /labfile/servicefile
[root@k8s-master labelfile]# cd /labfile/servicefile/
```

步骤 2　创建后端 httpd Deployment

```
[root@k8s-master servicefile]# vim httpd-deploy.yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: httpd
spec:
  replicas: 3
  selector:
    matchLabels:
      app: httpd
  template:
    metadata:
      labels:
        app: httpd
    spec:
      containers:
      - name: httpd
        image: httpd
        ports:
        - containerPort: 80
```

步骤 3　部署 deployment

```
[root@k8s-master servicefile]# kubectl apply -f httpd-deploy.yaml
deployment.apps/httpd created
```

查看

```
[root@k8s-master servicefile]# kubectl get deployment
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
httpd   3/3     3            3           3m29s
```

步骤 4 创建 httpd-service.yaml 文件

```
[root@k8s-master servicefile]# vim httpd-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: httpd-svc
spec:
  selector:
    app: httpd
  ports:
  - protocol: TCP
    port: 8080
    targetPort: 80
```

步骤 5 创建 service

```
[root@k8s-master servicefile]# kubectl apply -f httpd-service.yaml
service/httpd-svc created
```

步骤 6 查看 service 信息，注意！千万不可删除名称为 kubernetes 的服务！该服务为系统服务！

```
[root@k8s-master servicefile]# kubectl get service
NAME         TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)    AGE
httpd-svc    ClusterIP   10.96.116.135   <none>        8080/TCP   53s
kubernetes   ClusterIP   10.96.0.1       <none>        443/TCP    10d
```

步骤 7 查看 service 中的 endpoints

```
[root@k8s-master servicefile]# kubectl get endpoints
NAME         ENDPOINTS                                            AGE
httpd-svc    10.244.0.14:80,10.244.1.57:80,10.244.2.79:80         3m48s
kubernetes   192.168.137.11:6443                                  10d
```

步骤 8 测试服务可用性，使用 curl <服务 IP：端口号>查看回显，显示“It works！”表示成功。服务 IP 和端口号为步骤 6 中获取的值。

```
[root@k8s-master servicefile]# curl 10.96.116.135:8080
<html><body><h1>It works!</h1></body></html>
```

步骤 9　删除刚刚创建的服务，注意！千万不要删除 kubernetes 服务！

```
[root@k8s-master servicefile]# kubectl delete service httpd-svc
service "httpd-svc" deleted
```

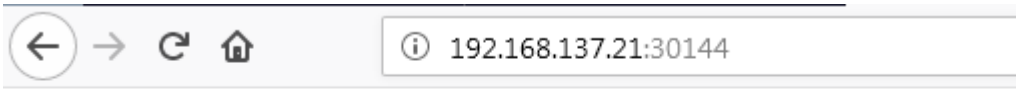
步骤 10　创建 httpd-expose.yaml

```
[root@k8s-master servicefile]# vim httpd-expose.yaml
apiVersion: v1
kind: Service
metadata:
  name: httpd-svc
spec:
  type: NodePort
  selector:
    app: httpd
  ports:
  - protocol: TCP
    port: 8080
    targetPort: 80
    nodePort: 30144
```

步骤 11　创建 httpd 服务。

```
[root@k8s-master servicefile]# kubectl apply -f httpd-expose.yaml
service/httpd-svc created
```

步骤 12　使用跳板机浏览器登录<节点 IP 地址：端口>，显示“It works！”表示成功



It works!

2.6.2 使用 DNS

步骤 1　创建 client.yaml，用于创建一个客户端 Pod，测试 DNS 功能。

```
[root@k8s-master servicefile]# vim client.yaml
apiVersion: v1
```

```
kind: Pod
metadata:
  name: clientpod
spec:
  containers:
    - name: clientpod
      image: busybox:1.28.3
      args:
      - /bin/sh
      - -c
      - sleep 30000
```

步骤 2　创建并进入 Pod 命令行界面

```
[root@k8s-master servicefile]# kubectl apply -f client.yaml
pod/clientpod created

 [root@k8s-master servicefile]# kubectl exec -it clientpod /bin/sh
/ #
```

步骤 3　使用 nslookup 命令查看服务域名

```
/ # nslookup 10.105.48.161
Server:    10.96.0.10
Address 1: 10.96.0.10 kube-dns.kube-system.svc.cluster.local

Name:      10.105.48.161
Address 1: 10.105.48.161 httpd-svc.default.svc.cluster.local
```

步骤 4　使用 wget 命令通过域名访问服务

```
/ # wget httpd-svc:8080
Connecting to httpd-svc:8080 (10.105.48.161:8080)
index.html           100% |****************************************|    45
0:00:00 ETA
```

验证成功后删除 index 文件并退出容器

```
/ # rm index.html
/ # exit
```

2.6.3 使用 Headless Service

步骤 1　删除 httpd service，注意！不可删除 kubernetes 服务！

步骤 2　创建 headless 服务的 yaml 文件

```
[root@k8s-master servicefile]# vim headless-svc.yaml
apiVersion: v1
kind: Service
metadata:
  name: headless-svc
spec:
  selector:
    app: httpd
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
  clusterIP: None
```

步骤 3　创建 headless 服务，可以看到 headless 服务没有 IP

```
[root@k8s-master servicefile]# kubectl apply -f headless-svc.yaml
service/headless-svc created
[root@k8s-master servicefile]# kubectl get service
NAME           TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
headless-svc   ClusterIP   None         <none>        80/TCP    25s
```

步骤 4　进入 client Pod，查看 DNS 解析

```
[root@k8s-master servicefile]# kubectl exec -it clientpod /bin/sh
/ # nslookup headless-svc
Server:    10.96.0.10
Address 1: 10.96.0.10 kube-dns.kube-system.svc.cluster.local

Name:      headless-svc
Address 1: 10.244.0.30 10-244-0-30.headless-svc.default.svc.cluster.local
Address 2: 10.244.2.130 10-244-2-130.headless-svc.default.svc.cluster.local
Address 3: 10.244.1.103 10-244-1-103.headless-svc.default.svc.cluster.local
```

步骤 5　访问服务，可以看到本次访问的服务由上一步中 address3 的 pod 提供。

```
/ # wget headless-svc
Connecting to headless-svc (10.244.1.103:80)
index.html           100% |****************************************|    45
0:00:00 ETA

/ # cat index.html
<html><body><h1>It works!</h1></body></html>
```

exit 退出

2.6.4 实训任务

步骤 1 创建 deployment1，要求：

1. 2 副本，镜像类型 httpd

步骤 2 创建 deployment2，要求：

1. 3 副本，镜像类型 httpd

步骤 3 创建 service1，service1 后端为 deployment1 和 deployment2 中所有 pod。

步骤 4 创建 service2，service2 后端为 deployment1 中的第一个 pod 和 deployment2 中的第一个 pod。

步骤 5 在 clientpod 中使用域名访问 service2 中第一个 pod（wget 命令）。

步骤 6 删除本次实验创建的 pod，deployment 和 service，注意！千万不要删除 kubernetes 服务！

2.7 DaemonSet 与 Job

2.7.1 使用 DaemonSet

步骤 1 创建文件夹，用于保存 DaemonSet 和 Job 文件。

```
[root@k8s-master ~]# mkdir /labfile/djfile
[root@k8s-master ~]# cd /labfile/djfile/
```

步骤 2 创建 DaemonSet 的 yaml 文件。

```
[root@k8s-master djfile]# vim DS-nginx.yaml
```

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: nginx-daemonset
spec:
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
       app: nginx
```

```
  spec:
    containers:
    - name: nginx
      image: nginx:1.7.9
      ports:
      - containerPort: 80
```

步骤 3 创建 DaemonSet

```
[root@k8s-master djfile]# kubectl apply -f DS-nginx.yaml
daemonset.apps/nginx-daemonset created
```

步骤 4 查看创建的 DaemonSet

```
[root@k8s-master djfile]# kubectl get daemonset
NAME              DESIRED  CURRENT   READY   UP-TO-DATE   AVAILABLE    AGE
nginx-daemonset   3        3         3       3            3            87s
```

步骤 5 查看 DaemonSet 中 Pod 位置

```
[root@k8s-master djfile]# kubectl get pod -o wide
NAME                    READY   STATUS    RESTARTS   AGE   IP            NODE
nginx-daemonset-4xbw9   1/1     Running   0          13s   10.244.1.61   k8s-node2
nginx-daemonset-99pgf   1/1     Running   0          13s   10.244.2.84   k8s-node1
nginx-daemonset-qdk5t   1/1     Running   0          13s   10.244.0.16   k8s-master
```

步骤 6 删除任意一个 pod，查看 DaemonSet 自动恢复功能。

```
[root@k8s-master djfile]# kubectl delete pod nginx-daemonset-qdk5t（根据实际情况修改 pod 名称）
[root@k8s-master djfile]# kubectl get pod -o wide
NAME                    READY   STATUS    RESTARTS   AGE   IP            NODE
nginx-daemonset-4xbw9   1/1     Running   0          14m   10.244.1.61   k8s-node2
nginx-daemonset-6hsm5   1/1     Running   0          15s   10.244.0.17   k8s-master
nginx-daemonset-99pgf   1/1     Running   0          14m   10.244.2.84   k8s-node1
```

2.7.2 使用 Job

步骤 1 创建 Job 的 Yaml 文件

```
[root@k8s-master djfile]# vim pi-job.yaml
apiVersion: batch/v1
kind: Job
```

```
metadata:
  name: pi
spec:
  template:
    spec:
      containers:
      - name: pi
        image: perl
        command: ["perl",  "-Mbignum=bpi", "-wle", "print bpi(2000)"]
      restartPolicy: Never
  backoffLimit: 4
```

步骤 2　创建 Job 并且查看 Job 的运行状态，可以看到 Job 在 10s 后运行完毕。

```
[root@k8s-master djfile]# kubectl apply -f pi-job.yaml && kubectl get job -w
job.batch/pi created
NAME   COMPLETIONS   DURATION   AGE
pi     0/1           0s         0s
pi     1/1           10s        10s
```

步骤 3　查看任务运行的结果

```
[root@k8s-master djfile]# kubectl get pod
NAME                 READY   STATUS      RESTARTS   AGE
pi-lmrng             0/1     Completed   0          75s
```

查看 Pod 的日志信息

```
[root@k8s-master djfile]# kubectl logs pi-lmrng
3.14159265358979323……
```

2.7.3 使用 CronJob

步骤 1　创建 CronJob 的 yaml 文件，设置为每一分钟运行一次返回一次 hello

```
[root@k8s-master djfile]# vim CJ-hello.yaml
```

```
apiVersion: batch/v1beta1
kind: CronJob
metadata:
  name: hello
spec:
  schedule: "*/1 * * * *"
  jobTemplate:
    spec:
      template:
```

```
    spec:
      containers:
      - name: hello
        image: busybox
        args:
        - /bin/sh
        - -c
        - date; echo Hello from the Kubernetes cluster
      restartPolicy: OnFailure
```

步骤 2 运行 cronjob，稍等一段时间，查看 cronjob 运行情况。

```
[root@k8s-master djfile]# kubectl apply -f CJ-hello.yaml
[root@k8s-master djfile]# kubectl get pod
NAME                     READY   STATUS      RESTARTS   AGE
hello-1561952100-js7z5   0/1     Completed   0          10s

[root@k8s-master djfile]# kubectl logs hello-1561952100-js7z5
Mon Jul  1 03:35:04 UTC 2019
Hello from the Kubernetes cluster
```

步骤 3 等待数分钟，再次查看 cronjob 运行情况。可以看到每隔 1 分钟，cronjob 就创建一个新的 pod.

```
[root@k8s-master djfile]# kubectl get pod
NAME                     READY   STATUS              RESTARTS   AGE
hello-1561952580-tksrs   0/1     Completed           0          3m1s
hello-1561952640-hdg27   0/1     Completed           0          2m1s
hello-1561952700-qpk6p   0/1     Completed           0          61s
hello-1561952760-dj52p   0/1     ContainerCreating   0          1s
```

2.7.4 实训任务

步骤 1 创建一个 DaemonSet，要求如下：

1. 包含两个 pod
2. 镜像为 nginx

步骤 2 创建一个 job，用于输出 helloworld。

步骤 3 创建一个 cronjob，在每日的 xx 小时 xx 点输出 helloworld。

步骤 4 删除本次实验创建 DaemonSet，Job 和 CronJob。

2.8 健康检查

2.8.1 使用存活探针

步骤 1　创建文件夹，用于存放健康检查实验相关文件。

```
[root@k8s-master djfile]# mkdir /labfile/probefile
[root@k8s-master djfile]# cd /labfile/probefile/
```

步骤 2　创建使用 execaction 模式的存活探针 pod 的 yaml 文件。

```
[root@k8s-master probefile]# vim liveness-exec.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    test: liveness
  name: liveness-exec
spec:
  containers:
  - name: liveness
    args:
    - /bin/sh
    - -c
    - touch /tmp/healthy; sleep 30; rm -rf /tmp/healthy; sleep 600
    image: busybox
    livenessProbe:
      exec:
        command:
        - cat
        - /tmp/healthy
      initialDelaySeconds: 5
      periodSeconds: 5
```

步骤 3　创建该 Pod。

```
[root@k8s-master probefile]# kubectl apply -f liveness-exec.yaml
```

步骤 4　使用 kubectl get pod -w 命令持续监控 pod 状态。可以看到 Pod 反复重启。

```
[root@k8s-master probefile]# kubectl get pod -w
```

```
NAME                    READY   STATUS    RESTARTS   AGE
liveness-exec           1/1     Running   0          23s
```

执行 Ctrl + C 退出监控

步骤 5　使用 describe 命令查看详细 pod 信息

```
[root@k8s-master probefile]# kubectl describe pod liveness-exec
```

检查其中 events 项，确认探针工作流程。

步骤 6 创建使用 http 存活探针的 pod 的 yaml 文件。

```
[root@k8s-master probefile]# vim liveness-http.yaml
apiVersion: v1
kind: Pod
metadata:
  labels:
    test: liveness
  name: liveness-http
spec:
  containers:
  - name: liveness
    image: mirrorgooglecontainers/liveness
    args:
    - /server
    livenessProbe:
      httpGet:
        path: /healthz
        port: 8080
        httpHeaders:
        - name: X-Custom-Header
          value: Awesome
      initialDelaySeconds: 3
      periodSeconds: 3
```

步骤 7 创建该 pod

```
[root@k8s-master probefile]# kubectl apply -f liveness-http.yaml
pod/liveness-http created
```

步骤 8 使用 get 命令观察 pod 状态变化。

```
[root@k8s-master probefile]# kubectl get pod -w
NAME           READY   STATUS             RESTARTS   AGE
liveness-http  1/1     Running            3          70s
liveness-http  0/1     CrashLoopBackOff   3          72s
```

步骤 9 使用 describe 命令查看 pod 的 events

```
[root@k8s-master probefile]# kubectl describe pod liveness-http
Events:
  Type     Reason     Age                From               Message
  ----     ------     ----               ----               -------
```

```
  Normal   Scheduled  2m39s                default-scheduler   Successfully
assigned default/liveness-http to k8s-node1
  Normal   Pulled     2m2s (x3 over 2m37s)  kubelet, k8s-node1  Successfully
pulled image "mirrorgooglecontainers/liveness"
  Normal   Created    2m2s (x3 over 2m37s)  kubelet, k8s-node1  Created
container liveness
  Normal   Started    2m2s (x3 over 2m37s)  kubelet, k8s-node1  Started
container liveness
  Normal   Pulling    105s (x4 over 2m38s)  kubelet, k8s-node1  Pulling image
"mirrorgooglecontainers/liveness"
  Warning  Unhealthy  105s (x9 over 2m27s)  kubelet, k8s-node1  Liveness probe
failed: HTTP probe failed with statuscode: 500
  Normal   Killing    105s (x3 over 2m21s)  kubelet, k8s-node1  Container
liveness failed liveness probe, will be restarted
```

步骤 10　创建使用 tcp 存活探针的 pod 的 yaml，模板采用 httpd 容器镜像。

```
[root@k8s-master probefile]# vim liveness-tcp.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: liveness-tcp
  labels:
    app: httpd
spec:
  containers:
  - name: httpd
    image: httpd
    livenessProbe:
      tcpSocket:
        port: 80
      initialDelaySeconds: 10
      periodSeconds: 10
```

步骤 11　创建 pod，并且稍等两分钟，待内部所有任务完成。

```
[root@k8s-master probefile]# kubectl apply -f liveness-tcp.yaml
[root@k8s-master probefile]# kubectl get pod
```

步骤 12　进入容器，使用如下命令修改提供服务的端口，从默认的 80 端口修改为 8080 端口，并重启容器内 httpd 服务以应用端口变更。

```
[root@k8s-master probefile]# kubectl exec -it liveness-tcp /bin/bash
# /etc/init.d/openbsd-inetd stop
```

```
[root@k8s-master probefile]# kubectl exec -it liveness-tcp /bin/bash
root@liveness-tcp:/usr/local/apache2/conf# sed -i "52c listen 8080"
/usr/local/apache2/conf/httpd.conf
```

```
root@liveness-tcp:/usr/local/apache2/conf# sed -i "241c ServerName
localhost:8080" /usr/local/apache2/conf/httpd.conf
root@liveness-tcp:/usr/local/apache2/conf# httpd -k restart
```

注意提示符，以上命令有些由于过长分行了，在输入过程中不要分行。

exit 退出

步骤 13 等待一段时间后，使用 kubectl get pod 命令查看，可以看到 pod 的 restarts 次数变成了 1。

```
[root@k8s-master probefile]# kubectl get pod
NAME           READY   STATUS             RESTARTS   AGE
liveness-exec  0/1     CrashLoopBackOff   23         76m
liveness-tcp   1/1     Running            1          18m
```

这是由于未通过存活探针检测，pod 进行了重启，业务又恢复正常了，并且端口也恢复到了默认的 80 端口。

步骤 14 使用 describe 命令可以看到 pod 之前未通过 liveness 的记录。

```
[root@k8s-master probefile]# kubectl describe pod
Events:
  Type     Reason     Age                  From               Message
  ----     ------     ----                 ----               -------
  Normal   Scheduled  13m                  default-scheduler  Successfully
assigned default/liveness-tcp to k8s-master
  Normal   Pulling    6m51s (x2 over 13m)  kubelet, k8s-master  Pulling
image "httpd"
  Warning  Unhealthy  6m51s (x3 over 7m11s)  kubelet, k8s-master  Liveness
probe failed: dial tcp 10.244.0.14:80: connect: connection refused
  Normal   Killing    6m51s                kubelet, k8s-master  Container httpd
failed liveness probe, will be restarted
  Normal   Pulled     6m50s (x2 over 13m)  kubelet, k8s-master  Successfully
pulled image "httpd"
  Normal   Created    6m50s (x2 over 13m)  kubelet, k8s-master  Created
container httpd
  Normal   Started    6m50s (x2 over 13m)  kubelet, k8s-master  Started
container httpd
```

2.8.2 使用 readiness 探针

步骤 1 创建 http 的 deployment 的 yaml 文件，其中配置 readiness 探针。

```
[root@k8s-master probefile]# vim httpd-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: httpd-deployment
spec:
```

```
  replicas: 3
  selector:
    matchLabels:
      app: httpd
  template:
    metadata:
      labels:
        app: httpd
    spec:
      containers:
      - name: httpd
        image: httpd
        ports:
        - containerPort: 80
        readinessProbe:
          exec:
            command:
            - cat
            - /usr/local/apache2/htdocs/index.html
          initialDelaySeconds: 5
          periodSeconds: 5
```

步骤 2　创建 deployment

```
[root@k8s-master probefile]# kubectl apply -f httpd-deployment.yaml
deployment.apps/httpd-deployment created
```

步骤 3　创建 http 服务的 yaml 文件。

```
[root@k8s-master probefile]# vim httpd-svc.yaml
apiVersion: v1
kind: Service
metadata:
  name: httpd-svc
spec:
  selector:
    app: httpd
  ports:
  - protocol: TCP
    port: 8080
    targetPort: 80
```

步骤 4　创建 service

```
[root@k8s-master probefile]# kubectl apply -f httpd-svc.yaml
```

步骤 5　使用 describe 命令查看 http 服务的 endpoint

```
[root@k8s-master probefile]# kubectl describe service httpd-svc
Endpoints:        10.244.0.18:80,10.244.1.84:80,10.244.2.104:80
```

步骤 6　进入一个 httpd 容器，删除/usr/local/apache2/htdocs/index.html 文件。

```
[root@k8s-master probefile]# kubectl exec -it httpd-deployment-859778b7b6-57m8b
/bin/sh
# rm /usr/local/apache2/htdocs/index.html
```

exit 退出

步骤 7　使用 describe 命令查看 endpoint，可以看到该 pod 的地址已经从 endpoint 中移除。

```
[root@k8s-master probefile]# kubectl describe service httpd-svc
Endpoints:        10.244.0.18:80,10.244.2.104:80
```

步骤 8　查看 pod 的详细信息，可以看到 pod 未通过探针检测。

```
[root@k8s-master probefile]# kubectl describe pod httpd-deployment-859778b7b6-
57m8b
Events:
  Type     Reason     Age                From               Message
  ----     ------     ----               ----               -------
  Normal   Scheduled  40m                default-scheduler  Successfully
assigned default/httpd-deployment-859778b7b6-57m8b to k8s-node2
  Normal   Pulling    40m                kubelet, k8s-node2  Pulling image
"httpd"
  Normal   Pulled     40m                kubelet, k8s-node2  Successfully pulled
image "httpd"
  Normal   Created    40m                kubelet, k8s-node2  Created container
httpd
  Normal   Started    40m                kubelet, k8s-node2  Started container
httpd
  Warning  Unhealthy  6s (x25 over 2m6s)  kubelet, k8s-node2  Readiness probe
failed: cat: /usr/local/apache2/htdocs/index.html: No such file or directory
```

步骤 9　查看 pod 信息，可以看到 pod 处于 notready 状态

```
[root@k8s-master probefile]# kubectl get pod
NAME                              READY   STATUS    RESTARTS   AGE
httpd-deployment-859778b7b6-57m8b   0/1     Running   0          41m
httpd-deployment-859778b7b6-9b6p2   1/1     Running   0          41m
httpd-deployment-859778b7b6-qrkc5   1/1     Running   0          41m
```

2.8.3 实训任务

步骤 1 创建一个 http 服务（包括 deployment 和 service），deployment 中包含 3 副本。

步骤 2 使用就绪探针和存活探针，存活探针使用 execaction 模式，就绪探针使用 http 模式。

1. httpd 镜像提供的服务默认路径为/,默认端口为 80

2.9 Kubernetes 网络

2.9.1 Node 与 Pod 间通信

步骤 1 创建一个 pod

```
[root@k8s-master ~]# kubectl run nginx --image=nginx --port 80
```

```
[root@k8s-master ~]# kubectl run nginx --image=nginx --port 80
kubectl run --generator=deployment/apps.v1 is DEPRECATED and will be removed in a future version. Use kubectl run --generator=run-pod/
v1 or kubectl create instead.
deployment.apps/nginx created
```

步骤 2 查看该 pod 的 ip 地址及所处节点

```
[root@k8s-master ~]# kubectl get pods -o wide
```

```
[root@k8s-master ~]# kubectl get pods -o wide
NAME                              READY   STATUS    RESTARTS   AGE     IP            NODE        NOMINATED NODE   READINESS GATES
counter                           1/1     Running   0          20m     10.244.2.2    k8s-node2   <none>           <none>
jenkins-0                         1/1     Running   0          9m28s   10.244.2.3    k8s-node2   <none>           <none>
myapp-mychart-7956d98c68-qxtmp    1/1     Running   0          57m     10.244.1.5    k8s-node1   <none>           <none>
nginx-755464dd6c-lvnj2            1/1     Running   0          20s     10.244.1.11   k8s-node1   <none>           <none>
```

步骤 3 从 k8s-master 节点访问该 pod

```
[root@k8s-master ~]# curl 10.244.1.11
```

实际 IP 以实际输出为准

```
[root@k8s-master ~]# curl 10.244.1.11
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
    body {
        width: 35em;
        margin: 0 auto;
        font-family: Tahoma, Verdana, Arial, sans-serif;
    }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

步骤 4　查看主机上的路由可知，主机通过 Flannel 网络与 pod 通信

```
[root@k8s-master ~]# ip route
```

```
default via 192.168.137.1 dev eth0 proto static metric 100
10.244.0.0/24 dev cni0 proto kernel scope link src 10.244.0.1
10.244.1.0/24 via 10.244.1.0 dev flannel.1 onlink
10.244.2.0/24 via 10.244.2.0 dev flannel.1 onlink
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1
192.168.137.0/24 dev eth0 proto kernel scope link src 192.168.137.11 metric 100
```

flannel.1 是 flannel 插件生成的逻辑接口

2.9.2 Pod 和 Pod 间通信

步骤 1　紧接上一个实验，创建一个 client pod，并进入该 pod 的 tty

```
[root@k8s-master ~]# kubectl run -i -t busybox --image=busybox
```

#该命令执行后将自动进入容器环境

#另起一个终端，查看 pods 运行所在节点

```
[root@k8s-master ~]# kubectl get pods -o wide
```

```
[root@k8s-master ~]# kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE     IP             NODE        NOMINATED NODE   READINESS GATES
busybox-594898f99b-qjbch            1/1     Running   0          3m38s   10.244.1.12    k8s-node1   <none>           <none>
counter                             1/1     Running   0          33m     10.244.2.2     k8s-node2   <none>           <none>
jenkins-0                           1/1     Running   0          22m     10.244.2.3     k8s-node2   <none>           <none>
myapp-mychart-7956d98c68-qxtmp      1/1     Running   0          70m     10.244.1.5     k8s-node1   <none>           <none>
nginx-755464dd6c-lvnj2              1/1     Running   0          13m     10.244.1.11    k8s-node1   <none>           <none>
```

步骤 2　连通性测试，从该 busybox pod 访问 nginx pod 服务。

容器中无 curl，用 telnet 命令模拟访问

```
/ # telnet 10.244.1.41 80
```

```
/ # telnet 10.244.1.11 80
Connected to 10.244.1.11
```

GET #使用 GET 方法获取

```
/ # telnet 10.244.1.11 80
Connected to 10.244.1.11
GET
HTTP/1.1 400 Bad Request
Server: nginx/1.17.2
Date: Thu, 25 Jul 2019 02:17:09 GMT
Content-Type: text/html
Content-Length: 157
Connection: close

<html>
<head><title>400 Bad Request</title></head>
<body>
<center><h1>400 Bad Request</h1></center>
<hr><center>nginx/1.17.2</center>
</body>
</html>
Connection closed by foreign host
```

exit 退出

2.9.3 集群外访问实验（NodePort 方式）

步骤 1 查看已经运行的 pod 的 label

```
[root@k8s-master ~]# kubectl get pod --show-labels
```

```
[root@k8s-master ~]# kubectl get pod --show-labels
NAME                                READY   STATUS    RESTARTS   AGE   LABELS
busybox-594898f99b-qjbch            1/1     Running   1          21m   pod-template-hash=594898f99b,run=busybox
counter                             1/1     Running   0          51m   <none>
jenkins-0                           1/1     Running   0          40m   controller-revision-hash=jenkins-5b6bd8f9d4,name=jenkins,statefuls
et.kubernetes.io/pod-name=jenkins-0
myapp-mychart-7956d98c68-qxtmp      1/1     Running   0          88m   app.kubernetes.io/instance=myapp,app.kubernetes.io/name=mychart,po
d-template-hash=7956d98c68
nginx-755464dd6c-lvnj2              1/1     Running   0          31m   pod-template-hash=755464dd6c,run=nginx
```

步骤 2 在 master 节点创建/labfile/log 目录，用于保存配置文件。后续本章的 yaml 文件保存在此处。

```
[root@k8s-master /]# cd /
[root@k8s-master /]# mkdir labfile
[root@k8s-master /]# cd labfile/
[root@k8s-master labfile]# mkdir network/
```

步骤 3 创建 type 为 nodePort 的 Service 的 yaml 文件

```
[root@k8s-master network]# vim serviceNodeport.yaml
```

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-access
  namespace: default
spec:
  selector:
```

```
  run: nginx
clusterIP: 10.96.123.123
type: NodePort
ports:
- port: 80   #service 端口
  targetPort: 80 #容器端口
  nodePort: 30080    #node 节点端口
```

步骤 4　创建该 Service

```
[root@k8s-master network]# kubectl apply -f serviceNodeport.yaml
```

步骤 5　查看该 service 状态

```
[root@k8s-master ~]# kubectl get svc -o wide
```

```
[root@k8s-master ~]# kubectl get svc -o wide
NAME            TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)               AGE   SELECTOR
jenkins         ClusterIP   10.104.231.144   <none>        8080/TCP,50000/TCP    47m   name=jenkins
kubernetes      ClusterIP   10.96.0.1        <none>        443/TCP               36d   <none>
myapp-mychart   NodePort    10.103.103.36    <none>        60000:30549/TCP       16h   app.kubernetes.io/instance=myapp,app.kubernetes.
io/name=mychart
nginx-access    NodePort    10.96.123.123    <none>        80:30080/TCP          12s   run=nginx
```

步骤 6　集群外主机可使用 Node 节点地址访问该 Pod 服务（任一 Node 节点地址均可）。



步骤 7　在节点上用 ss 命令查看可知，每个节点上都有一个 kube-proxy 发起的进程监听 30080 端口

```
[root@k8s-master ~]# ss -ntlp | grep 30080
```

```
LISTEN   0    128    :::30080    :::*    users:(("kube-proxy",pid=15859,fd=12))
```

2.10 Kubernetes 存储

2.10.1 使用 emptyDir

步骤 1　创建存储实验使用的目录。

```
[root@k8s-master ~]# mkdir /labfile/storagefile
[root@k8s-master ~]# cd /labfile/storagefile/
```

步骤 2　创建使用 emptyDir 的 pod 的 yaml 文件。

```
[root@k8s-master storagefile]# vim empty-pod.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: em
spec:
  containers:
  - image: ubuntu
    name: test-container
    volumeMounts:
    - mountPath: /cache
      name: cache-volume
    args:
    - /bin/sh
    - -c
    - sleep 30000
  volumes:
  - name: cache-volume
    emptyDir: {}
```

步骤 3　创建该 Pod

```
[root@k8s-master storagefile]# kubectl apply -f empty-pod.yaml
pod/em created
```

步骤 4　进入容器，在 emptyDir 挂载的目录中创建一个文件

```
[root@k8s-master storagefile]# kubectl exec em -it /bin/sh
# cd /cache
# cat > hello.file<<EOF
> hello
> world
> EOF
# cat hello.file
hello
world
# exit
```

步骤 5　查看 pod 所在节点，可以看到该 pod 运行在 node2 上。

```
[root@k8s-master storagefile]# kubectl get pod -o wide
NAME   READY   STATUS    RESTARTS   AGE     IP           NODE        NOMINATED
em     1/1     Running   0          6m36s   10.244.1.86  k8s-node2   <none>
```

步骤 6　登录 node2 节点命令行，查看运行的容器

```
[root@k8s-node2 ~]# docker ps
CONTAINER ID        IMAGE
COMMAND                  CREATED             STATUS              PORTS
NAMES
93269d8a6fec        ubuntu
"/bin/sh -c 'sleep 3…"   9 minutes ago       Up 9 minutes
k8s_test-container_em_default_4dae7b7f-9c63-11e9-ac91-000c290a92ce_0
```

步骤 7　在其中找到之前创建的 pod 的主容器，使用 docker inspect 查看信息，显示数据如下

```
[root@k8s-node2 ~]# docker inspect 93269d8a6fec
……
"Mounts": [
          {
              "Type": "bind",
              "Source": "/var/lib/kubelet/pods/4dae7b7f-9c63-11e9-ac91-
000c290a92ce/volumes/kubernetes.io~empty-dir/cache-volume",
              "Destination": "/cache",
              "Mode": "Z",
              "RW": true,
              "Propagation": "rprivate"
          },
```

步骤 8　进入该文件中显示的目录。

```
[root@k8s-node2 ~]# cd /var/lib/kubelet/pods/4dae7b7f-9c63-11e9-ac91-
000c290a92ce/volumes/kubernetes.io~empty-dir/cache-volume
[root@k8s-node2 cache-volume]# ls
hello.file

 [root@k8s-node2 cache-volume]# cat hello.file
hello
world
```

步骤 9　回到 master 节点，删除 pod

```
[root@k8s-master probefile]# kubectl delete pod em
pod "em" deleted
```

步骤 10　回到 node2 节点，发现文件，以及目录都不见了。

```
[root@k8s-node2 ..]# cd /var/lib/kubelet/pods
[root@k8s-node2 pods]# ls
```

```
463a49af-91bd-11e9-ac91-000c290a92ce  da2eba90-91bc-11e9-ac91-000c290a92ce
b0d139c3-91be-11e9-ac91-000c290a92ce  ff76e384-91bf-11e9-ac91-000c290a92ce
```

2.10.2 使用 emptyDir 的容量限制功能

步骤 1　创建一个带容量限制的 emptyDir pod yaml

```
[root@k8s-master storagefile]# vim limit-pod.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: em2
spec:
  containers:
  - image: ubuntu
    name: test-container2
    volumeMounts:
    - mountPath: /cache
      name: cache-volume
    args:
    - /bin/sh
    - -c
    - sleep 30000
  volumes:
  - name: cache-volume
    emptyDir:
      sizeLimit: 1Gi
```

步骤 2　创建该 Pod

```
[root@k8s-master storagefile]# kubectl apply -f limit-pod.yaml
```

```
pod/em2 created
```

步骤 3　进入该 pod 后，在/cache 文件夹内创建一个 2G 的文件。

```
[root@k8s-master storagefile]# kubectl exec -it em2 /bin/sh
# cd /cache
# dd if=/dev/zero of=/cache/test2g bs=1M count=2048
```

```
2048+0 records in
2048+0 records out
2147483648 bytes (2.1 GB, 2.0 GiB) copied, 2.67003 s, 804 MB/s
```

步骤 4　查看容器状态，进入 evicted 状态了。

```
[root@k8s-master storagefile]# kubectl get pod
```

```
NAME   READY   STATUS    RESTARTS   AGE
em2    0/1     Evicted   0          8m52s
```

2.10.3 hostPath

步骤 1　在三个节点上执行以下命令，各创建一个文件夹，用于挂载给 pod

```
[root@k8s-master storagefile]# mkdir /testdir
```

步骤 2　创建用于 hostPath 的 pod 的 yaml，挂载给 pod 的目录地址为主机上存在的文件夹

```
[root@k8s-master storagefile]# vim hostPath-pod.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: hppod
spec:
  containers:
  - image: ubuntu
    name: hp-container
    volumeMounts:
    - mountPath: /hp-dir
      name: hp-volume
    args:
    - /bin/sh
    - -c
    - sleep 30000
  volumes:
  - name: hp-volume
    hostPath:
      path: /testdir
      type: Directory
```

步骤 3　创建该 pod

```
[root@k8s-master storagefile]# kubectl apply -f hostPath-pod.yaml
pod/hppod created

[root@k8s-master storagefile]# kubectl get pod
NAME    READY   STATUS    RESTARTS   AGE
hppod   1/1     Running   0          9s
```

步骤 4　进入 pod，在挂载 hostPath 的文件夹内写入一个文件。

```
[root@k8s-master storagefile]# kubectl exec -it hppod /bin/sh
# cd /hp-dir
```

```
# ls
# cat >hello2 <<EOF
> hello
> again
> EOF
# exit
```

步骤 5　进入对应节点的对应目录，查看该文件是否存在

```
[root@k8s-node2 pods]# cd /testdir
[root@k8s-node2 testdir]# ls
hello2

 [root@k8s-node2 testdir]# cat hello2
hello
again
```

步骤 6　删除 pod，查看该文件是否仍然存在。可以看到，这次文件没有被删除。

```
[root@k8s-master storagefile]# kubectl delete pod hppod
pod "hppod" deleted

 [root@k8s-node2 testdir]# cat hello2
hello
again
```

2.10.4 PV 和 PVC

步骤 1　在开始本次实验前，需要已经配置好 nfs 服务，本次实验使用的 nfs 服务搭建在 master

```
[root@k8s-master storagefile]# showmount -e localhost
Export list for localhost:
/nfs/ *
```

步骤 2　在 nfs 文件夹下创建 pv1 目录

```
[root@k8s-master nfs]# mkdir pv1
[root@k8s-master nfs]# ls
pv1
```

步骤 3　编写 PV 的 yaml 文件

```
[root@k8s-master storagefile]# vim pv1.yaml
```

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: mypv
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Recycle
  nfs:
    path: /nfs/pv1
    server: 192.168.137.11   #此处为 nfs 服务器 IP（即 master 节点 IP）
```

步骤 4　创建 PV

```
[root@k8s-master storagefile]# kubectl apply -f pv1.yaml
persistentvolume/mypv created
```

查看 PV

```
[root@k8s-master storagefile]# kubectl get pv
NAME   CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS      CLAIM       AGE
mypv   1Gi        RWO            Recycle          Available               101s
```

步骤 5　创建 PVC 的 yaml 文件，配置时指定 PV 的名称。

```
[root@k8s-master storagefile]# vim pvc1.yaml
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mypvc
spec:
  accessModes:
    - ReadWriteOnce
  volumeName: mypv
  resources:
    requests:
      storage: 1Gi
```

步骤 6　创建 PVC

```
[root@k8s-master storagefile]# kubectl apply -f pvc1.yaml
persistentvolumeclaim/mypvc created
```

步骤 7　查看 PV 和 PVC 的状态，可以看到，PV 的状态从 Available 变为 Bound，而 PVC 的状态也是 Bound。

```
[root@k8s-master storagefile]# kubectl get pv
NAME   CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM
mypv   1Gi         RWO            Recycle          Bound    default/mypvc
[root@k8s-master storagefile]# kubectl get pvc
NAME    STATUS   VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   AGE
mypvc   Bound    mypv     1Gi         RWO                           112s
```

步骤 8　创建 Pod，使用该 PVC

```
[root@k8s-master storagefile]# vim testpod.yaml
apiVersion: v1
kind: Pod
metadata:
  labels:
    test: pvctest
  name: pvcpod
spec:
  containers:
  - name: busybox
    args:
    - /bin/sh
    - -c
    - sleep 30000;
    image: busybox
    volumeMounts:
    - mountPath: /pvcdir
      name: pvc-volume
  volumes:
    - name: pvc-volume
      persistentVolumeClaim:
        claimName: mypvc
```

步骤 9　创建 Pod，并进入 pod 写入文件。

```
[root@k8s-master storagefile]# kubectl apply -f testpod.yaml
pod/pvcpod created
[root@k8s-master storagefile]# kubectl exec -it pvcpod /bin/sh
/ # cd /pvcdir
/pvcdir # ls
/pvcdir # touch hello
/pvcdir # exit
```

步骤 10　在 nfs 目录查看是否存在 hello 文件

```
[root@k8s-master storagefile]# cd /nfs/pv1
[root@k8s-master pv1]# ls
hello
```

步骤 11 删除 pod 和 PVC，并等待系统自动回收完成。

```
[root@k8s-master pv1]# kubectl delete pod pvcpod
pod "pvcpod" deleted
```

```
[root@k8s-master storagefile]# kubectl delete pvc mypvc && kubectl get pod -w
persistentvolumeclaim "mypvc" deleted
NAME                READY   STATUS              RESTARTS   AGE
recycler-for-mypv   0/1     ContainerCreating   0          0s
recycler-for-mypv   0/1     Completed           0          0s
recycler-for-mypv   0/1     Terminating         0          0s
recycler-for-mypv   0/1     Terminating         0          0s
```

步骤 12 再次查看 PV 状态，再次变为 available 的可用状态

```
[root@k8s-master storagefile]# kubectl get pv
NAME    CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS      CLAIM   STORAGECLASS   REASON   AGE
mypv    1Gi        RWO            Recycle          Available                                   2m31s
```

步骤 13 查看 nfs 中是否存在 hello 文件，可以看到已经被清空

```
[root@k8s-master storagefile]# cd /nfs
[root@k8s-master nfs]# ls
pv1
[root@k8s-master nfs]# cd pv1
[root@k8s-master pv1]# ls
[root@k8s-master pv1]#
```

步骤 14 使用 PVC2.yaml 文件创建 PVC

```
[root@k8s-master storagefile]# vim pvc2.yaml
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mypvc2
spec:
  accessModes:
    - ReadWriteOnce
  volumeName: mypv
  resources:
    requests:
```

```
    storage: 1Gi
```

```
[root@k8s-master storagefile]# kubectl apply -f pvc2.yaml
persistentvolumeclaim/mypvc2 created
```

步骤 15　验证 PV 的状态再次变为 bound 状态，并且绑定给了 mypvc2

```
[root@k8s-master storagefile]# kubectl get pv
NAME   CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM           AGE
mypv   1Gi        RWO            Recycle          Bound    default/mypvc2  12m
```

```
[root@k8s-master storagefile]# kubectl get pvc
NAME     STATUS   VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   AGE
mypvc2   Bound    mypv     1Gi        RWO                           37s
```

2.10.5 使用 StorageClass 方式关联 PV 和 PVC

步骤 1　在 nfs 目录中创建 PV2 目录

```
[root@k8s-master storagefile]# mkdir /nfs/pv2
```

步骤 2　创建 PV 的 yaml 文件

```
[root@k8s-master storagefile]# vim pv-sc.yaml
```

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-sc
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Recycle
  storageClassName: nfs
  nfs:
    path: /nfs/pv2
     server: 192.168.137.11
```

步骤 3　创建 pvc 的 yaml 文件

```
[root@k8s-master storagefile]# vim pvc-sc.yaml
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
```

```
  name: pvc-sc
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: nfs
  resources:
    requests:
      storage: 1Gi
```

步骤 4　创建 pv 和 pvc

```
[root@k8s-master storagefile]# kubectl apply -f pv-sc.yaml
persistentvolume/pv-sc created

[root@k8s-master storagefile]# kubectl apply -f pvc-sc.yaml
persistentvolumeclaim/pvc-sc created
```

步骤 5　查看 PV 和 PVC 的绑定

```
[root@k8s-master storagefile]# kubectl get pv
NAME    CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM           STORAGECLASS   AGE
pv-sc   1Gi        RWO            Recycle          Bound    default/pvc-sc  nfs            33s

[root@k8s-master storagefile]# kubectl get pvc
NAME     STATUS   VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   AGE
pvc-sc   Bound    pv-sc    1Gi        RWO            nfs            28s
```

2.10.6 实训任务

步骤 1　创建一个 PodA，和一个 PodB，镜像为 busybox。可以实现

1. PodA 中在指定位置创建一个文件，PodB 可以查看该文件。

步骤 2　在本地创建一个文件 index.html，内容如下

```
<html><body><h1>I am a new man</h1></body></html>
```

步骤 3　创建一个 httpd 的 Pod，使用 PVC，PV 的形式将 index.html 映射到 pod 中。

步骤 4　进入 Pod，该 index.html 文件替换 pod 中/usr/local/apache2/htdocs/index.html 文件。

步骤 5　在 pod 外使用 curl 命令查看 httpd 服务展示的主页是否发生变化。

2.11 Configmap 和 Secret

2.11.1 创建 configmap

步骤 1　创建一个目录用于存放 configmap 和 secret 实验的文件

```
[root@k8s-master storagefile]# mkdir /labfile/configmap
[root@k8s-master storagefile]# cd /labfile/configmap
```

步骤 2　创建两个文件，用于保存配置文件。

```
[root@k8s-master configmap]# vim game.properties
```

```
enemies=aliens
lives=3
enemies.cheat=true
enemies.cheat.lever=noGoodRotten
```

```
[root@k8s-master configmap]# vim ui.properties
```

```
color.good=purple
color.bad=yello
allow.textmode=true
```

步骤 3　从这两个文件创建 configmap

```
[root@k8s-master configmap]# kubectl create configmap game-config --from-
file=/labfile/configmap
```

```
configmap/game-config created
```

步骤 4　查看创建的 configmap

```
[root@k8s-master configmap]# kubectl get configmap
```

```
NAME          DATA   AGE
game-config     2     58s
```

查看 configmap 详细信息

```
[root@k8s-master configmap]# kubectl describe configmap game-config
```

```
Name:        game-config
Namespace:    default
Labels:      <none>
Annotations:  <none>

Data
====
game.properties:
----
```

```
enemies=aliens
lives=3
enemies.cheat=true
enemies.cheat.lever=noGoodRotten

ui.properties:
----
color.good=purple
color.bad=yello
allow.textmode=true

Events:  <none>
```

步骤 5　从文件创建 configmap

```
[root@k8s-master configmap]# kubectl create configmap game-config-3 --from-
file=/labfile/configmap/game.properties --from-
file=/labfile/configmap/ui.properties
```

```
configmap/game-config-3 created
```

步骤 6　查看 game-config-3 的详细信息，与 game-config 对比，内容一致。

```
[root@k8s-master configmap]# kubectl describe configmap game-config-3
```

```
Name:         game-config-3
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
game.properties:
----
enemies=aliens
lives=3
enemies.cheat=true
enemies.cheat.lever=noGoodRotten

ui.properties:
----
color.good=purple
color.bad=yello
allow.textmode=true

Events:  <none>
```

步骤 7　直接从参数创建 configmap

```
[root@k8s-master configmap]# kubectl create configmap special-config --from-literal=special.how=very --from-literal=special.type=charm
```

步骤 8　查看创建的 configmap 结果

```
[root@k8s-master configmap]# kubectl describe configmap special-config
Name:         special-config
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
special.how:
----
very
special.type:
----
charm
Events:  <none>
```

步骤 9　从 yaml 文件创建 configmap

```
[root@k8s-master configmap]# vim configmap2.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: specialconfig-2
data:
  key1: value1
  pro.property: |
    key2: value2
    key3: value3
```

步骤 10　创建 configmap 并查看结果。

```
[root@k8s-master configmap]# kubectl apply -f configmap2.yaml
```

查看创建的结果

```
[root@k8s-master configmap]# kubectl describe configmap specialconfig-2
Name:         specialconfig-2
Namespace:    default
Labels:       <none>
Annotations:  kubectl.kubernetes.io/last-applied-configuration:
              {"apiVersion":"v1","data":{"key1":"value1","pro.property":"key2:
value2\nkey3: value3\n"},"kind":"ConfigMap","metadata":{"annotations":{},...
```

```
Data
====
key1:
----
value1
pro.property:
----
key2: value2
key3: value3

Events:  <none>
```

2.11.2 使用 configmap

步骤 1　创建 Pod 的 yaml 文件，引用 configmap 做为 env 参数。

```
[root@k8s-master configmap]# vim cmpod.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: cm-test-pod
spec:
  containers:
    - name: cm-container
      image: busybox
      args: ["/bin/sh", "-c", "env"]
      env:
        - name: special-env
          valueFrom:
            configMapKeyRef:
              name: specialconfig-2
              key: key1
      envFrom:
        - configMapRef:
            name: specialconfig-2
  restartPolicy: Never
```

步骤 2　创建 pod

```
[root@k8s-master configmap]# kubectl apply -f cmpod.yaml
```

步骤 3　查看 pod 输出的日志

```
[root@k8s-master configmap]# kubectl logs cm-test-pod
```

```
KUBERNETES_PORT=tcp://10.96.0.1:443
KUBERNETES_SERVICE_PORT=443
```

```
HOSTNAME=cm-test-pod
SHLVL=1
HOME=/root
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HTTPD_SVC_PORT_8080_TCP_ADDR=10.105.253.35
HTTPD_SVC_SERVICE_HOST=10.105.253.35
KUBERNETES_PORT_443_TCP_PORT=443
key1=value1
KUBERNETES_PORT_443_TCP_PROTO=tcp
HTTPD_SVC_PORT_8080_TCP_PORT=8080
special-env=value1
HTTPD_SVC_PORT_8080_TCP_PROTO=tcp
HTTPD_SVC_PORT=tcp://10.105.253.35:8080
HTTPD_SVC_SERVICE_PORT=8080
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
KUBERNETES_SERVICE_HOST=10.96.0.1
PWD=/
pro.property=key2: value2
key3: value3

HTTPD_SVC_PORT_8080_TCP=tcp://10.105.253.35:8080
```

步骤 4　通过 volume 使用 configmap，创建 pod 的 yaml 文件

```
[root@k8s-master configmap]# vim cmpod2.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: cmpod2
spec:
  containers:
  - name: cmpod2
    image: busybox
    args: [ "/bin/sh", "-c", "sleep 3000" ]
    volumeMounts:
    - name: db
      mountPath: "/etc/db"
      readOnly: true
  volumes:
  - name: db
    configMap:
      name: specialconfig-2
```

步骤 5　创建 pod

```
[root@k8s-master configmap]# kubectl apply -f cmpod2.yaml
pod/cmpod2 created
```

步骤 6 进入 pod，查看 volume 内文件。

```
[root@k8s-master configmap]# kubectl exec -it cmpod2 /bin/sh
/ # cd /etc/db
/etc/db # ls
key1        pro.property
/etc/db # cat key1
value1

/etc/db # cat pro.property
key2: value2
key3: value3
```

exit 退出

2.11.3 使用 Secret

步骤 1 创建两个文件，存放“用户”和“密码”两个参数。

```
[root@k8s-master configmap]# echo -n "admin" > username.txt
[root@k8s-master configmap]# echo -n "Huawei@123" > password.txt
```

步骤 2 使用命令行创建 Secret，名字为 db-user-pass

```
[root@k8s-master configmap]# kubectl create secret generic db-user-pass --from-file=username.txt --from-file=password.txt
```

步骤 3 查看 secret 信息，可以看到 secret 不直接显示明文数据，注意！请不要删除 default token！

```
[root@k8s-master configmap]# kubectl get secret
NAME                  TYPE                                  DATA   AGE
db-user-pass          Opaque                                2      33m
default-token-4vq75   kubernetes.io/service-account-token   3      14d
[root@k8s-master configmap]# kubectl describe secret db-user-pass
Name:         db-user-pass
Namespace:    default
Labels:       <none>
Annotations:  <none>

Type:  Opaque

Data
====
```

```
password.txt:  10 bytes
username.txt:  5 bytes
```

步骤 4　使用 base64 编码数据

```
[root@k8s-master configmap]# echo -n "admin" | base64
YWRtaW4=
[root@k8s-master configmap]# echo -n "Huawei@123" | base64
SHVhd2VpQDEyMw==
```

步骤 5　创建 Yaml 文件，写入编码后的数据

```
[root@k8s-master configmap]# vim secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: mysecret
type: Opaque
data:
  username: YWRtaW4=
  password: SHVhd2VpQDEyMw==
```

步骤 6　创建 secret

```
[root@k8s-master configmap]# kubectl apply -f secret.yaml
```

步骤 7　查看 secret 的详细信息

```
[root@k8s-master configmap]# kubectl describe secrets mysecret
Name:         mysecret
Namespace:    default
Labels:       <none>
Annotations:
Type:         Opaque

Data
====
password:  10 bytes
username:  5 bytes
```

2.11.4 使用 secret

步骤 1　创建 Pod 的 yaml 文件，以 volume 形式挂在 secrets

```
[root@k8s-master configmap]# vim spod.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: spod
spec:
  containers:
  - image: busybox
    name: spod
    args: ["/bin/sh","-c","sleep 3000"]
    volumeMounts:
    - name: secrets
      mountPath: "/etc/secret"
      readOnly: true
  volumes:
  - name: secrets
    secret:
      secretName: mysecret
```

步骤 2　创建 pod

```
[root@k8s-master configmap]# kubectl apply -f spod.yaml
```

步骤 3　进入 pod，查看 pod 内 secret 数据，可以看到数据已经被解码，并且一个“DATA”存储为一个文件。

```
[root@k8s-master configmap]# kubectl exec -it spod /bin/sh
/ # cd /etc/secret
/etc/secret # ls
password  username
/etc/secret # cat password
Huawei@123
/etc/secret # cat username
admin
```

步骤 4　使用 Secret 并且挂载指定值，创建使用 secret 的 pod

```
[root@k8s-master configmap]# vim spod2.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: spod2
spec:
  containers:
  - image: busybox
    name: spod2
    args: ["/bin/sh","-c","sleep 3000"]
    volumeMounts:
    - name: secrets
```

```
    mountPath: "/etc/secret"
    readOnly: true
volumes:
- name: secrets
  secret:
    secretName: mysecret
    items:
    - key: password
      path: my-group/my-passwd
```

步骤 5 创建 pod

```
[root@k8s-master configmap]# kubectl apply -f spod2.yaml
```

步骤 6 进入 Pod，查看 secret 数据。

```
[root@k8s-master configmap]# kubectl exec -it spod2 /bin/sh
/ # cd /etc/secret
/etc/secret # ls
my-group
/etc/secret # cd my-group/
/etc/secret/..2019_07_03_01_45_21.293660863/my-group # ls
my-passwd
/etc/secret/..2019_07_03_01_45_21.293660863/my-group # cat my-passwd
Huawei@123
```

2.11.5 实训任务

步骤 1 创建一个使用参数创建一个 configmap，包括如下内容

```
name：jack
age：19
address：street
city：gotham
```

步骤 2 使用文件创建一个 secret，包括如下内容

```
user：admin
password：Huawei@123
```

步骤 3 将 password 导入为 Pod 内容器的环境变量，环境变量的名称为 oldpassword，将 configmap 中所有参数供 Pod 可访问。

1. Pod 使用 busybox 镜像

步骤 4 修改 configmap 中 address 的值为 bridge，并且使其在 Pod 中实时热更新。

2.12 StatefulSet（可选）

2.12.1 创建 StatefulSet

步骤 1　创建一个文件夹用于存放 StatefulSet 相关文件。

```
[root@k8s-master labfile]# mkdir /labfile/statefulset
[root@k8s-master labfile]# cd /labfile/statefulset/
```

步骤 2　在/nfs 文件夹下创建 pv1，pv2，pv3 三个文件夹，用于给 pv 使用。

```
[root@k8s-master stateful]# cd /nfs
[root@k8s-master nfs]# mkdir pv1 pv2 pv3
```

步骤 3　创建 3 个 PV 的配置文件，样例如下。PV2 和 PV3 的配置文件中需要修改 metadata.name 项和 spec.nfs.path 项。

```
[root@k8s-master stateful]# vim mypv1.yaml
```

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: mypv1
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Recycle
  storageClassName: my-sc
  nfs:
    path: /nfs/pv1
    server: 192.168.137.11
```

步骤 4　创建三个 PV。

```
[root@k8s-master stateful]# kubectl apply -f mypv1.yaml
persistentvolume/mypv1 created
[root@k8s-master stateful]# kubectl apply -f mypv2.yaml
persistentvolume/mypv2 created
[root@k8s-master stateful]# kubectl apply -f mypv3.yaml
persistentvolume/mypv3 created
```

步骤 5　创建 StatefulSet 的 yaml 文件

```
[root@k8s-master stateful]# vim stateful.yaml
```

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
spec:
  selector:
    matchLabels:
      app: nginx
  serviceName: nginx
  replicas: 3
  template:
    metadata:
      labels:
        app: nginx
    spec:
      terminationGracePeriodSeconds: 10
      containers:
      - name: nginx
        image: nginx:1.7.9
        ports:
        - containerPort: 80
          name: web
        volumeMounts:
        - name: stor
          mountPath: /usr/share/nginx/html
  volumeClaimTemplates:
  - metadata:
      name: stor
    spec:
      accessModes:
        - ReadWriteOnce
      storageClassName: my-sc
      resources:
        requests:
          storage: 1Gi
```

步骤 6 创建 statefulSet

```
[root@k8s-master stateful]# kubectl apply -f stateful.yaml
```

步骤 7 查看 pv 和 pvc

```
[root@k8s-master stateful]# kubectl get pv
```

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM	STORAGECLASS
mypv1	1Gi	RWO	Recycle	Bound	default/stor-web-1	my-sc
mypv2	1Gi	RWO	Recycle	Bound	default/stor-web-2	my-sc
mypv3	1Gi	RWO	Recycle	Bound	default/stor-web-0	my-sc

```
[root@k8s-master stateful]# kubectl get pvc
NAME        STATUS   VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   AGE
stor-web-0   Bound    mypv3    1Gi        RWO            my-sc          100s
stor-web-1   Bound    mypv1    1Gi        RWO            my-sc          98s
stor-web-2   Bound    mypv2    1Gi        RWO            my-sc          95s
```

步骤 8 查看 statefulset 状态和事件

```
[root@k8s-master stateful]# kubectl get statefulset
NAME   READY   AGE
web    3/3     2m45s
```

详细信息

```
[root@k8s-master stateful]# kubectl describe statefulset
```

回显中可以看到事件信息：

```
Events:
  Type    Reason            Age    From                 Message
  ----    ------            ----   ----                 -------
  Normal  SuccessfulCreate  3m20s  statefulset-controller  create Claim stor-web-0 Pod web-0 in StatefulSet web success
  Normal  SuccessfulCreate  3m20s  statefulset-controller  create Pod web-0 in StatefulSet web successful
  Normal  SuccessfulCreate  3m18s  statefulset-controller  create Claim stor-web-1 Pod web-1 in StatefulSet web success
  Normal  SuccessfulCreate  3m18s  statefulset-controller  create Pod web-1 in StatefulSet web successful
  Normal  SuccessfulCreate  3m15s  statefulset-controller  create Claim stor-web-2 Pod web-2 in StatefulSet web success
  Normal  SuccessfulCreate  3m15s  statefulset-controller  create Pod web-2 in StatefulSet web successful
```

步骤 9 创建 headless 服务

```
[root@k8s-master stateful]# vim headless.yaml
```

```
apiVersion: v1
kind: Service
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  ports:
  - port: 80
    name: web
  clusterIP: None
```

```
  selector:
    app: nginx
```

```
[root@k8s-master stateful]# kubectl apply -f headless.yaml
```

步骤 10 创建一个 clientpod，用于测试 headless 服务（和之前在 service 章节创建的测试 pod 一致）

```
[root@k8s-master servicefile]# vim client.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: clientpod
spec:
  containers:
    - name: clientpod
      image: busybox:1.28.3
      args:
      - /bin/sh
      - -c
      - sleep 30000
```

```
[root@k8s-master servicefile]# kubectl apply -f client.yaml
```

步骤 11 进入 clientpod，查看 statefulset 服务的 nslookup 信息。可以看到，每个 pod 被用 web-n.nginx 命名。

```
[root@k8s-master servicefile]# kubectl exec -it clientpod /bin/sh
/ # nslookup nginx
```

```
Server:    10.96.0.10
Address 1: 10.96.0.10 kube-dns.kube-system.svc.cluster.local

Name:      nginx
Address 1: 10.244.0.31 web-1.nginx.default.svc.cluster.local
Address 2: 10.244.2.132 web-0.nginx.default.svc.cluster.local
Address 3: 10.244.1.104 web-2.nginx.default.svc.cluster.local
```

2.12.2 StatefulSet 常见操作

步骤 1 进入 statefulSet 中 pod 的持久存储目录（/usr/share/nginx/html），创建一个文件。

```
[root@k8s-master servicefile]# kubectl exec -it web-0 /bin/sh
# cd /usr/share/nginx/html
# cat << EOF > hello.file
> this is hello web-0
> EOF
```

步骤 2　删除 web-0 pod，可以看到 statefulSet 自动创建了一个新的 web-0 pod。

```
[root@k8s-master servicefile]# kubectl delete pod web-0
pod "web-0" deleted

[root@k8s-master servicefile]# kubectl get pod
NAME        READY   STATUS    RESTARTS   AGE
clientpod   1/1     Running   0          19m
web-0       1/1     Running   0          23s
web-1       1/1     Running   0          46m
web-2       1/1     Running   0          46m
```

步骤 3　进入 clientpod，验证能否用原来的域名访问 web-0

```
[root@k8s-master servicefile]# kubectl exec -it clientpod /bin/sh
/ # nslookup web-0.nginx
Server:    10.96.0.10
Address 1: 10.96.0.10 kube-dns.kube-system.svc.cluster.local

Name:      web-0.nginx
Address 1: 10.244.2.134 web-0.nginx.default.svc.cluster.local
```

步骤 4　进入 web-0 pod，查看保持在持久化存储上的文件是否存在。

```
[root@k8s-master servicefile]# kubectl exec -it web-0 /bin/sh
# cat /usr/share/nginx/html/hello.file
this is hello web-0
```

步骤 5　升级 statefulSet，将原有 statefulSet.yaml 中 nginx:1.7.9 的镜像版本变更为 nginx:1.9.1

```
[root@k8s-master stateful]# cp stateful.yaml stateful-new.yaml
[root@k8s-master stateful]# vim stateful-new.yaml
```

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
spec:
  selector:
    matchLabels:
      app: nginx
  serviceName: nginx
  replicas: 3
  template:
```

```
  metadata:
    labels:
      app: nginx
  spec:
    terminationGracePeriodSeconds: 10
    containers:
    - name: nginx
      image: nginx:1.9.1
      ports:
      - containerPort: 80
        name: web
      volumeMounts:
      - name: stor
        mountPath: /usr/share/nginx/html
volumeClaimTemplates:
- metadata:
    name: stor
  spec:
    accessModes:
      - ReadWriteOnce
    storageClassName: my-sc
    resources:
      requests:
        storage: 1Gi
```

步骤 6　应用更新并查看系统运行状态，可以看到系统从 web-2 Pod 开始更新的全过程。

```
[root@k8s-master stateful]# kubectl apply -f stateful-new.yaml && kubectl get
pod -w
```

2.12.3 实训任务

步骤 1　删除在 1.11statefulset 章节创建的所有对象。

步骤 2　创建一个 statefulset 及相应服务，要求如下：

1. 镜像使用 httpd
2. 两副本
3. 后端持久存储大小为 1g
4. headless 服务对外提供的端口为 8080

步骤 3　扩容该 statefulset 使其具备 4 个副本。

步骤 4　删除 statefulSet，要求安装 Pod 序号从大到小依次删除。

2.13 QoS

2.13.1 配置 Pod 的资源配额

步骤 1　创建一个文件夹，用于存放 qos 实验相关文件。

```
[root@k8s-master labfile]# mkdir /labfile/qosfile
[root@k8s-master labfile]# cd /labfile/qosfile
```

步骤 2　查看 node 资源。

```
[root@k8s-master qosfile]# kubectl get node k8s-node1 -o yaml
```

回显中可以找到如下一段参数

```
allocatable:
    cpu: "4"
    ephemeral-storage: "42495643169"
    hugepages-1Gi: "0"
    hugepages-2Mi: "0"
    memory: 7906792Ki
    pods: "110"
  capacity:
    cpu: "4"
    ephemeral-storage: 46110724Ki
    hugepages-1Gi: "0"
    hugepages-2Mi: "0"
    memory: 8009192Ki
    pods: "110"
```

步骤 3　创建 namespace

```
[root@k8s-master qosfile]# kubectl create namespace qos-namespace
```

步骤 4　配置一个 BestEffort 级别的 pod

```
[root@k8s-master qosfile]# vim best-pod.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: best-pod
  namespace: qos-namespace
spec:
  containers:
  - name: best-container
    image: nginx
```

步骤 5 创建 pod

```
[root@k8s-master qosfile]# kubectl apply -f best-pod.yaml
```

查看 pod 类型

```
[root@k8s-master qosfile]# kubectl describe pods best-pod -n qos-namespace
QoS Class:       BestEffort
```

步骤 6 创建 Burstable 的 Pod 的 yaml

```
[root@k8s-master qosfile]# vim bur-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: bur-pod
  namespace: qos-namespace
spec:
  containers:
  - name: bur-container
    image: nginx
    resources:
      limits:
        memory: "200Mi"
      requests:
        memory: "100Mi"
```

步骤 7 创建 pod

```
[root@k8s-master qosfile]# kubectl apply -f bur-pod.yaml
```

查看 pod 的 qos 类型

```
[root@k8s-master qosfile]# kubectl describe pod bur-pod -n qos-namespace
QoS Class:       Burstable
```

步骤 8 创建 Guaranteed 类型的 pod 的 yaml

```
[root@k8s-master qosfile]# vim gua-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: gua-pod
  namespace: qos-namespace
spec:
  containers:
  - name: gua-container
    image: nginx
    resources:
```

```
    limits:
      memory: "200Mi"
      cpu: "700m"
    requests:
      memory: "200Mi"
      cpu: "700m"
```

步骤 9　创建 pod

```
[root@k8s-master qosfile]# kubectl apply -f gua-pod.yaml
```

查看 pod 的 qos 类型

```
[root@k8s-master qosfile]# kubectl describe pod gua-pod -n qos-namespace
QoS Class:       Guaranteed
```

2.13.2 测试内存限制

步骤 1　创建一个 yaml 文件，该文件定义了一个 stress 应用的 Pod，并且 Pod 内程序运行会占用超过 Pod 限额的内存。

```
[root@k8s-master qosfile]# vim outmem.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: memory-demo
  namespace: qos-namespace
spec:
  containers:
  - name: memory-demo-ctr
    image: polinux/stress
    resources:
      limits:
        memory: "100Mi"
      requests:
        memory: "50Mi"
    command: ["stress"]
    args: ["--vm", "1", "--vm-bytes", "150M", "--vm-hang", "1"]
```

步骤 2　运行 Pod

```
[root@k8s-master qosfile]# kubectl apply -f outmem.yaml
pod/memory-demo created
```

步骤 3　查看 Pod 状态，可以看到状态是 OOMKilled。

```
[root@k8s-master qosfile]# kubectl get pod -n qos-namespace
NAME          READY   STATUS      RESTARTS   AGE
```

```
memory-demo   0/1     OOMKilled   4          116s
```

查看更详细的 Pod 信息可以看到，Pod 出现问题的原因是 OOMKilled

```
[root@k8s-master qosfile]# kubectl describe pod memory-demo -n qos-namespace
```

```
Name:             memory-demo
Namespace:        qos-namespace
……
    State:        Waiting
      Reason:     CrashLoopBackOff
    Last State:   Terminated
      Reason:     OOMKilled
      Exit Code:  1
      Started:    Fri, 09 Aug 2019 03:18:57 -0400
      Finished:   Fri, 09 Aug 2019 03:18:57 -0400
    Ready:        False
    Restart Count:  4
……
```

2.13.3 测试节点资源限额

步骤 1　创建一个 yaml，该文件定义了一个 pod，但这个 Pod 使用的资源超出节点拥有的资源。

```
[root@k8s-master qosfile]# vim outnode.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: memory-demo2
  namespace: qos-namespace
spec:
  containers:
  - name: memory-demo2-ctr
    image: polinux/stress
    resources:
      limits:
        memory: "1000Gi"
      requests:
        memory: "1000Gi"
    command: ["stress"]
    args: ["--vm", "1", "--vm-bytes", "150M", "--vm-hang", "1"]
```

步骤 2　创建该 Pod

```
[root@k8s-master qosfile]# kubectl apply -f outnode.yaml
```

```
pod/memory-demo2 created
```

步骤 3　查看 Pod 状态，Pod 一直无法创建成功。

```
[root@k8s-master qosfile]# kubectl get pod -n qos-namespace
NAME           READY   STATUS    RESTARTS   AGE
memory-demo2   0/1     Pending   0          13m
```

步骤 4　查看 Pod 创建不成功的原因，可以看到是由于没有节点有足够的资源运行 Pod。

```
[root@k8s-master qosfile]# kubectl describe pod memory-demo2 -n qos-namespace
Name:              memory-demo2
Namespace:         qos-namespace
……
Events:
  Type     Reason            Age                From               Message
  ----     ------            ----               ----               -------
  Warning  FailedScheduling  65s (x19 over 25m)  default-scheduler  0/3 nodes
are available: 3 Insufficient memory.
```

2.14 Kubernetes 资源调度

2.14.1 nodeselector：根据节点选择器调度

步骤 1　查看现有 node 及 label。

```
[root@k8s-master ~]# kubectl get nodes --show-labels
```

```
[root@k8s-master ~]# kubectl get nodes --show-labels
NAME         STATUS   ROLES    AGE   VERSION   LABELS
k8s-master   Ready    master   35d   v1.14.3   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=k8s-master,kubernetes.io/os=linux,node-role.kubernetes.io/master=
k8s-node1    Ready    <none>   35d   v1.14.3   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=k8s-node1,kubernetes.io/os=linux
k8s-node2    Ready    <none>   35d   v1.14.3   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=k8s-node2,kubernetes.io/os=linux
```

步骤 2　给 k8s-node1 添加一个 key 为 disktype，value 为 ssd 的 label。

```
[root@k8s-master ~]# kubectl label nodes k8s-node1 disktype=ssd
```

```
[root@k8s-master ~]# kubectl label nodes k8s-node1 disktype=ssd
node/k8s-node1 labeled
```

步骤 3　查看标签是否被添加

```
[root@k8s-master ~]# kubectl get nodes --show-labels
```

```
[root@k8s-master ~]# kubectl get nodes --show-labels
NAME         STATUS   ROLES    AGE   VERSION   LABELS
k8s-master   Ready    master   35d   v1.14.3   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=k8s-master,kubernetes.io/os=linux,node-role.kubernetes.io/master=
k8s-node1    Ready    <none>   35d   v1.14.3   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux disktype=ssd,kubernetes.io/arch=amd64,kubernetes.io/hostname=k8s-node1,kubernetes.io/os=linux
k8s-node2    Ready    <none>   35d   v1.14.3   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=k8s-node2,kubernetes.io/os=linux
```

如果想要删除标签，命令如下

```
[root@k8s-master ~]# kubectl label nodes k8s-node1 disktype-
```

```
node/node01 labeled
```

```
[root@k8s-master ~]# kubectl get nodes --show-labels
```

#查看标签是否被删除

```
[root@k8s-master ~]# kubectl get nodes --show-labels
NAME         STATUS   ROLES    AGE   VERSION   LABELS
k8s-master   Ready    master   35d   v1.14.3   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=k8s-master,kubernetes.io/os=linux,node-role.kubernetes.io/master=
k8s-node1    Ready    <none>   35d   v1.14.3   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=k8s-node1,kubernetes.io/os=linux
k8s-node2    Ready    <none>   35d   v1.14.3   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=k8s-node2,kubernetes.io/os=linux
```

如果想要修改标签，命令如下

```
[root@k8s-master ~]# kubectl label nodes k8s-node1 disktype=hdd --overwrite
```

```
[root@k8s-master ~]# kubectl get nodes --show-labels
```

#查看标签是否被修改

```
[root@k8s-master ~]# kubectl get nodes --show-labels
NAME         STATUS   ROLES    AGE   VERSION   LABELS
k8s-master   Ready    master   35d   v1.14.3   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=k8s-master,kubernetes.io/os=linux,node-role.kubernetes.io/master=
k8s-node1    Ready    <none>   35d   v1.14.3   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,disktype=hdd,kubernetes.io/arch=amd64,kubernetes.io/hostname=k8s-node1,kubernetes.io/os=linux
k8s-node2    Ready    <none>   35d   v1.14.3   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=k8s-node2,kubernetes.io/os=linux
```

步骤 4　在 master 节点 labfile 目录下创建 scheduler 目录，用于保存配置文件。后续该章节创建的 yaml 文件保存在此处。

```
[root@k8s-master /]# cd /
[root@k8s-master /]# mkdir labfile
[root@k8s-master /]# cd labfile/
[root@k8s-master labfile]# mkdir scheduler
[root@k8s-master labfile]# cd scheduler/
```

步骤 5　创建 pod.yaml 文件

```
[root@k8s-master scheduler]# vi pod.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    env: test
spec:
  containers:
  - name: nginx
    image: nginx
    imagePullPolicy: IfNotPresent
  nodeSelector:
    disktype: hdd
```

步骤 6　部署该 Pod

```
[root@k8s-master scheduler]# kubectl apply -f pod.yaml
```

```
pod/nginx created
```

步骤 7　查看该 Pod 状态

```
[root@k8s-master scheduler]# kubectl get pods -o wide
```

```
[root@k8s-master ~]# kubectl get pods -o wide
NAME        READY   STATUS    RESTARTS   AGE     IP           NODE        NOMINATED NODE   READINESS GATES
jenkins-0   0/1     Running   1          3m37s   10.244.1.6   k8s-node2   <none>           <none>
nginx       1/1     Running   0          115s    10.244.2.6   k8s-node1   <none>           <none>
```

注：可以看到该 Pod 被调度到了拥有字段 hdd 的 label 的 k8s-node1 节点。

步骤 8　修改 Pod 中 nodeselector 字段值

```
[root@k8s-master scheduler]# vim pod.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    env: test
spec:
  containers:
  - name: nginx
    image: nginx
    imagePullPolicy: IfNotPresent
  nodeSelector:
disktype: ssd      #修改了 value 值
```

步骤 9　先将已经部署在 k8s-node1 的 name 为 nginx 的 Pod 删除

```
[root@k8s-master scheduler]# kubectl delete -f pod.yaml
```

```
Pod "nginx" deleted
```

步骤 10　重新部署该 Pod

```
[root@k8s-master scheduler]# kubectl apply -f pod.yaml
```

```
pod/nginx created
```

步骤 11　查看该 Pod 状态

```
[root@k8s-master ~]# kubectl get pods -o wide
```

```
[root@k8s-master ~]# kubectl get pods -o wide
NAME        READY   STATUS    RESTARTS   AGE     IP           NODE        NOMINATED NODE   READINESS GATES
jenkins-0   1/1     Running   1          6m22s   10.244.1.6   k8s-node2   <none>           <none>
nginx       0/1     Pending   0          15s     <none>       <none>      <none>           <none>
```

可以看到该 Pod 处于 pending 状态

步骤 12　我们使用 kubectl describe pods 命令查看一下该 pod 的 event

```
[root@k8s-master ~]# kubectl describe pods nginx
```

```
Events:
  Type     Reason            Age                 From               Message
  ----     ------            ----                ----               -------
  Warning  FailedScheduling  54s (x4 over 3m40s)  default-scheduler  0/3 nodes are available: 3 node(s) didn't match node s
elector.
```

可以看到 Events 中记录了一个 Reason 为 FailedScheduling 的 event

步骤 13 修改 pod 中的标签，重新部署后成功调度，该步骤略

2.14.2 Pod 资源亲和调度

步骤 1 编写 Pod1 YAML 文件

```
[root@k8s-master scheduler]# vim pod1.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-first-affinity
  namespace: default
  labels:
    app: myapp1
spec:
  containers:
  - name: nginx
    image: nginx
    imagePullPolicy: IfNotPresent
```

步骤 2 编写 Pod2 YAML 文件

```
[root@k8s-master scheduler]# vim pod2.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-second-affinity
spec:
  affinity:
    podAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:      #硬亲和性
      - labelSelector:
          matchExpressions:           #匹配规则
          - key: app
            operator: In
            values:
            - myapp1
        topologyKey: kubernetes.io/hostname        #亲和性生效的作用域，亲和性要求相关的Pod运行在同一位置，这里的“位置”实际上取决于节点的位置拓扑，拓扑的方式不同，Pod是否在同一位置的判定结果也会有所不同。
  containers:
  - name: nginx
    image: nginx
    imagePullPolicy: IfNotPresent
```

步骤 3　部署 Pod1，并查看该 Pod 详细信息

```
[root@k8s-master scheduler]# # kubectl apply -f pod1.yaml
pod/pod-first-affinity created
```

```
[root@k8s-master ~]# kubectl get pods -o wide
[root@k8s-master ~]# kubectl get pods -o wide
NAME                READY   STATUS    RESTARTS   AGE     IP            NODE        NOMINATED NODE   READINESS GATES
jenkins-0           1/1     Running   0          6m36s   10.244.1.7    k8s-node2   <none>           <none>
pod-first-affinity  1/1     Running   0          96s     10.244.2.8    k8s-node1   <none>           <none>
```

注：该 pod 被调度到了 k8s-node1

步骤 4　部署 Pod2，并查看该 Pod 详细信息

```
[root@k8s-master scheduler]# kubectl apply -f pod2.yaml
pod/pod-second-affinity created created
```

```
[root@k8s-master ~]# kubectl get pods -o wide
[root@k8s-master ~]# kubectl get pods -o wide
NAME                READY   STATUS    RESTARTS   AGE     IP            NODE        NOMINATED NODE   READINESS GATES
jenkins-0           1/1     Running   0          9m55s   10.244.1.7    k8s-node2   <none>           <none>
pod-first-affinity  1/1     Running   0          4m55s   10.244.2.8    k8s-node1   <none>           <none>
pod-second-affinity 1/1     Running   0          2m58s   10.244.2.10   k8s-node1   <none>           <none>
```

可以看到两个 pod 被调度到了同一个 node 上，即 k8s-node1

步骤 5　修改 Pod2 匹配规则，重新部署

```
#先删除之前创建的 pod2
[root@k8s-master ~]# kubectl delete -f pod2.yaml

#编辑 pod2.yaml
[root@k8s-master scheduler]# vim pod2.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-second-affinity
spec:
  affinity:
    podAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
          - key: myapp                #修改了 key 值
            operator: In
            values:
            - myapp1
        topologyKey: kubernetes.io/hostname
containers:
  - name: nginx
    image: nginx
    imagePullPolicy: IfNotPresent
```

```
[root@k8s-master scheduler]# kubectl apply -f pod2.yaml
pod/pod-second-affinity created
```

步骤 6　查看修改之后 Pod2 的状态

```
[root@k8s-master ~]# kubectl get pods -o wide
[root@k8s-master ~]# kubectl get pods -o wide
NAME                  READY   STATUS    RESTARTS   AGE   IP            NODE        NOMINATED NODE   READINESS GATES
jenkins-0             1/1     Running   0          27m   10.244.1.7    k8s-node2   <none>           <none>
pod-first-affinity    1/1     Running   0          22m   10.244.2.8    k8s-node1   <none>           <none>
pod-second-affinity   0/1     Pending   0          6s    <none>        <none>      <none>           <none>
```

可以看到 Pod2 的状态为 pending

步骤 7　查看 Pod2 的 Event

```
[root@k8s-master ~]# kubectl describe pods pod-second-affinity
Events:
  Type     Reason            Age                  From               Message
  ----     ------            ----                 ----               -------
  Warning  FailedScheduling  24s (x3 over 3m17s)  default-scheduler  0/3 nodes are available: 1 node(s) had taints that the
 pod didn't tolerate, 2 node(s) didn't match pod affinity rules, 2 node(s) didn't match pod affinity/anti-affinity.
```

可以看到 Events 中记录了一个 Reason 为 FailedScheduling 的 event

2.14.3 创建 Kubernetes 调度优先级 PriotityClass

步骤 1　创建 PriorityClass 对象

```
[root@k8s-master scheduler]# vim PriorityClass.yaml
```

```
apiVersion: scheduling.k8s.io/v1beta1
kind: PriorityClass
metadata:
  name: high-priority
value: 1000000
globalDefault: false
description: "This priority class should be used for XYZ service pods only."
```

```
[root@k8s-master scheduler]# kubectl apply -f PriorityClass.yaml
priorityclass.scheduling.k8s.io/high-priority created
[root@k8s-master scheduler]# kubectl get priorityclasses
[root@k8s-master ~]# kubectl apply -f PriorityClass.yaml
priorityclass.scheduling.k8s.io/high-priority created
[root@k8s-master ~]#
[root@k8s-master ~]#
[root@k8s-master ~]#
[root@k8s-master ~]# kubectl get priorityclasses.scheduling.k8s.io
NAME                      VALUE        GLOBAL-DEFAULT   AGE
high-priority             1000000      false            24s
system-cluster-critical   2000000000   false            35d
system-node-critical      2000001000   false            35d
```

步骤 2　部署 Pod，声明使用该 PriorityClass 对象

```
[root@k8s-master scheduler]# vim prioritypod.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx1
  labels:
    env: test
spec:
  containers:
  - name: nginx
    image: nginx
    imagePullPolicy: IfNotPresent
  priorityClassName: high-priority
```

```
[root@k8s-master scheduler]# kubectl apply -f prioritypod.yaml
pod/nginx1 created
```

步骤 3 查看该 Pod 相关信息

```
[root@k8s-master ~]# kubectl describe pods nginx1
```

```
[root@k8s-master ~]# kubectl describe pod nginx
Name:               nginx
Namespace:          default
Priority:           1000000
PriorityClassName:  high-priority
Node:               k8s-node1/192.168.137.22
Start Time:         Wed, 24 Jul 2019 15:25:38 +0800
Labels:             env=test
Annotations:        kubectl.kubernetes.io/last-applied-configuration:
                      {"apiVersion":"v1","kind":"Pod","metadata":{"annotations":{},"labels":{"env":"test"},"name":
"},"spec":{"contai...
Status:             Running
IP:                 10.244.2.12
Containers:
```

2.14.4 实训任务

需求：有一个 web-server 有 2 个实例，该 web-server 会使用到 redis 做为缓存。先需要将 redis 调度到和 web-server 同一个 node。

步骤 1 部署 redis，label app=store 保证 redis 和 web-server 部署到相同的 node。

步骤 2 部署 web-server，与 redis 部署到一起，但是 web-server 之间不部署到一起。

2.15 Kubernetes Dashboard

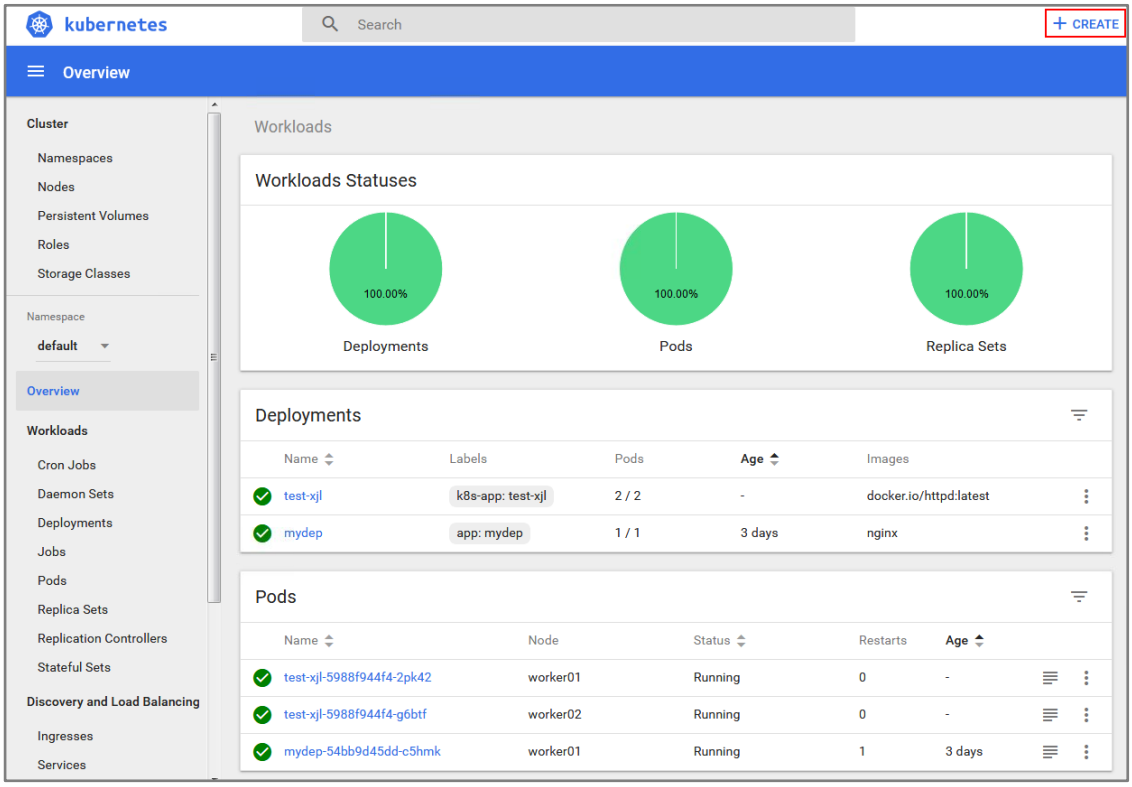
2.15.1 登录 Dashboard

在本实验中，使用以下格式登录浏览器，输入 URL： “k8s-master 的 IP 地址:30090”,这里是 192.168.137.11:30090

具体可回顾搭建指南中的 dashboard.yaml（使用 Nodeport 方式暴露 URL）

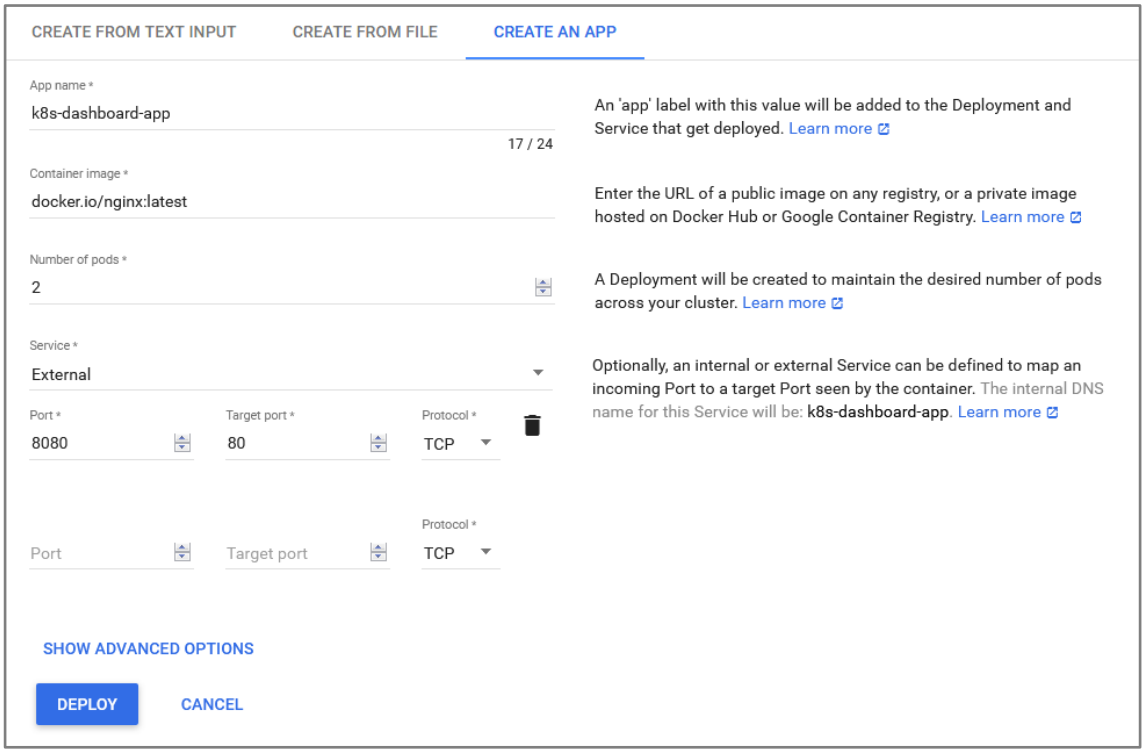
2.15.2 “CREATE AN APP”方式部署 Deployment

步骤 1 进入 Kubernetes Dashboard 界面，点击右上角“+CREATE”按钮，进入“Resource creation”界面。



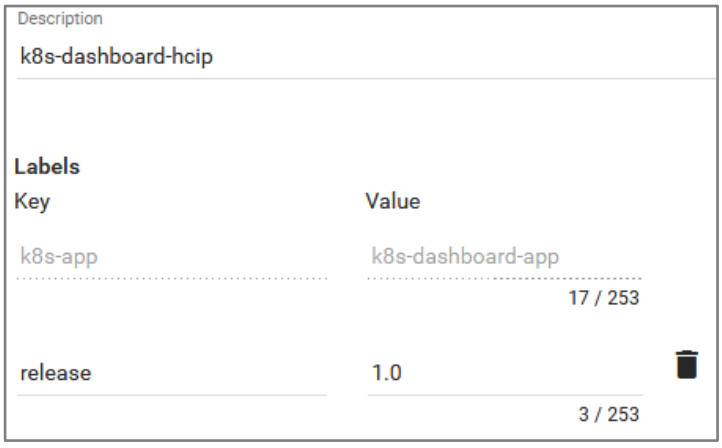
步骤 2 选择“CREATE AN APP”，基本信息填写如下规划参数：

- App name：填写“k8s-dashboard-app”。
- Container image：填写“docker.io/nginx:latest”。
- Number of pods：填写“2”。
- Service：选择“External”。
- Port：填写“8080”。
- Target port：填写“80”。
- Protocol：选择“TCP”。



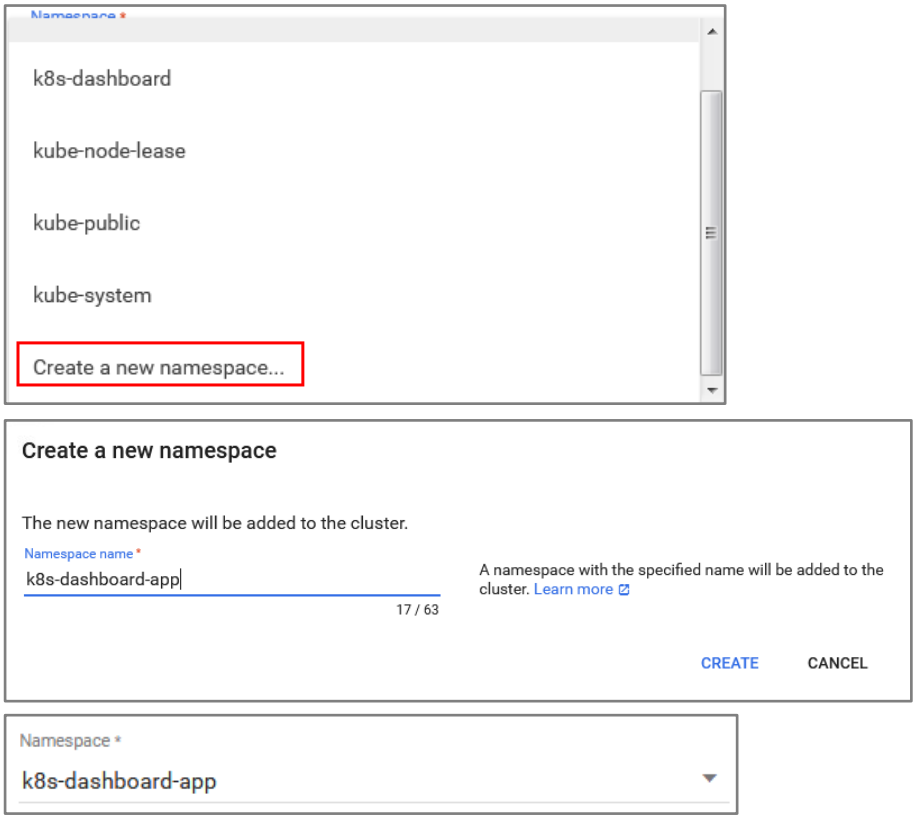
步骤 3　点击“SHOW ADVANCED OPTIONS”，高级选项填写如下规划参数：

- Description：填写“k8s-dashboard-hcip”。
- Labels：Key 填写“release”，Value 填写“1.0”。



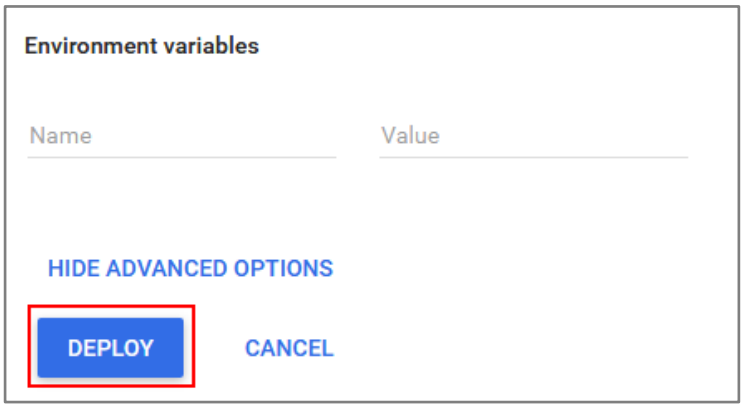
- Namespace：点击右边“ ▼ ”，在下拉列表中，点击“Create a new namespace”，弹出窗口中，填写“k8s-dashboard-app”后，点击“CREATE”。



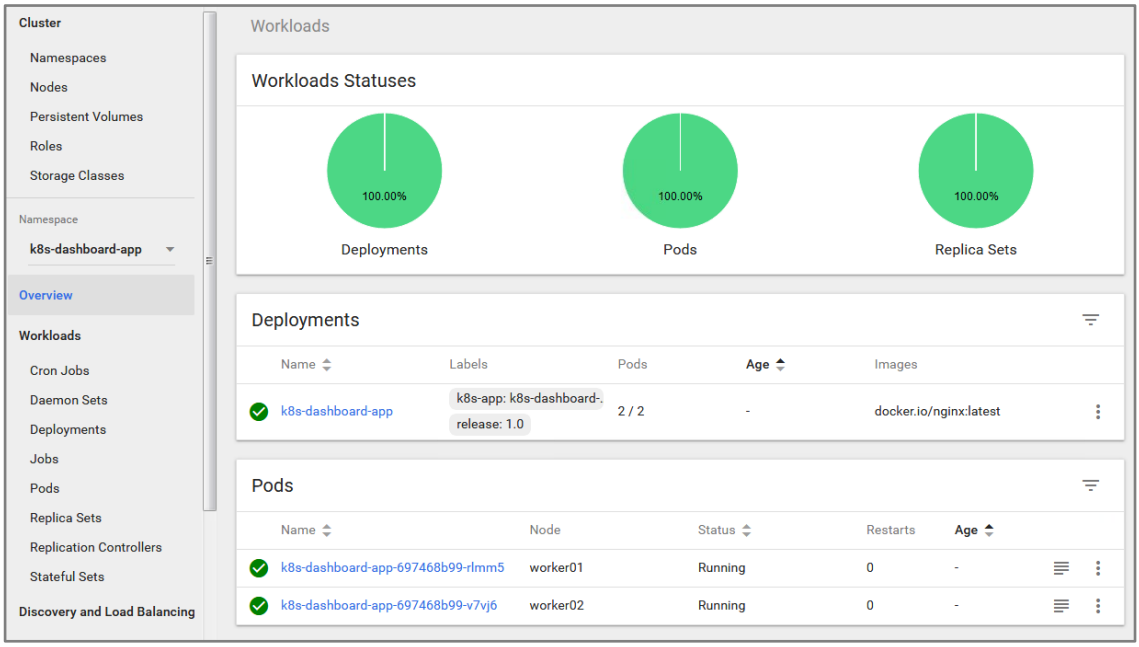


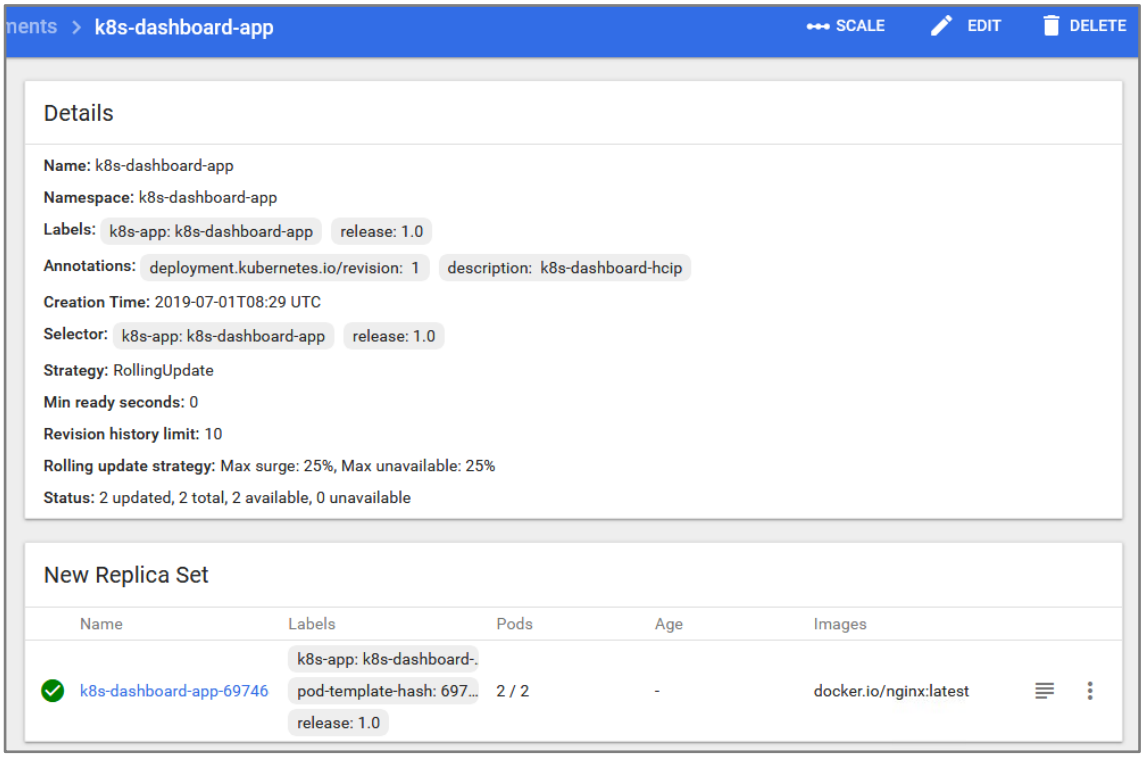
- Image Pull Secret：只有私有镜像才需要填写此项，本实验 pull 的公有镜像，无需填写。
- 其他参数无需填写，也可以根据自己的需要进行填写。

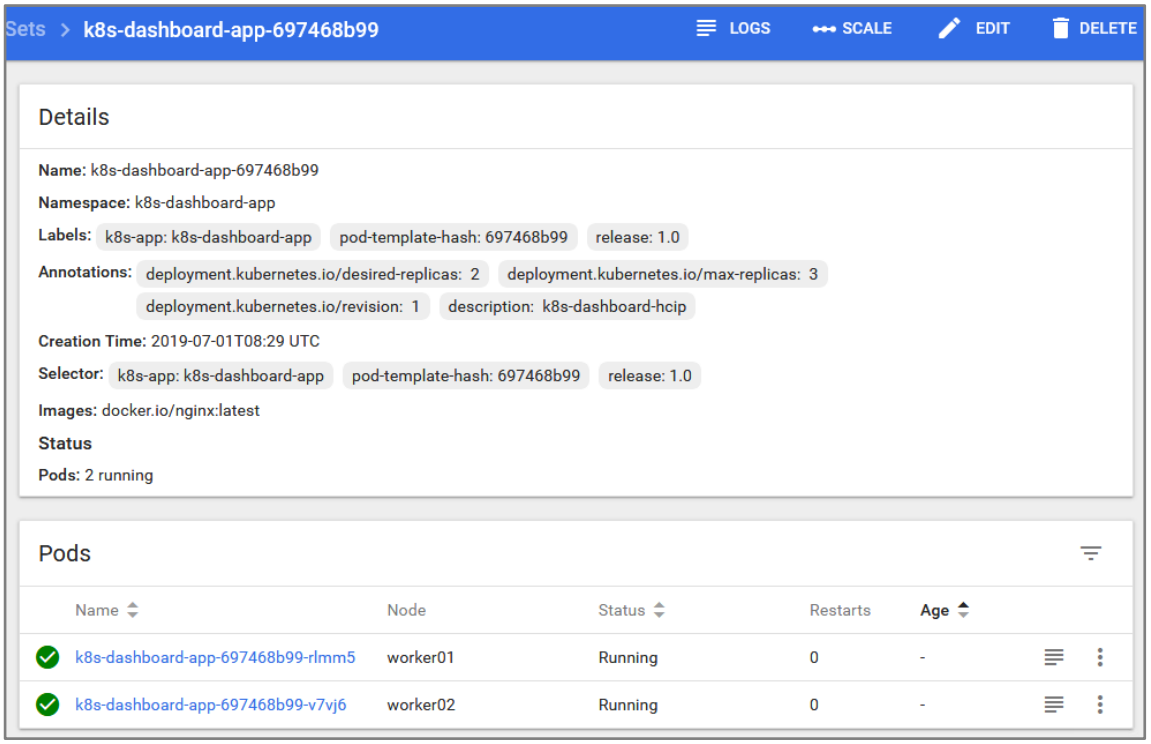
步骤 4　参数填写完毕后，点击“DEPLOY”。

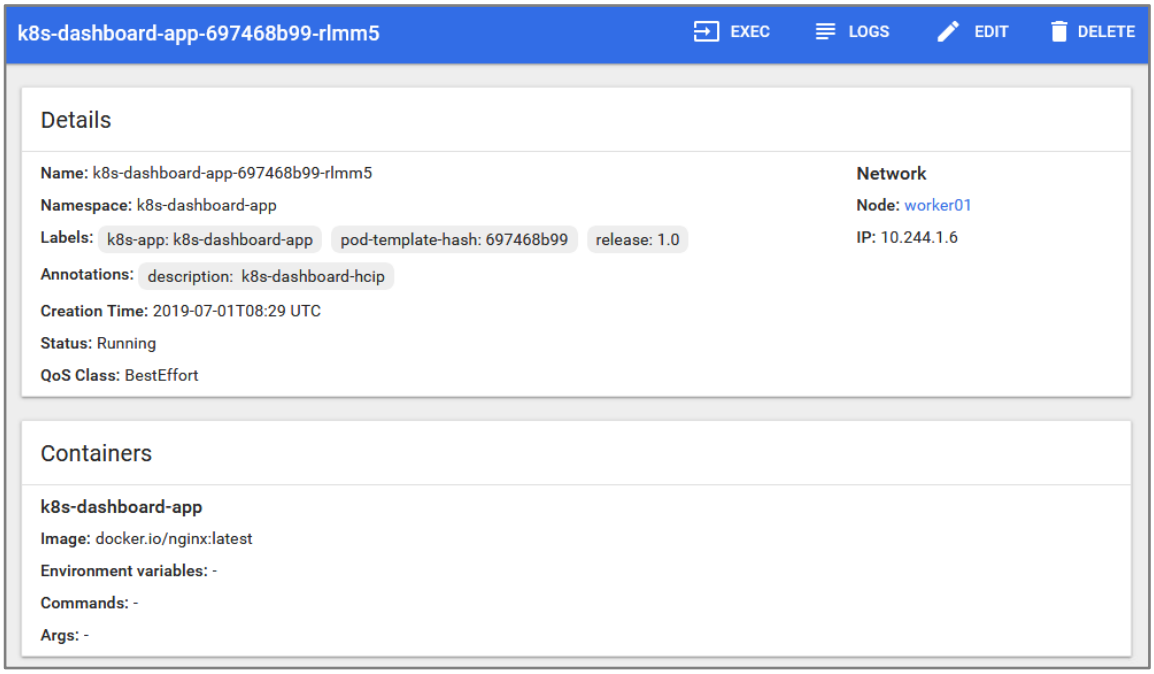


步骤 5　点击创建的应用的名称，进入对应的详情页。


Cluster
Namespaces
Nodes
Persistent Volumes
Roles
Storage Classes
Namespace
k8s-dashboard-app
Overview
Workloads
Cron Jobs
Daemon Sets
Deployments
Jobs
Pods
Replica Sets
Replication Controllers
Stateful Sets
Discovery and Load Balancing
Workloads
Workloads Statuses
100.00%
Deployments
100.00%
Pods
100.00%
Replica Sets
Deployments
Name
Labels
Pods
Age
Images
k8s-dashboard-app
k8s-app: k8s-dashboard-.
release: 1.0
2 / 2
-
docker.io/nginx:latest
Pods
Name
Node
Status
Restarts
Age
k8s-dashboard-app-697468b99-rlmm5
worker01
Running
0
-
k8s-dashboard-app-697468b99-v7vj6
worker02
Running
0
-

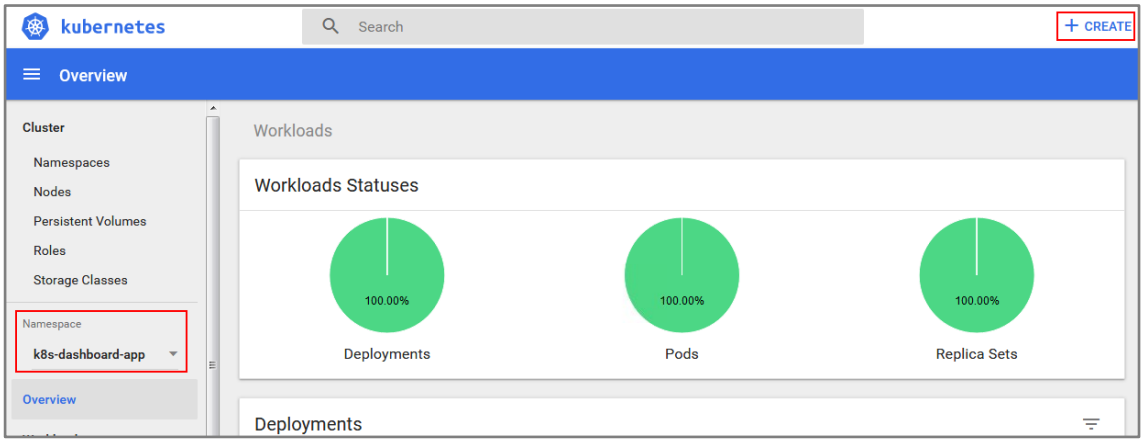

nents > k8s-dashboard-app
SCALE
EDIT
DELETE
Details
Name: k8s-dashboard-app
Namespace: k8s-dashboard-app
Labels: k8s-app: k8s-dashboard-app release: 1.0
Annotations: deployment.kubernetes.io/revision: 1 description: k8s-dashboard-hcip
Creation Time: 2019-07-01T08:29 UTC
Selector: k8s-app: k8s-dashboard-app release: 1.0
Strategy: RollingUpdate
Min ready seconds: 0
Revision history limit: 10
Rolling update strategy: Max surge: 25%, Max unavailable: 25%
Status: 2 updated, 2 total, 2 available, 0 unavailable
New Replica Set
Name
Labels
Pods
Age
Images
k8s-dashboard-app-69746
k8s-app: k8s-dashboard-.
pod-template-hash: 697...
release: 1.0
2 / 2
-
docker.io/nginx:latest





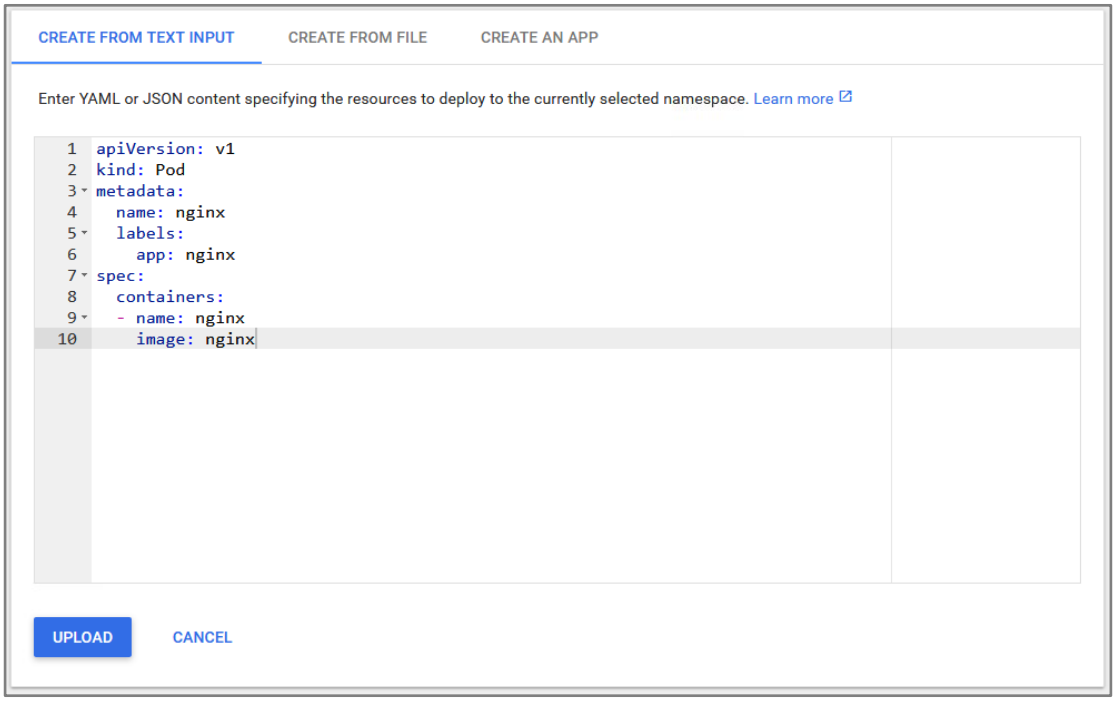
2.15.3 “CREATE FROM TEXT INPUT”方式部署 Pod

步骤 1 进入 Kubernetes Dashboard 界面，Namespace 选择 Dashboard 实验第一小节创建的“k8s-dashboard-app”，选择后点击，右上角“+CREATE”按钮。



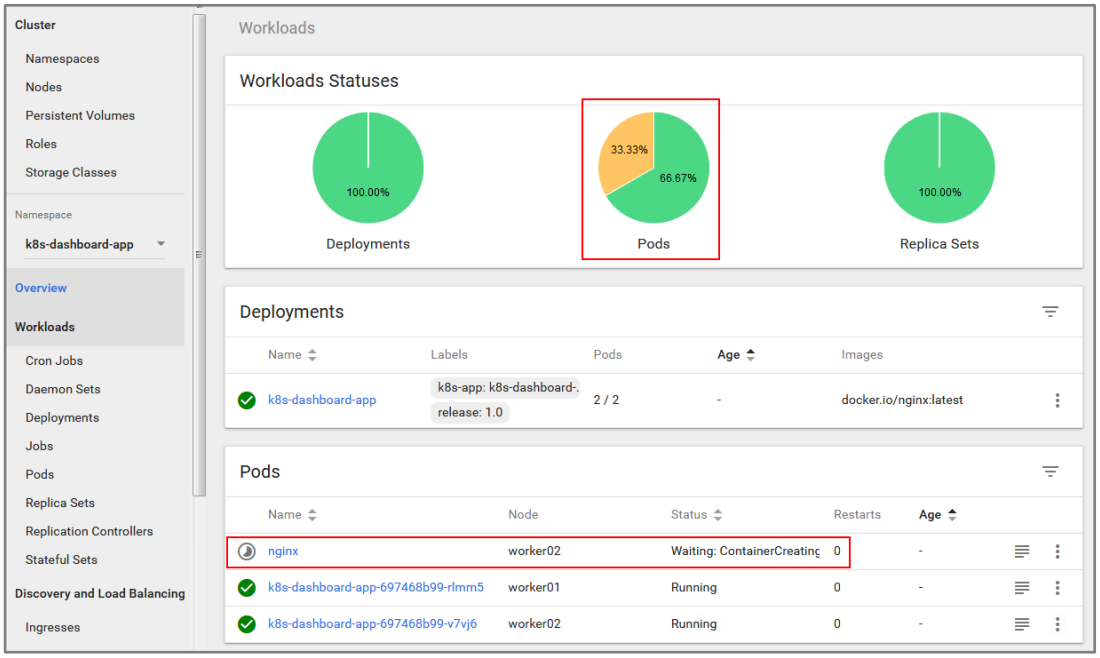
步骤 2　点击“CREATE FROM TEXT INPUT”，输入如下文本信息：

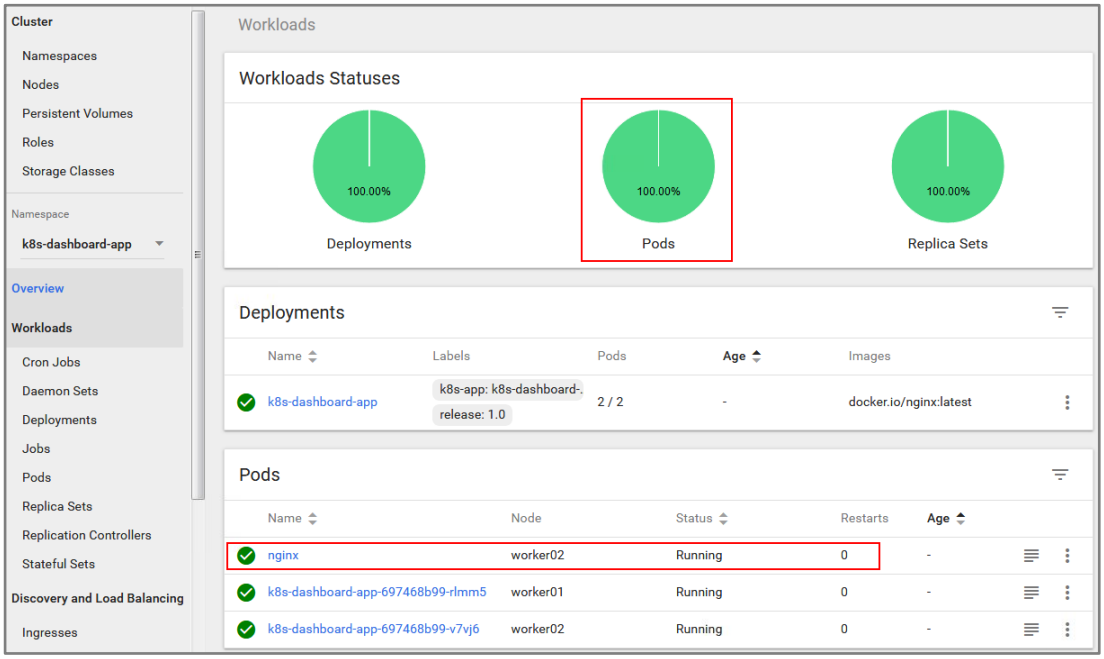
```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  containers:
  - name: nginx
    image: nginx
```



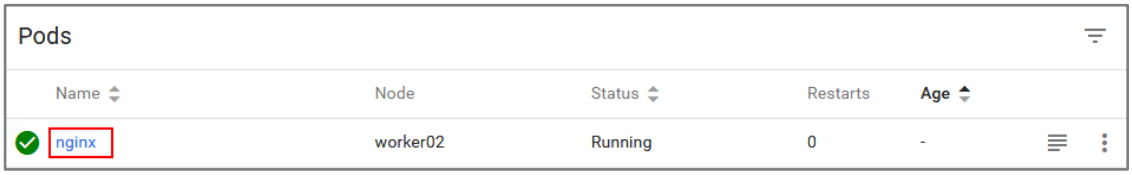
步骤 3　输入完成后，点击“UPLOAD”，等待 Pod 创建成功。

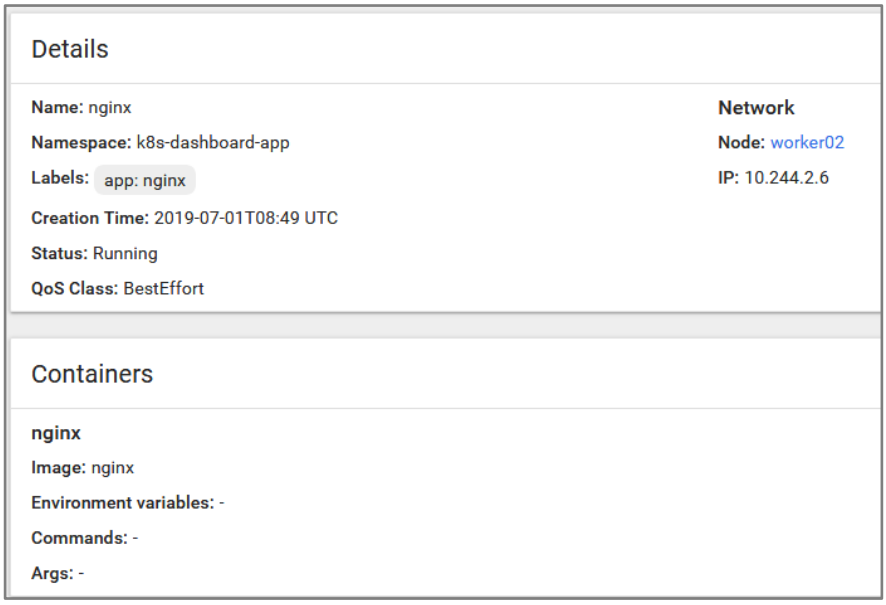






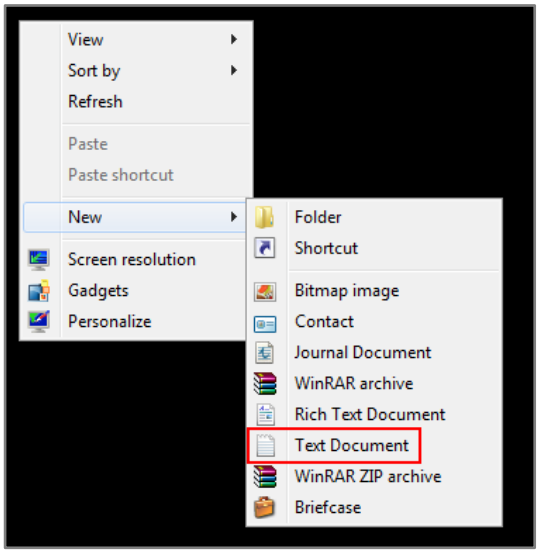
步骤 4　点击刚刚创建的 Pod 名称，可以看到 Pod 的详细信息。

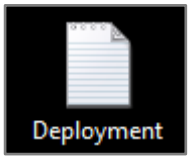




2.15.4 “CREATE FROM FILE”方式部署 Deployment

步骤 1 在客户端（以 Windows 7 PC 机为例）本地，右击新建一个名称为“Deployment”的 txt 文件。



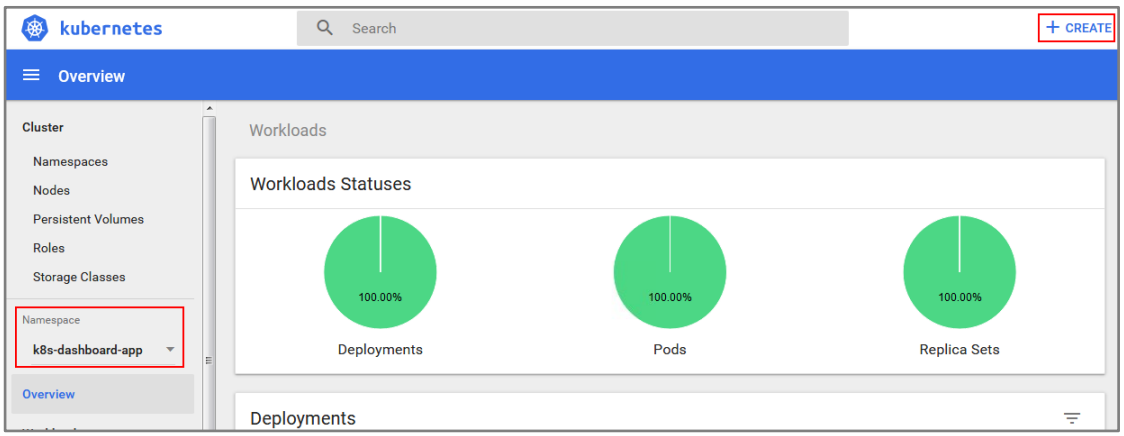


步骤 2 双击打开 txt 文件，并在文件中添加如下图所示文本，并保存。

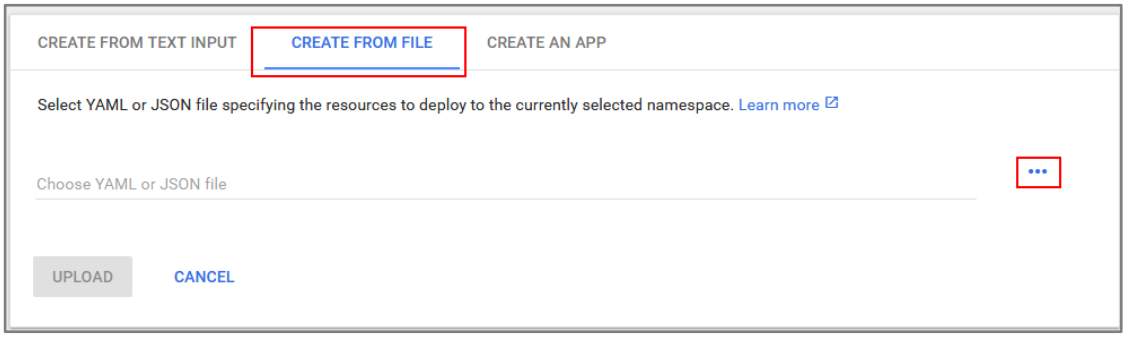
```
apiVersion: apps/v1
kind: Deployment
metadata:
```

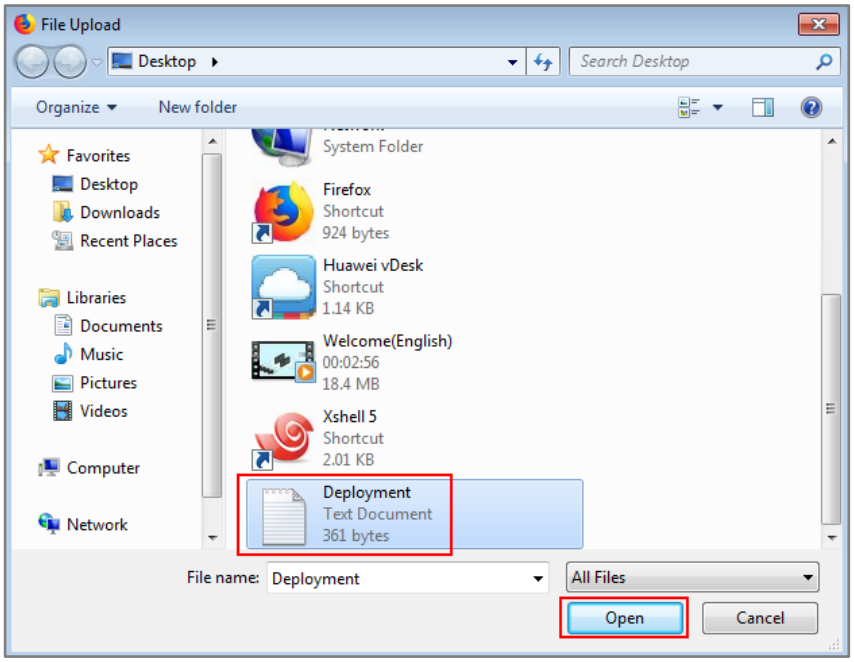
```
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.7.9
        ports:
        - containerPort: 80
```

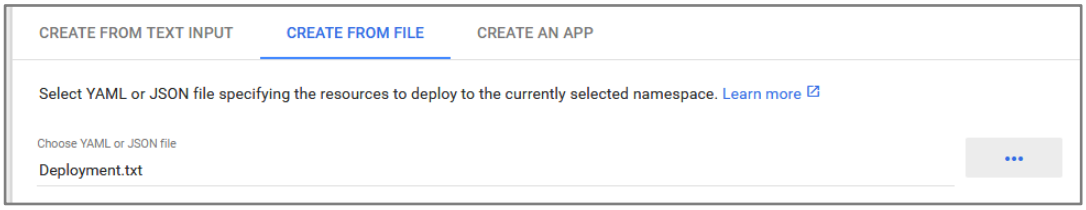
步骤 3　进入 Kubernetes Dashboard 界面，Namespace 选择 Dashboard 实验第一小节创建的“k8s-dashboard-app”，选择后点击，右上角“+CREATE”按钮。



步骤 4　选择“CREATE FROM FILE”,点击“ ••• ”，选中刚刚创建保存的 txt 文件。

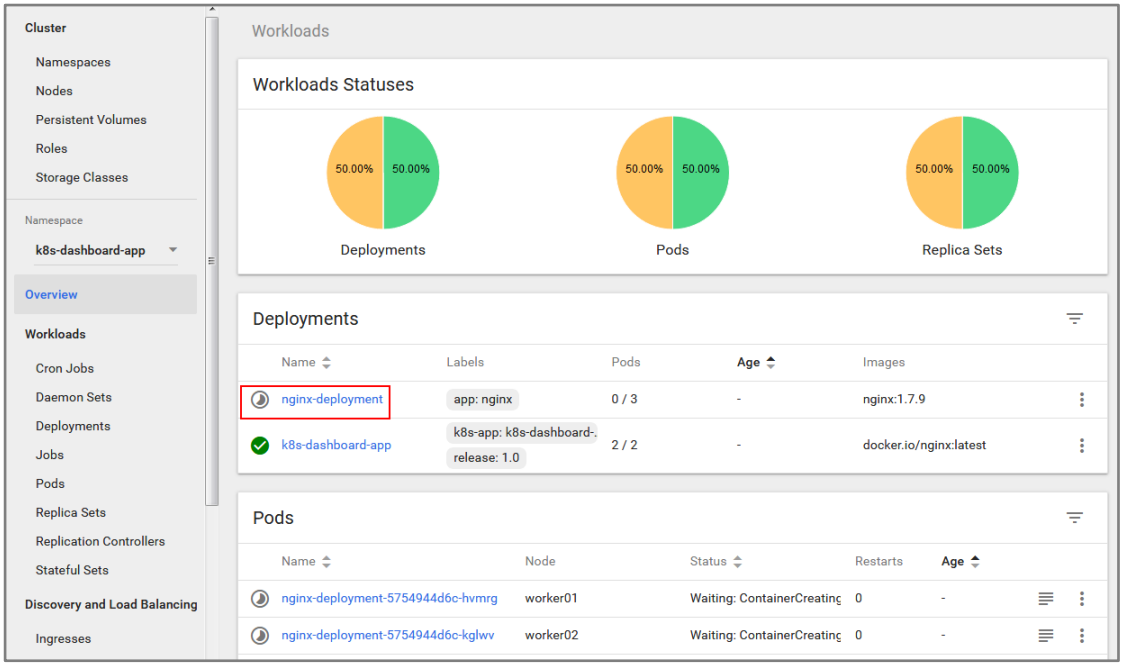




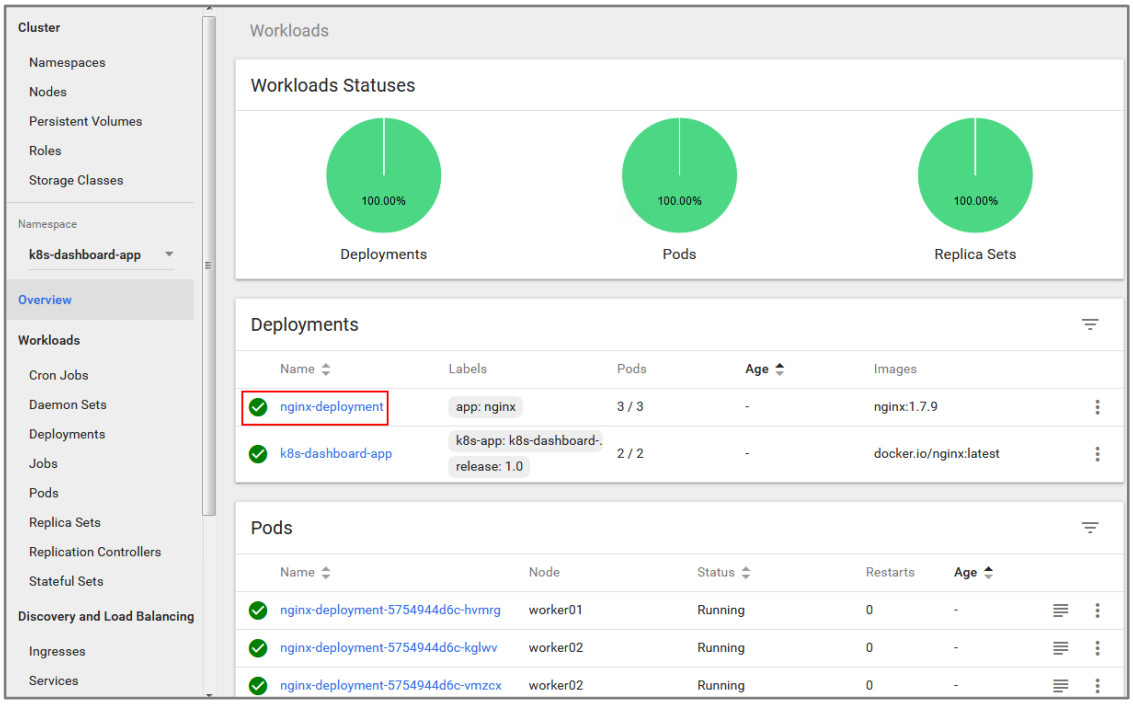


步骤 5　点击“UPLOAD”，等待 Deployment 创建成功。





步骤 6　点击刚刚创建的 Deployment 名称，可以查看 Deployment 详细信息。



2.16 Helm 包管理工具

2.16.1 自定义创建 Chart

步骤 1　生成一个空的 Charts。

```
[root@k8s-master ~]# mkdir -p /root/.helm/cache/archive
[root@k8s-master ~]# cd /root/.helm/cache/archive
[root@k8s-master archive]# helm create mychart
Creating mychart
```

注：该命令能够在当前目录中创建一个名为 mychart 的图表，建议进入 chart 包下载默认路径下创建，默认路径为/root/.helm/cache/archive

```
[root@k8s-master ~]# cd /root/.helm/cache/archive/
[root@k8s-master archive]# ls
[root@k8s-master archive]# helm create mychart
Creating mychart
[root@k8s-master archive]#
[root@k8s-master archive]#
[root@k8s-master archive]#
[root@k8s-master archive]# ls
mychart
```

查看一下该 Charts 文件的目录结构，有以下几个核心文件：

```
[root@k8s-master archive]# tree mychart/
```

```
[root@k8s-master archive]# tree mychart/
mychart/
├── charts
├── Chart.yaml
├── templates
│   ├── deployment.yaml
│   ├── _helpers.tpl
│   ├── ingress.yaml
│   ├── NOTES.txt
│   └── service.yaml
└── values.yaml

2 directories, 7 files
```

进入以下目录查看 Charts 元数据文件：

```
[root@k8s-master ~]# cd /root/.helm/cache/archive/mychart
[root@k8s-master mychart]# cat Chart.yaml
```

```
[root@k8s-master mychart]# pwd
/root/.helm/cache/archive/mychart
[root@k8s-master mychart]# cat Chart.yaml
apiVersion: v1
appVersion: "1.0"
description: A Helm chart for Kubernetes
name: mychart
version: 0.1.0
```

以下为 values.yaml 文件内容中的一段，可以看到定义了一个可以直接安装容器化 **Nginx** 应用的 Charts：

```
replicaCount: 1

image:
  repository: nginx
  tag: stable
  pullPolicy: IfNotPresent

nameOverride: ""
fullnameOverride: ""

service:
  type: ClusterIP
  port: 80
```

步骤 2　修改 Charts 以部署自定义服务。

```
[root@k8s-master mychart]# vim values.yaml
```

```
service:
  type: NodePort
  port: 60000
```

```
ingress:
  enabled false
  annotations: {}
    # kubernetes.io/ingress.class: nginx
    # kubernetes.io/tls-acme: "true"
  path: /
  hosts:
    - chart-example.local
  tls: []
  #  - secretName: chart-example-tls
  #    hosts:
  #      - chart-example.local
```

注：此处故意漏掉了 enabled 后的“:”

步骤 3　检查 chart 是否有语法错误，注意在创建的 chart 所在的目录运行。

```
[root@k8s-master ~]# cd /root/.helm/cache/archive
[root@k8s-master archive]# helm lint mychart
```

```
[root@k8s-master mychart]# helm lint ../mychart/
==> Linting ../mychart/
[INFO] Chart.yaml: icon is recommended
[ERROR] values.yaml: unable to parse YAML
        error converting YAML to JSON: yaml: line 20: mapping values are not allowed in this context

Error: 1 chart(s) linted, 1 chart(s) failed
```

可以看到 helm lint 检查出了错误所在，修改之后

```
ingress:
  enabled: false
  annotations: {}
    # kubernetes.io/ingress.class: nginx
    # kubernetes.io/tls-acme: "true"
  path: /
  hosts:
    - chart-example.local
  tls: []
  #  - secretName: chart-example-tls
  #    hosts:
  #      - chart-example.local
```

运行 helm lint 再次检查正常

```
[root@k8s-master archive]# helm lint mychart
```

```
[root@k8s-master archive]# helm lint mychart/
==> Linting mychart/
[INFO] Chart.yaml: icon is recommended

1 chart(s) linted, no failures
```

步骤 4　使用--dry-run 和--debug 参数调试 chart 包。

```
[root@k8s-master ~]# cd /root/.helm/cache/archive
[root@k8s-master archive]# helm install mychart --name myapp --dry-run --debug
```

```
[root@k8s-master archive]# helm install mychart --name myapp --dry-run --debug
[debug] Created tunnel using local port: '32943'

[debug] SERVER: "127.0.0.1:32943"

[debug] Original chart version: ""
[debug] CHART PATH: /root/.helm/cache/archive/mychart

NAME:   myapp
REVISION: 1
RELEASED: Wed Jul 24 17:47:46 2019
CHART: mychart-0.1.0
USER-SUPPLIED VALUES:
{}

COMPUTED VALUES:
affinity: {}
fullnameOverride: ""
image:
  pullPolicy: IfNotPresent
  repository: nginx
  tag: stable
ingress:
  annotations: {}
  enabled: false
  hosts:
  - chart-example.local
```

步骤 5　确认上述输出无误后，移除--dry-run 参数再运行一次 helm install 命令。

```
[root@k8s-master archive]# helm install mychart/ --name myapp
```

```
[root@k8s-master archive]# helm install mychart/ --name myapp
NAME:   myapp
LAST DEPLOYED: Wed Jul 24 17:54:43 2019
NAMESPACE: default
STATUS: DEPLOYED

RESOURCES:
==> v1/Pod(related)
NAME                          READY  STATUS             RESTARTS  AGE
myapp-mychart-7956d98c68-2b6k4  0/1    ContainerCreating  0         0s

==> v1/Service

NAME           AGE
myapp-mychart  0s

==> v1beta2/Deployment
myapp-mychart  0s


NOTES:
1. Get the application URL by running these commands:
  export NODE_PORT=$(kubectl get --namespace default -o jsonpath="{.spec.ports[0].nodePort}" services myapp-mychart)
  export NODE_IP=$(kubectl get nodes --namespace default -o jsonpath="{.items[0].status.addresses[0].address}")
  echo http://$NODE_IP:$NODE_PORT
```

步骤 6　查看 helm 状态。

```
[root@k8s-master archive]# helm status myapp
```

```
[root@k8s-master archive]# helm status myapp
LAST DEPLOYED: Wed Jul 24 17:54:43 2019
NAMESPACE: default
STATUS: DEPLOYED

RESOURCES:
==> v1/Service
NAME           AGE
myapp-mychart  29s

==> v1beta2/Deployment
myapp-mychart  29s

==> v1/Pod(related)

NAME                          READY  STATUS             RESTARTS  AGE
myapp-mychart-7956d98c68-2b6k4  0/1    ContainerCreating  0         29s


NOTES:
1. Get the application URL by running these commands:
  export NODE_PORT=$(kubectl get --namespace default -o jsonpath="{.spec.ports[0].nodePort}" services myapp-mychart)
  export NODE_IP=$(kubectl get nodes --namespace default -o jsonpath="{.items[0].status.addresses[0].address}")
  echo http://$NODE_IP:$NODE_PORT
```

步骤 7　通过 status 中输出的 NOTES 提示运行相关命令验证服务。

```
export NODE_PORT=$([root@k8s-master ~]# kubectl get --namespace default -o
jsonpath="{.spec.ports[0].nodePort}" services myapp-mychart)

export NODE_IP=$([root@k8s-master ~]# kubectl get nodes --namespace default -o
jsonpath="{.items[0].status.addresses[0].address}")

echo http://$NODE_IP:$NODE_PORT
```

```
NOTES:
1. Get the application URL by running these commands:
  export NODE_PORT=$(kubectl get --namespace default -o jsonpath="{.spec.ports[0].nodePort}" services myapp-mychart)
  export NODE_IP=$(kubectl get nodes --namespace default -o jsonpath="{.items[0].status.addresses[0].address}")
  echo http://$NODE_IP:$NODE_PORT

[root@k8s-master archive]# export NODE_PORT=$(kubectl get --namespace default -o jsonpath="{.spec.ports[0].nodePort}" services myapp-m
ychart)
[root@k8s-master archive]# export NODE_IP=$(kubectl get nodes --namespace default -o jsonpath="{.items[0].status.addresses[0].address}
")
[root@k8s-master archive]# echo http://$NODE_IP:$NODE_PORT
http://192.168.137.21:30549
```

curl 192.168.137.11:30549（此处 IP 地址以实际输出为准）

```
[root@k8s-master archive]# curl http://192.168.137.21:30549
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
    body {
        width: 35em;
        margin: 0 auto;
        font-family: Tahoma, Verdana, Arial, sans-serif;
    }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

2.16.2 将自定义的应用发布到本地 Repository

步骤 1　打包自己定义的应用。

```
[root@k8s-master ~]# cd /root/.helm/cache/archive
[root@k8s-master archive]# helm package mychart
```

```
[root@k8s-master archive]# helm package mychart
Successfully packaged chart and saved it to: /root/.helm/cache/archive/mychart-0.1.0.tgz
```

注：请在 mychart 文件所在位置执行 package 命令

步骤 2　新版的 Helm 在安装的过程中，提供了两个仓库：stable 和 local。

```
[root@k8s-master ~]# helm repo list
```

```
[root@k8s-master archive]# helm repo list
NAME    URL
stable  https://kubernetes.oss-cn-hangzhou.aliyuncs.com/charts
local   http://127.0.0.1:8879/charts
```

Stable 是搭建手册中阿里提供的默认仓库（因 google 仓库无法连接，采用国内阿里的仓库）

local 即本地仓库

步骤 3 开启配置的默认 Repository Server

```
[root@k8s-master archive]# helm serve
```

(该命令执行后光标会悬停在另一行闪烁，请另开一个窗口执行下面的其它命令)

```
[root@k8s-master archive]# helm repo list
NAME    URL
stable  https://kubernetes.oss-cn-hangzhou.aliyuncs.com/charts
local   http://127.0.0.1:8879/charts
[root@k8s-master archive]# helm serve
Regenerating index. This may take a moment.
Now serving you on 127.0.0.1:8879
```

步骤 4 更新仓库

```
[root@k8s-master ~]# helm repo update
```

```
[root@k8s-master ~]# helm repo update
Hang tight while we grab the latest from your chart repositories...
...Skip local chart repository
...Successfully got an update from the "stable" chart repository
Update Complete. ⎈ Happy Helming!⎈
```

步骤 5 查看我们自定义的 chart

```
[root@k8s-master ~]# helm search mychart
```

```
[root@k8s-master ~]# helm search mychart
NAME            CHART VERSION   APP VERSION     DESCRIPTION
local/mychart   0.1.0           1.0             A Helm chart for Kubernetes
```

2.16.3 helm 升级和回退一个应用

步骤 1 打包自己定义的应用。修改 Chart.yaml 文件，将版本号从 0.1.0 修改为 0.2.0, 然后使用 helm package 命令打包并发布到本地仓库。

```
[root@k8s-master ~]# cd /root/.helm/cache/archive
[root@k8s-master archive]# vi mychart/Chart.yaml
```

```
[root@k8s-master archive]# vi mychart/Chart.yaml
apiVersion: v1
appVersion: "1.0"
description: A Helm chart for Kubernetes
name: mychart
version: 0.2.0
```

```
[root@k8s-master archive]# helm package mychart
```

```
[root@k8s-master archive]# helm package mychart
Successfully packaged chart and saved it to: /root/.helm/cache/archive/mychart-0.2.0.tgz
```

步骤 2 查询本地仓库中 mychart 的信息，注意加上-l 参数显示所有版本号。

```
[root@k8s-master ~]# helm search mychart  -l
```

```
[root@k8s-master archive]# helm search mychart -l
NAME            CHART VERSION   APP VERSION     DESCRIPTION
local/mychart   0.2.0           1.0             A Helm chart for Kubernetes
local/mychart   0.1.0           1.0             A Helm chart for Kubernetes
```

步骤 3　使用 helm upgrade 已经部署的 release，可以通过--version 来指定版本号，如果未有指定则自动升级到最新版本。

```
[root@k8s-master archive]# helm list    #查询使用 mychart 部署的 release 名称
```

```
[root@k8s-master archive]# helm list
NAME    REVISION        UPDATED                         STATUS          CHART           APP VERSION     NAMESPACE
myapp   1               Wed Jul 24 17:54:43 2019        DEPLOYED        mychart-0.1.0   1.0             default
```

```
[root@k8s-master archive]# helm upgrade myapp local/mychart
```

```
[root@k8s-master archive]# helm upgrade myapp local/mychart
Release "myapp" has been upgraded. Happy Helming!
LAST DEPLOYED: Wed Jul 24 18:13:06 2019
NAMESPACE: default
STATUS: DEPLOYED

RESOURCES:
==> v1/Service
NAME           AGE
myapp-mychart  18m

==> v1beta2/Deployment
myapp-mychart  18m

==> v1/Pod(related)

NAME                            READY  STATUS   RESTARTS  AGE
myapp-mychart-7956d98c68-2b6k4  1/1    Running  0         18m


NOTES:
1. Get the application URL by running these commands:
  export NODE_PORT=$(kubectl get --namespace default -o jsonpath="{.spec.ports[0].nodePort}" services myapp-mychart)
  export NODE_IP=$(kubectl get nodes --namespace default -o jsonpath="{.items[0].status.addresses[0].address}")
  echo http://$NODE_IP:$NODE_PORT
```

步骤 4　使用 helm history 查看 release 的历史

```
[root@k8s-master archive]# helm history myapp
```

```
[root@k8s-master archive]# helm history myapp
REVISION        UPDATED                         STATUS          CHART           DESCRIPTION
1               Wed Jul 24 17:54:43 2019        SUPERSEDED      mychart-0.1.0   Install complete
2               Wed Jul 24 18:13:06 2019        DEPLOYED        mychart-0.2.0   Upgrade complete
```

步骤 5　回退一个应用

```
[root@k8s-master archive]# helm rollback myapp 1
```

```
Rollback was a success! Happy Helming!
```

步骤 6　回退后查看 release 历史

```
[root@k8s-master archive]# helm history myapp
```

```
[root@k8s-master archive]# helm history myapp
REVISION        UPDATED                         STATUS          CHART           DESCRIPTION
1               Wed Jul 24 17:54:43 2019        SUPERSEDED      mychart-0.1.0   Install complete
2               Wed Jul 24 18:13:06 2019        SUPERSEDED      mychart-0.2.0   Upgrade complete
3               Wed Jul 24 18:14:19 2019        DEPLOYED        mychart-0.1.0   Rollback to 1
```

2.16.4 搭建 helm 私有仓库

步骤 1　创建一个 repo 目录，用来存放 chart

```
[root@k8s-master ~]# mkdir -p /dcos/appstore/local-repo
```

步骤 2　启动本地 repo 仓库的服务

#IP 地址以实际情况为准（此处执行后光标在另一行悬停闪烁，请另外打开一个终端执行下列命令）

```
[root@k8s-master ~]# helm serve --address 192.168.137.11:8880 --repo-path
/dcos/appstore/local-repo
Regenerating index. This may take a moment.
Now serving you on 192.168.137.11:8880
```

步骤 3 将私有仓库添加到本地

```
[root@k8s-master ~]# helm repo add local-repo http://192.168.137.11:8880
"local-repo" has been added to your repositories
```

步骤 4 查看是否添加成功

```
[root@k8s-master ~]# helm repo list
NAME            URL
local           http://127.0.0.1:8879/charts
stable          https://kubernetes.oss-cn-hangzhou.aliyuncs.com/charts
local-repo      http://192.168.137.11:8880
```

步骤 5 拷贝一份 chart 包到对应路径下

```
[root@k8s-master ~]# cp ~/.helm/cache/archive/mychart-0.1.0.tgz
/dcos/appstore/local-repo
```

步骤 6 生成仓库 index 文件

```
[root@k8s-master ~]# cd /dcos/appstore/local-repo
[root@k8s-master local-repo]# helm repo index . --
url=http://192.168.137.11:8880
```

步骤 7 更新仓库

```
[root@k8s-master ~]# helm repo update
Hang tight while we grab the latest from your chart repositories...
...Skip local chart repository
...Successfully got an update from the "local-repo" chart repository
...Successfully got an update from the "stable" chart repository
Update Complete. ⎈ Happy Helming!⎈
```

步骤 8 查看 chart

```
[root@k8s-master ~]# helm search mychart
NAME                    CHART VERSION   APP VERSION     DESCRIPTION
local-repo/mychart      0.1.0           1.0             A Helm chart for Kubernetes
local/mychart           0.2.0           1.0             A Helm chart for Kubernetes
```

2.16.5 实训任务

步骤 1 搭建一个 helm 的私有仓库，使用一个独立的路径作为存储

步骤 2 下载 Mysql 的 chart，使用 inspect 查看使用方法后，参照以下要求部署该 chart

1. 设置数据库密码为 mytest123
2. 根据使用方法中相关描述申请 PV

步骤 3 回滚 Mysql 到上一个版本

2.17 RBAC 权限控制

2.17.1 创建 User Account

步骤 1 查看当前用户认证信息

```
[root@k8s-master ~]# kubectl config view
```

```
输出回显:
apiVersion: v1
clusters:                          #集群信息，可能有多个集群
- cluster:
    certificate-authority-data: DATA+OMITTED
    server: https://192.168.137.11:6443
  name: kubernetes
contexts:              #context 定义 cluster,user,namespace 元组的名称,用来向指定的集群使用提供的认证信息和命名空间向指定的集群发送请求。可以有多个 context
- context:
    cluster: kubernetes
    user: kubernetes-admin
  name: kubernetes-admin@kubernetes
- context:
    cluster: kubernetes
    user: user1
  name: user1@kubernetes
current-context: kubernetes-admin@kubernetes      #当前使用的账号所在的 context
kind: Config
preferences: {}
users:                             #用户清单
- name: kubernetes-admin
  user:
    client-certificate-data: REDACTED
    client-key-data: REDACTED
- name: user1
  user:
    client-certificate-data: REDACTED
   client-key-data: REDACTED
```

从配置中可以看出当前只有一个 cluster，我们现在使用的是名为 kubernetes-admin 的账户，用户列表中除了 kubernetes-admin，还有一个 user1 的用户。

步骤 2 在 kubernetes 默认存放用户秘钥和证书的目录里，为新用户 user2 账号生成秘钥，

```
[root@k8s-master ~]# cd /etc/kubernetes/pki
[root@k8s-master pki]# (umask 077; openssl genrsa -out user2.key 2048)
```

步骤 3 创建证书签署请求

```
[root@k8s-master pki]# openssl req -new -key user2.key -out user2.csr -subj
"/CN=user2/O=kubeusers"
```

步骤 4 使用 kubernetes 集群创建时候的 CA 为用户颁发证书，设置有效时间

```
[root@k8s-master pki]# openssl x509 -req -in user2.csr -CA ca.crt -CAkey ca.key
-CAcreateserial -out user2.crt -days 365
```

```
[root@master pki]# openssl x509 -req -in user2.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out user2.crt -days 365
Signature ok
subject=/CN=user2/O=kubeusers
Getting CA Private Key
```

2.17.2 使用 User Account 设定 kube-config 配置文件

步骤 1 配置客户端证书及秘钥

```
[root@k8s-master ~]# kubectl config set-credentials user2 --client-
certificate=/etc/kubernetes/pki/user2.crt --client-
key=/etc/kubernetes/pki/user2.key
```

```
User "user2" set.
```

步骤 2 配置 user2 的 context，组合 cluster 和 credentials，使得可以通过 user-context 来切换

```
[root@k8s-master ~]# kubectl config set-context user2@kubernetes --
cluster=kubernetes --user=user2
```

```
Context "user2@kubernetes" created
```

步骤 3 上下文切换

#切换至刚添加的新用户

```
[root@k8s-master ~]# kubectl config use-contex kube-user2@kubernetes
```

#切换回 admin 用户

```
[root@k8s-master ~]# kubectl config use-context kubernetes-admin@kubernetes
```

#临时切换

```
[root@k8s-master ~]# kubectl --context=user2@kubernetes get pods
```

步骤 4 使用刚创建未有任何授权的账户进行测试

```
[root@k8s-master ~]# kubectl --context=user2@kubernetes get pods
Error from server (Forbidden): pods is forbidden: User "user2" cannot list
resource "pods" in API group "" in the namespace "default"
```

#可以看到没有权限获取该资源

2.17.3 使用内置的 ClusterRole 绑定新建账户

步骤 1 查看 kubernetes 内建 clusterrole

```
[root@k8s-master ~]# kubectl get clusterrole
```

我们选择 view 这个内建 clusterrole

步骤 2 查看 view 这个 clusterrole 的权限

```
[root@k8s-master ~]# kubectl describe clusterrole view
```

```
namespaces/status                  []    []    [get list watch]
namespaces                         []    []    [get list watch]
persistentvolumeclaims             []    []    [get list watch]
pods/log                           []    []    [get list watch]
pods/status                        []    []    [get list watch]
pods                               []    []    [get list watch]
replicationcontrollers/scale       []    []    [get list watch]
replicationcontrollers/status      []    []    [get list watch]
replicationcontrollers             []    []    [get list watch]
resourcequotas/status              []    []    [get list watch]
resourcequotas                     []    []    [get list watch]
serviceaccounts                    []    []    [get list watch]
```

注：对于大部分资源都是 get、list、watch 的查询权限

步骤 3 在 master 节点创建/labfile/rbac 目录，用于保存配置文件。后续有关本章节的 yaml 文件均保存在此处

```
[root@k8s-master /]# cd /
[root@k8s-master /]# mkdir labfile
[root@k8s-master /]# cd labfile/
[root@k8s-master labfile]# mkdir rbac
[root@k8s-master labfile]# cd rbac/
```

步骤 4 创建一个 ClusterRoleBinding 对象，绑定该 ClusterRole（即 view）

```
[root@k8s-master rbac]# vim ClusterRolebindinguser2.yaml
```

```
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: clusterrolebindinguser2
subjects:
- kind: User
  name: user2
  apiGroup: rbac.authorization.k8s.io
```

```
roleRef:
  kind: ClusterRole
  name: view
  apiGroup: rbac.authorization.k8s.io
```

```
[root@k8s-master rbac]# kubectl apply -f ClusterRolebindinguser2.yaml
clusterrolebinding.rbac.authorization.k8s.io/clusterrolebindinguser2 created
```

步骤 5　再次测试 user2 账号，已经可以获取到相关信息

```
[root@k8s-master ~]# kubectl --context=user2@kubernetes get pods
NAME                              READY   STATUS    RESTARTS   AGE
httpd-686c9d9595-2mq6t            1/1     Running   2          44d
httpd-686c9d9595-clkjj            1/1     Running   2          44d
httpd-686c9d9595-zf7lg            1/1     Running   0          44d
myapp-mychart-7956d98c68-t9kzc    1/1     Running   0          13h
nginx                             1/1     Running   0          19h
pod-first-affinity                1/1     Running   0          24h
pod-second-affinity               0/1     Pending   0          24h
```

2.17.4 创建 Service Account 等相关对象

步骤 1　创建一个名为 mynamespace 的命名空间

```
[root@k8s-master ~]# kubectl create namespace mynamespace
namespace/mynamespace created
```

查看一下 namespace，确认已经创建成功

```
NAME              STATUS   AGE
default           Active   47d
kube-node-lease   Active   47d
kube-public       Active   47d
kube-system       Active   47d
mynamespace       Active   4d
```

步骤 2　编辑 YAML 文件创建一个 Service Account 类型的账户

```
[root@k8s-master rbac]# vim serviceaccount.yaml
```

以下为 yaml 文件中的内容：

```
apiVersion: v1
kind: ServiceAccount
metadata:
  namespace: mynamespace
  name: example-sa
```

步骤 3　创建该 Service Account 账户

```
[root@k8s-master rbac]# kubectl apply -f serviceaccount.yaml
serviceaccount/example-sa created
```

步骤 4　确认账号创建成功

```
[root@k8s-master ~]# kubectl get sa -n mynamespace
[root@k8s-master ~]# kubectl get sa -n mynamespace  example-sa -o yaml
```

```
apiVersion: v1
kind: ServiceAccount
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"ServiceAccount","metadata":{"annotations":{},"name":"example-sa","
  creationTimestamp: "2019-07-01T07:07:34Z"
  name: example-sa
  namespace: mynamespace
  resourceVersion: "5961988"
  selfLink: /api/v1/namespaces/mynamespace/serviceaccounts/example-sa
  uid: e66695be-9bce-11e9-b39d-525400cbb5d4
secrets:
- name: example-sa-token-lbrnr
```

步骤 5　编辑 Role 的 YAML 文件

```
[root@k8s-master rbac]# vim roletemplate.yaml
```

```
kind: Role
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  namespace: mynamespace
  name: example-role
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "watch", "list"]
```

步骤 6　使用该 YAML 文件创建该 Role

```
[root@k8s-master rbac]# kubectl apply -f roletemplate.yaml
```

```
role.rbac.authorization.k8s.io/example-role configured
```

检查 role 对象是否创建成功

```
[root@k8s-master rbac]# kubectl get role example-role -n mynamespace
[root@k8s-master rbac]# kubectl get role example-role -n mynamespace -o yaml
```

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"rbac.authorization.k8s.io/v1","kind":"Role","metadata":{"annotations":{},"
e":"mynamespace"},"rules":[{"apiGroups":[""],"resources":["pods"],"verbs":["get","watch","list"
  creationTimestamp: "2019-06-27T06:09:35Z"
  name: example-role
  namespace: mynamespace
  resourceVersion: "5455581"
  selfLink: /apis/rbac.authorization.k8s.io/v1/namespaces/mynamespace/roles/example-role
  uid: 23184aa6-98a2-11e9-b39d-525400cbb5d4
rules:
- apiGroups:
  - ""
  resources:
  - pods
  verbs:
  - get
  - watch
  - list
```

步骤 7 编写 Rolebinding 的 YAML 文件来为 Service Account 分配权限。

```
[root@k8s-master rbac]# vim RoleBinding.yaml
```

```
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: example-rolebinding
  namespace: mynamespace
subjects:
- kind: ServiceAccount
  name: example-sa
  namespace: mynamespace
roleRef:
  kind: Role
  name: example-role
  apiGroup: rbac.authorization.k8s.io
```

步骤 8 使用该 YAML 创建 RoleBinding。

```
[root@k8s-master rbac]# kubectl apply -f RoleBinding.yaml
rolebinding.rbac.authorization.k8s.io/example-rolebinding configured
```

rolebinding.rbac.authorization.k8s.io/example-rolebinding configured

检查 rolebinding 对象是否创建成功

```
[root@k8s-master rbac]# kubectl get rolebinding -n mynamespace
[root@k8s-master rbac]# kubectl get rolebinding -n mynamespace -o yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"rbac.authorization.k8s.io/v1","kind":"RoleBinding","metadata":{"annotations":{},"n
ing","namespace":"mynamespace"},"roleRef":{"apiGroup":"rbac.authorization.k8s.io","kind":"Role","name":
cts":[{"kind":"ServiceAccount","name":"example-sa","namespace":"mynamespace"}]}
  creationTimestamp: "2019-06-27T13:07:30Z"
  name: example-rolebinding
  namespace: mynamespace
  resourceVersion: "5971562"
  selfLink: /apis/rbac.authorization.k8s.io/v1/namespaces/mynamespace/rolebindings/example-rolebinding
  uid: 85007771-98dc-11e9-b39d-525400cbb5d4
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: example-role
subjects:
- kind: ServiceAccount
  name: example-sa
  namespace: mynamespace
```

2.17.5 创建 Pod 引用该 Service Account

步骤 1 编辑 Pod YAML 文件引用该 Service Account。

```
[root@k8s-master rbac]# vim nginx.yaml
```

```
apiVersion: v1
kind: Pod
```

```
metadata:
  namespace: mynamespace
  name: nginx-test
spec:
  containers:
  - name: nginx
    image: nginx:1.7.9
  serviceAccountName: example-sa
```

步骤 2　使用该 YAML 文件创建 Pod

```
[root@k8s-master rbac]# kubectl apply -f nginx.yaml
pod/nginx-test created
```

查看 pod 状态

```
[root@k8s-master ~]# kubectl get pod -n mynamespace
```

步骤 3　查看该 Pod 相关信息

```
[root@k8s-master ~]# kubectl describe pod nginx-test -n mynamespace
Name:               nginx-test
Namespace:          mynamespace
Priority:           0
PriorityClassName:  <none>
Node:               worker02/192.168.137.15
Start Time:         Mon, 01 Jul 2019 15:58:50 +0800
Labels:             <none>
Annotations:        kubectl.kubernetes.io/last-applied-configuration:
                      {"apiVersion":"v1","kind":"Pod","metadata":{"annotations":{},"na
ce"},"spec":{"containers":[{"image"...
Status:             Running
IP:                 10.244.1.15
Containers:
  nginx:
    Container ID:   docker://f1d17cd69f271b41e69b8a441bacc677b4588f1b37d00c0168c48a11d
    Image:          nginx:1.7.9
    Image ID:       docker-pullable://nginx@sha256:e3456c851a152494c3e4ff5fcc26f240206
    Port:           <none>
    Host Port:      <none>
    State:          Running
      Started:      Mon, 01 Jul 2019 15:59:33 +0800
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from example-sa-token-lbrnr (ro)
Conditions:
  Type              Status
  Initialized       True
  Ready             True
  ContainersReady   True
  PodScheduled      True
Volumes:
  example-sa-token-lbrnr:
    Type:        Secret (a volume populated by a Secret)
    SecretName:  example-sa-token-lbrnr
    Optional:    false
QoS Class:       BestEffort
Node-Selectors:  <none>
```

注意这里使用的 SecretsName 和之前创建的名为 example-sa 的 Service Account 中分配的 secretsname 一致。(使用 kubectl get sa -n mynamespace -o yaml 查看)

步骤 4 进入之前创建的 Pod 中查看该 secret 对象

#进入容器

```
[root@k8s-master ~]# kubectl exec -it nginx-test -n mynamespace -- /bin/bash
```

#查看容器中对应路径下的文件

```
root@nginx-test:/ls /var/run/secrets/kubernetes.io/serviceaccount/
root@nginx-test:/# ls /var/run/secrets/kubernetes.io/serviceaccount/
ca.crt namespace  token
```

该 secret 对象被挂载在该 Pod 对应的目录下，容器里的应用就可以使用 ca.crt 来访问 API Server，此时能做 GET、WATCH 和 LIST 的操作。因为 example-sa 这个 ServiceAccount 的权限已经被我们用 role 做了绑定

exit 退出

2.17.6 实训任务

步骤 1 创建一个 User Account，一个 namespace，名字自定义

步骤 2 创建一个 role，作用域为之前新创建的 namespace，赋予 get 权限

步骤 3 将该 role 绑定到新创建的 User Account 上

步骤 4 验证该 User 的权限

2.18 Kubernetes 日志管理（可选）

2.18.1 使用 sidecar 输出日志

步骤 1 在 master 节点创建/labfile/log 目录，用于保存配置文件。后续本章的 yaml 文件保存在此处。

```
[root@k8s-master /]# cd /
[root@k8s-master /]# mkdir labfile
[root@k8s-master /]# cd labfile/
[root@k8s-master labfile]# mkdir log/
```

步骤 2 部署一个 Pod，将日志输出到容器内部

```
[root@k8s-master ~]# cd/labfile/log
[root@k8s-master log]# vim logpod.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: counter
spec:
  containers:
  - name: count
    image: busybox
    args:
    - /bin/sh
    - -c
    - >
      i=0;
      while true;
      do
        echo "$i: $(date)" >> /var/log/1.log;
        echo "$(date) INFO $i" >> /var/log/2.log;
        i=$((i+1));
        sleep 1;
      done
    volumeMounts:
    - name: varlog
      mountPath: /var/log
  volumes:
  - name: varlog
  emptyDir: {}
```

```
[root@k8s-master log]# kubectl apply -f logpod.yaml
[root@k8s-master log]# kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
counter	1/1	Running	0	18s
httpd-686c9d9595-2mq6t	1/1	Running	2	52d
httpd-686c9d9595-clkjj	1/1	Running	2	52d
httpd-686c9d9595-zf7lg	1/1	Running	0	52d
myapp-mychart-7956d98c68-t9kzc	1/1	Running	0	8d

```
[root@k8s-master ~]# kubectl logs counter(pod 名)
```

#可以看到没有任何回显，即[root@k8s-master ~]# kubectl logs 读不到任何日志，因为标准输出的日志被重定向到了容器中的文件下。

步骤 3 现在修改原来的 yaml 文件，为这个 Pod 添加两个 sidecar 文件，分别将上述两个日志文件里的内容重新以 stdout 和 stderr 的方式输出。

```
[root@k8s-master log]# vim logpod.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
```

```
  name: counter
spec:
  containers:
  - name: count
    image: busybox
    args:
    - /bin/sh
    - -c
    - >
      i=0;
      while true;
      do
        echo "$i: $(date)" >> /var/log/1.log;
        echo "$(date) INFO $i" >> /var/log/2.log;
        i=$((i+1));
        sleep 1;
      done
    volumeMounts:
    - name: varlog
      mountPath: /var/log
  - name: count-log-1
    image: busybox
    args: [/bin/sh, -c, 'tail -n+1 -f /var/log/1.log']
    volumeMounts:
    - name: varlog
      mountPath: /var/log
  - name: count-log-2
    image: busybox
    args: [/bin/sh, -c, 'tail -n+1 -f /var/log/2.log']
    volumeMounts:
    - name: varlog
      mountPath: /var/log
  volumes:
  - name: varlog
  emptyDir: {}
```

先删除 Pod

```
[root@k8s-master log]# kubectl delete -f logpod.yaml
```

重新部署

```
[root@k8s-master log]# kubectl apply -f logpod.yaml
```

部署完成后使用 kubectl logs 查看一下是否有日志输出。

```
[root@k8s-master log]# kubectl logs counter cout-log-1
```

查看 pod 中容器的命令格式为 kubectl logs [Pod 名称] [容器名称]

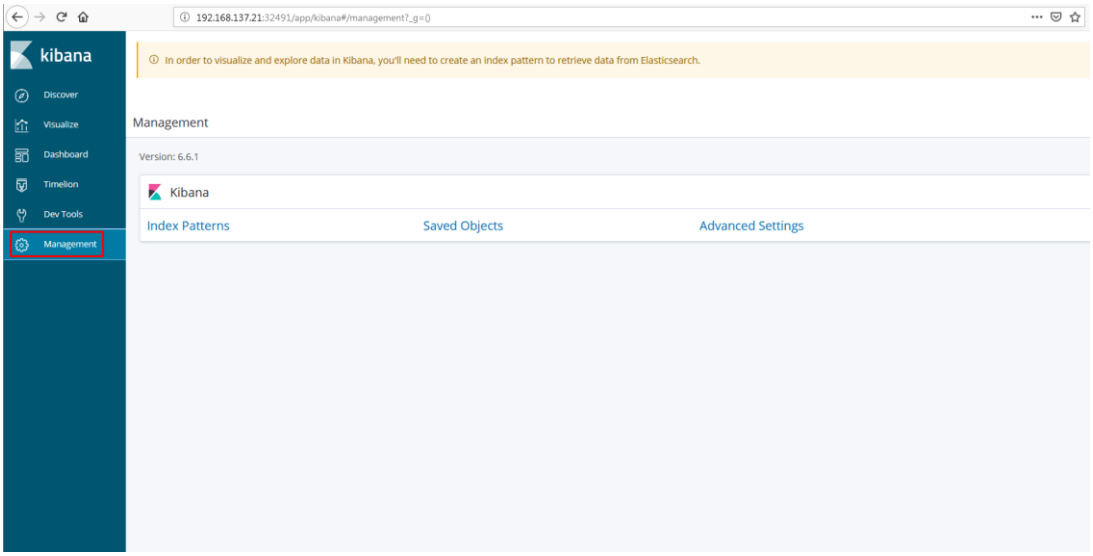
```
[root@master log]# kubectl logs counter count-log-1
0: Fri Jul 12 02:37:08 UTC 2019
1: Fri Jul 12 02:37:09 UTC 2019
2: Fri Jul 12 02:37:10 UTC 2019
3: Fri Jul 12 02:37:11 UTC 2019
4: Fri Jul 12 02:37:12 UTC 2019
5: Fri Jul 12 02:37:13 UTC 2019
6: Fri Jul 12 02:37:14 UTC 2019
```

```
[root@k8s-master log]# kubectl logs counter cout-log-1
```

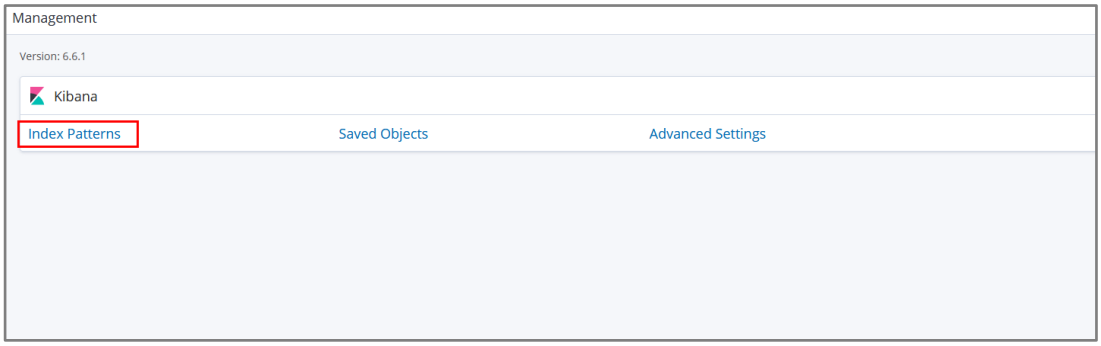
```
[root@master log]# kubectl logs counter count-log-2
Fri Jul 12 02:37:08 UTC 2019 INFO 0
Fri Jul 12 02:37:09 UTC 2019 INFO 1
Fri Jul 12 02:37:10 UTC 2019 INFO 2
Fri Jul 12 02:37:11 UTC 2019 INFO 3
Fri Jul 12 02:37:12 UTC 2019 INFO 4
Fri Jul 12 02:37:13 UTC 2019 INFO 5
Fri Jul 12 02:37:14 UTC 2019 INFO 6
Fri Jul 12 02:37:15 UTC 2019 INFO 7
Fri Jul 12 02:37:16 UTC 2019 INFO 8
Fri Jul 12 02:37:17 UTC 2019 INFO 9
Fri Jul 12 02:37:18 UTC 2019 INFO 10
```

2.18.2 EFK 的简单使用

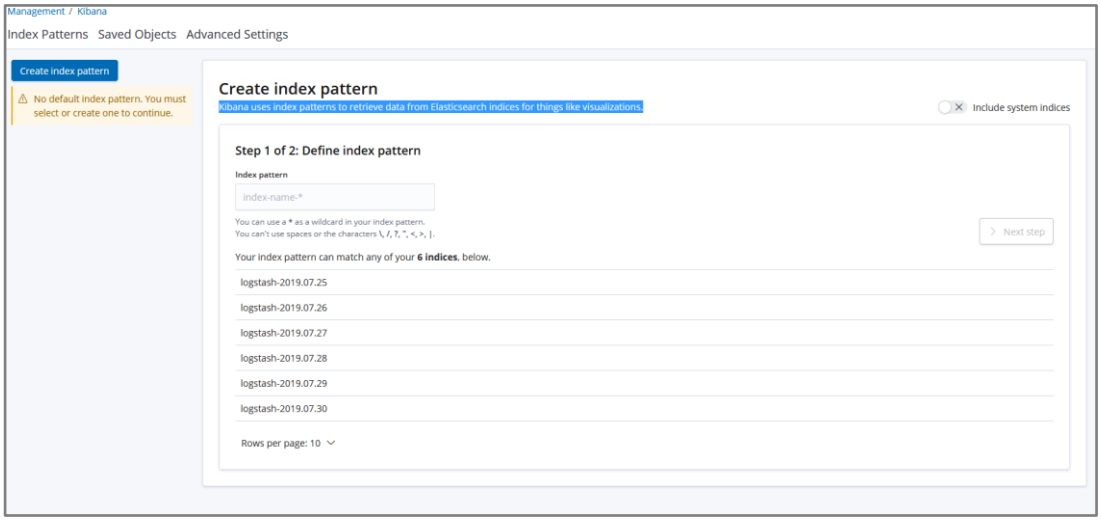
步骤 1　登录 kibana 页面，选择 Management 标签（页面地址为 NodeIP:svc 端口号）



步骤 2　点击 Index Patterns，创建 Index

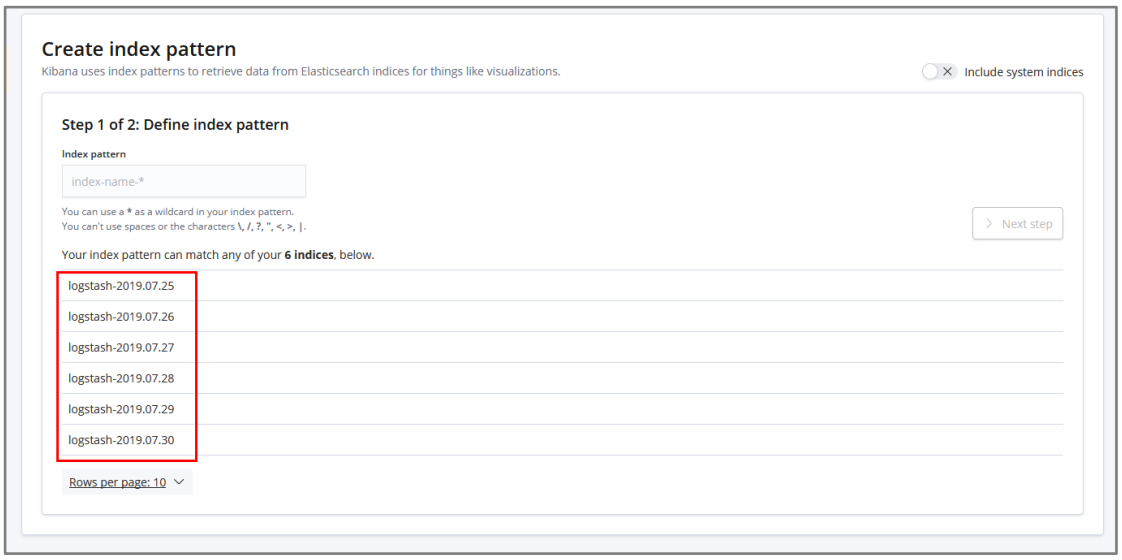


步骤 3　开始创建索引模式（Create index pattern）



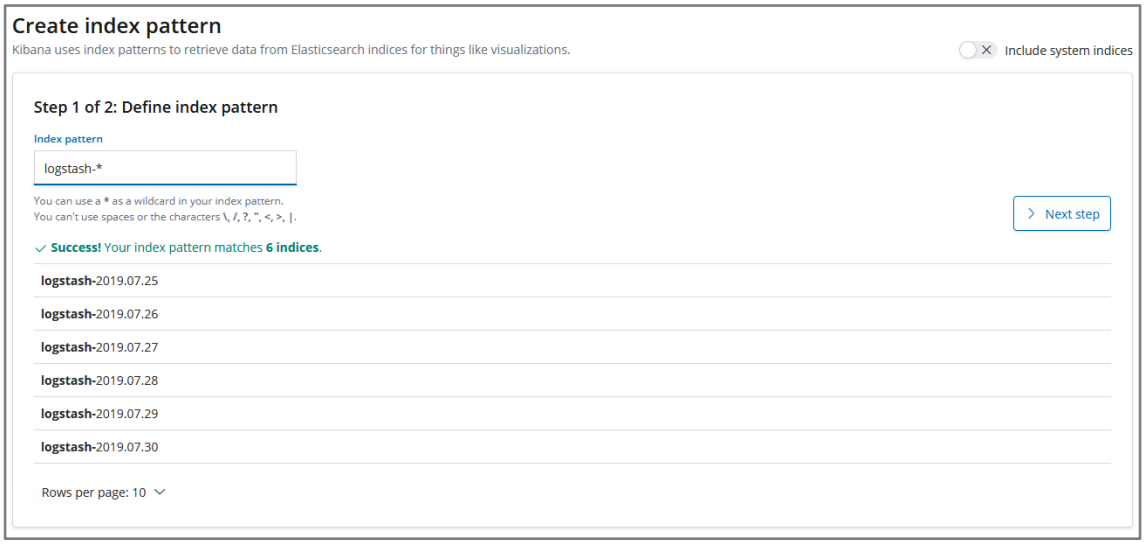
注：kibana 使用索引模式来检索数据

步骤 4　可以看到 kibana 已经从 elasticsearch 检索到了一些日志并且列在了页面上。

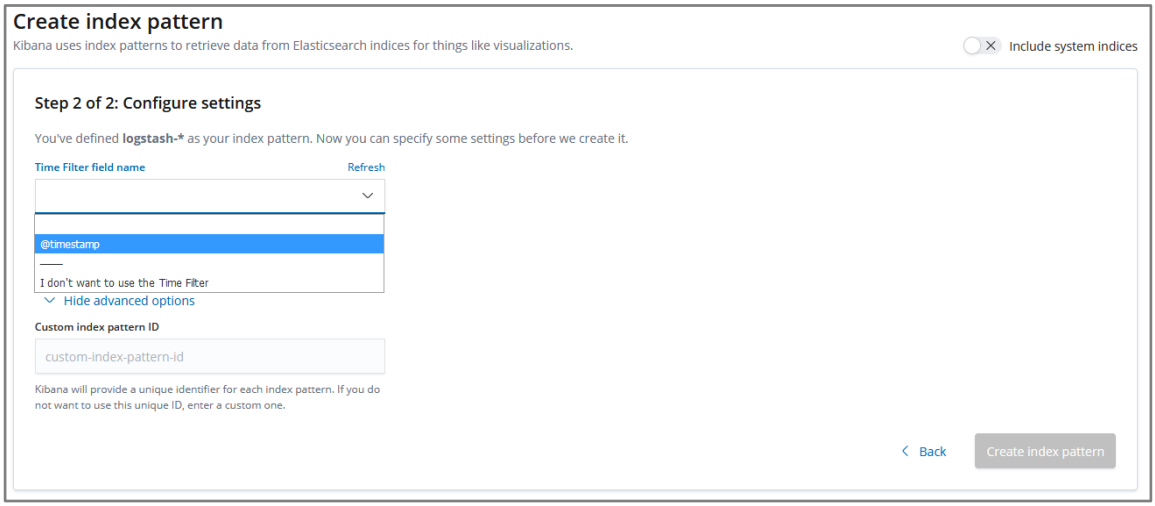


可以发现时序数据库收集到的日志格式统一都为 logstash-后跟时间戳

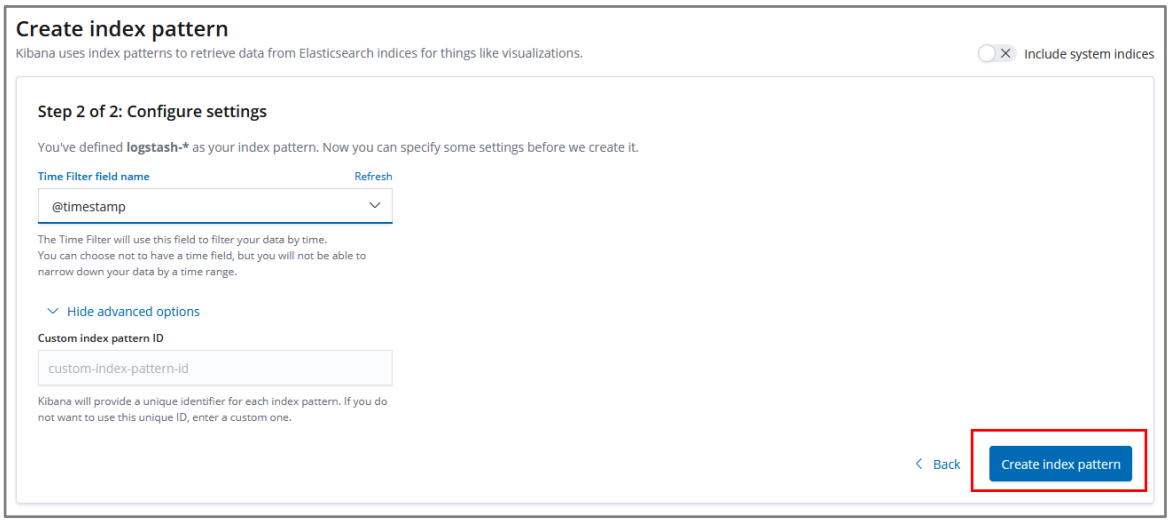
步骤 5　定义索引模式（Define index pattern）为 logstash-*，点击下一步



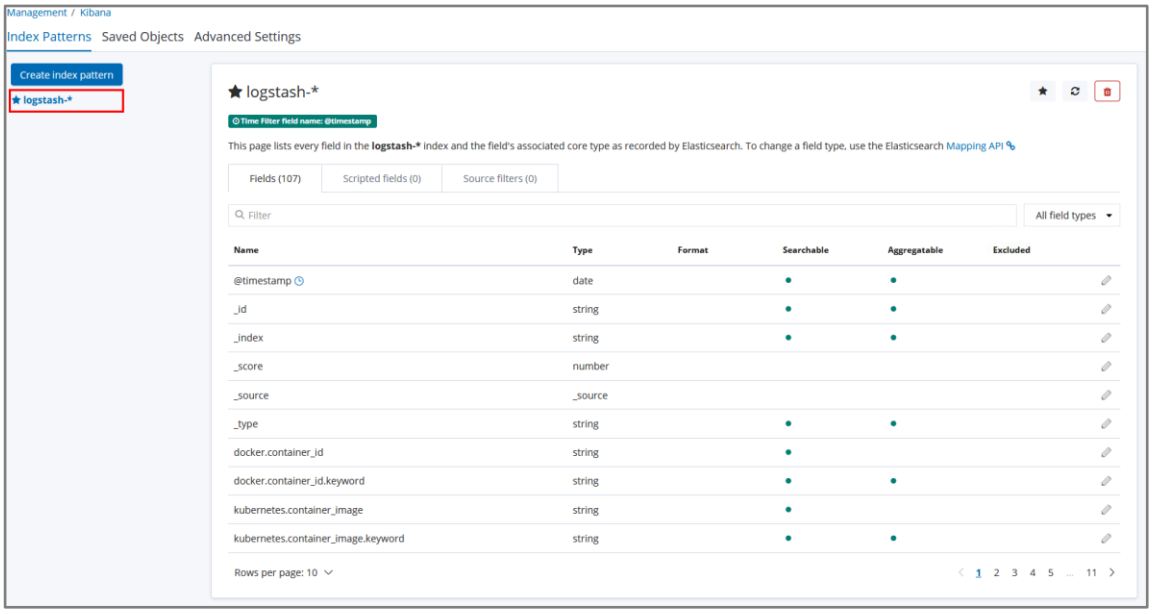
步骤 6 配置过滤条件（一般使用时间戳来过滤出想要查看的日志，即 timestamp）



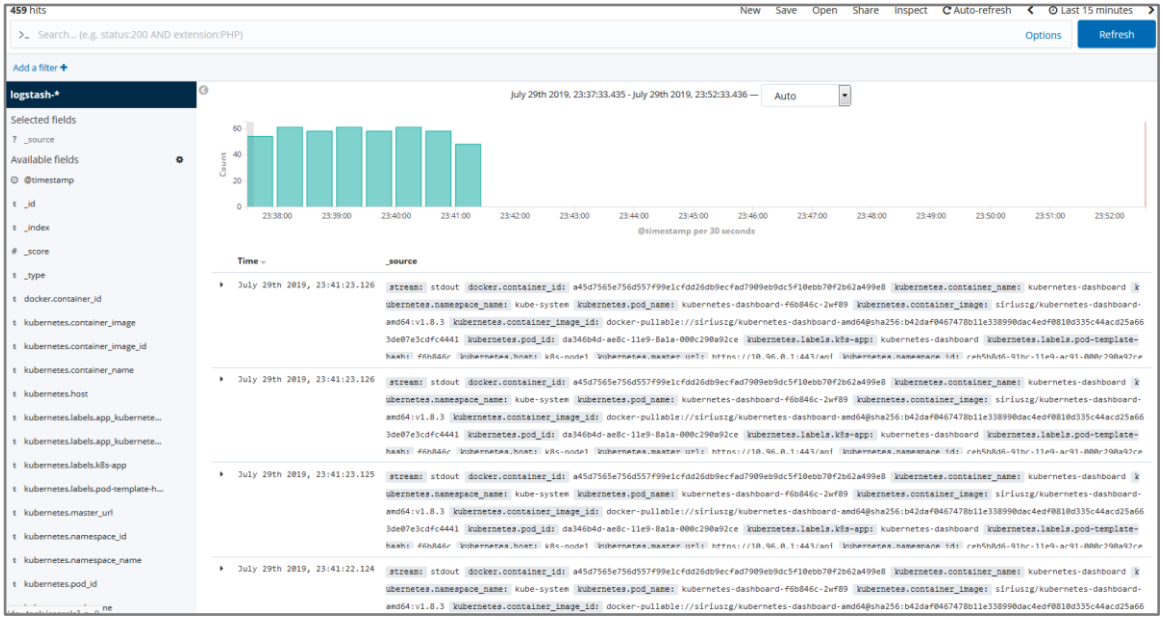
步骤 7 选择完后，点击创建即配置完成



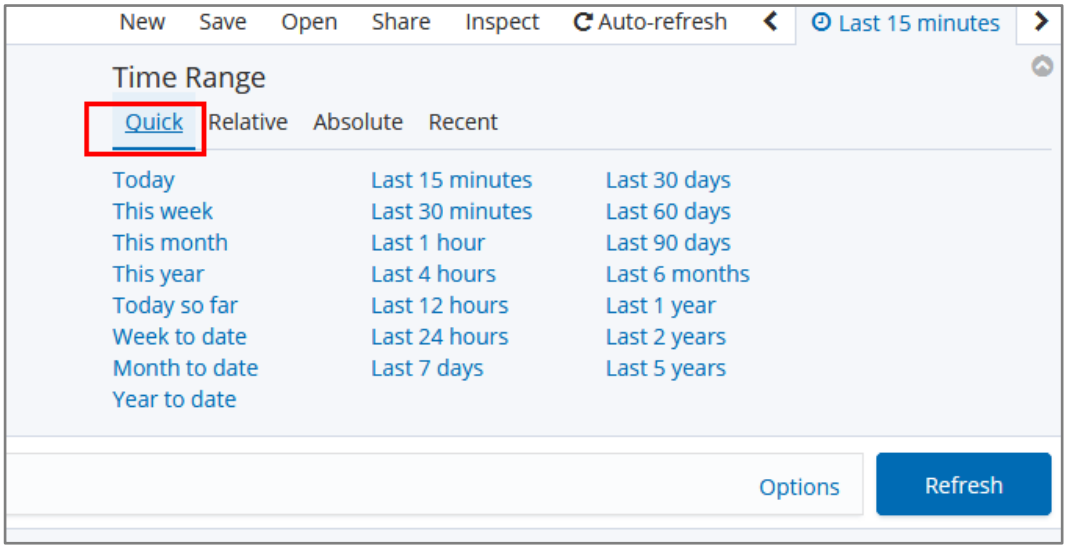
步骤 8 可以看到 Index Patterns 下多了一个 logstash-*的索引

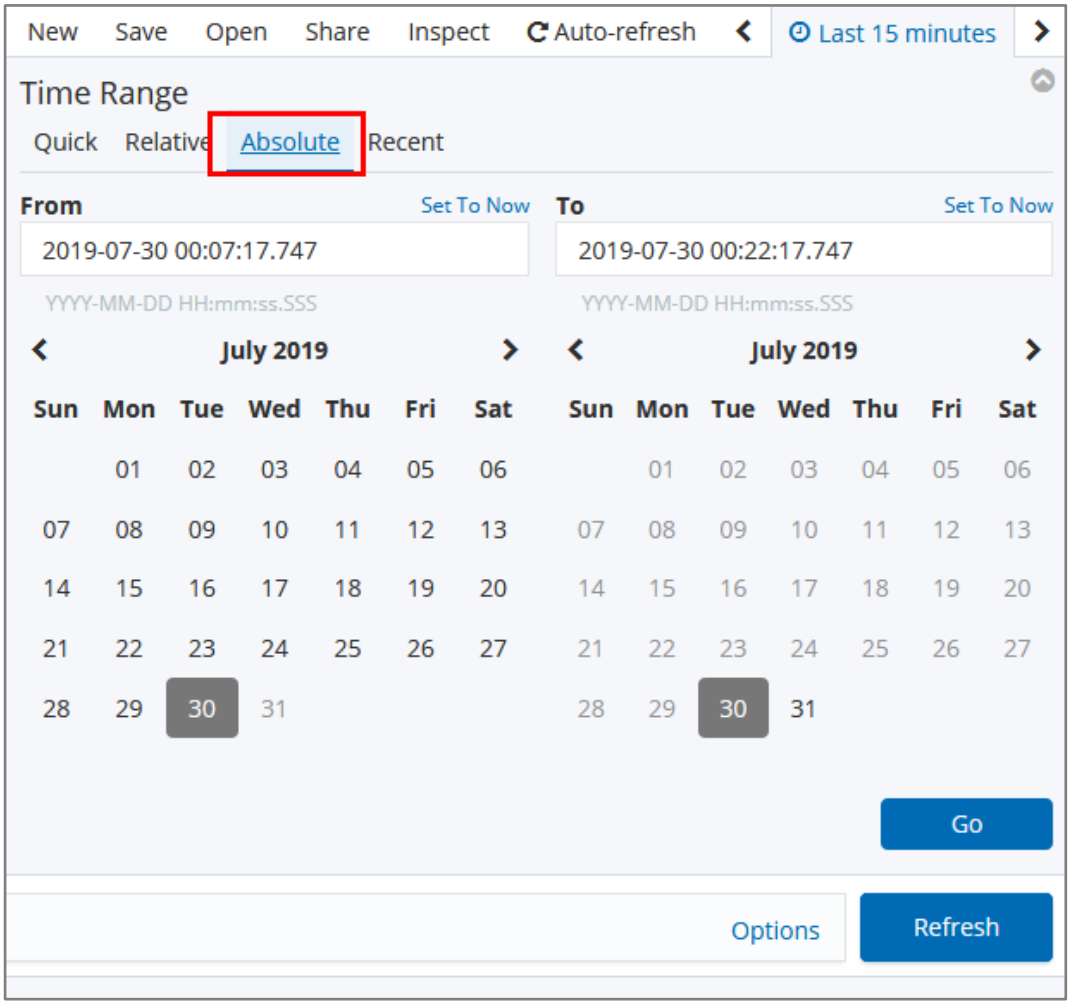


步骤 9 点击左侧的 Discover 标签，可以看到 kibana 已经开始展示日志

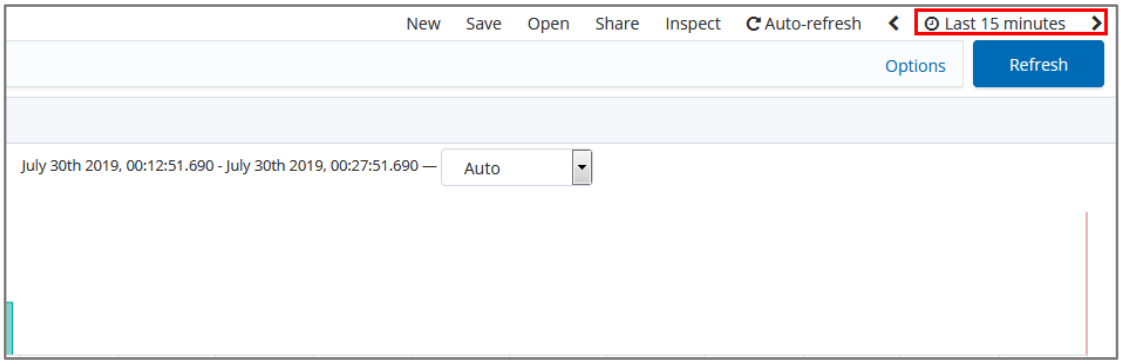


步骤 10 因日志量较为庞大，kibana 当前页面默认仅展示过去 15 分钟的日志，可以在界面右上角进行时间范围的选择，可以快速查看近期日志，也可以选择时间范围进行查看。

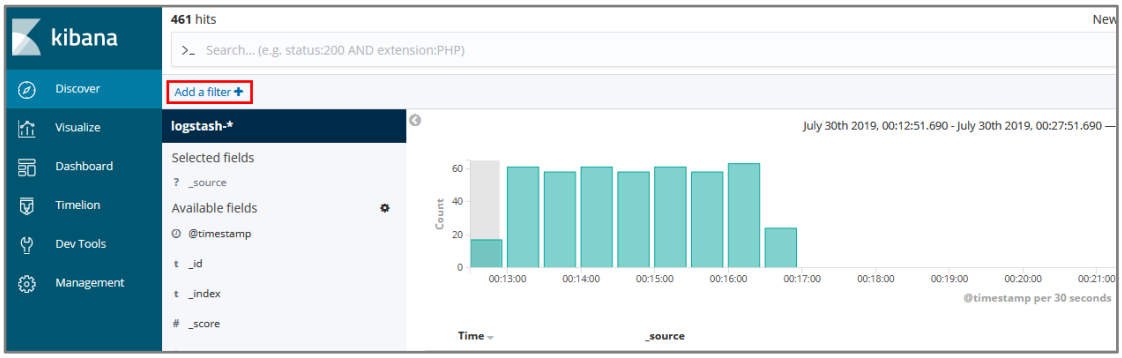




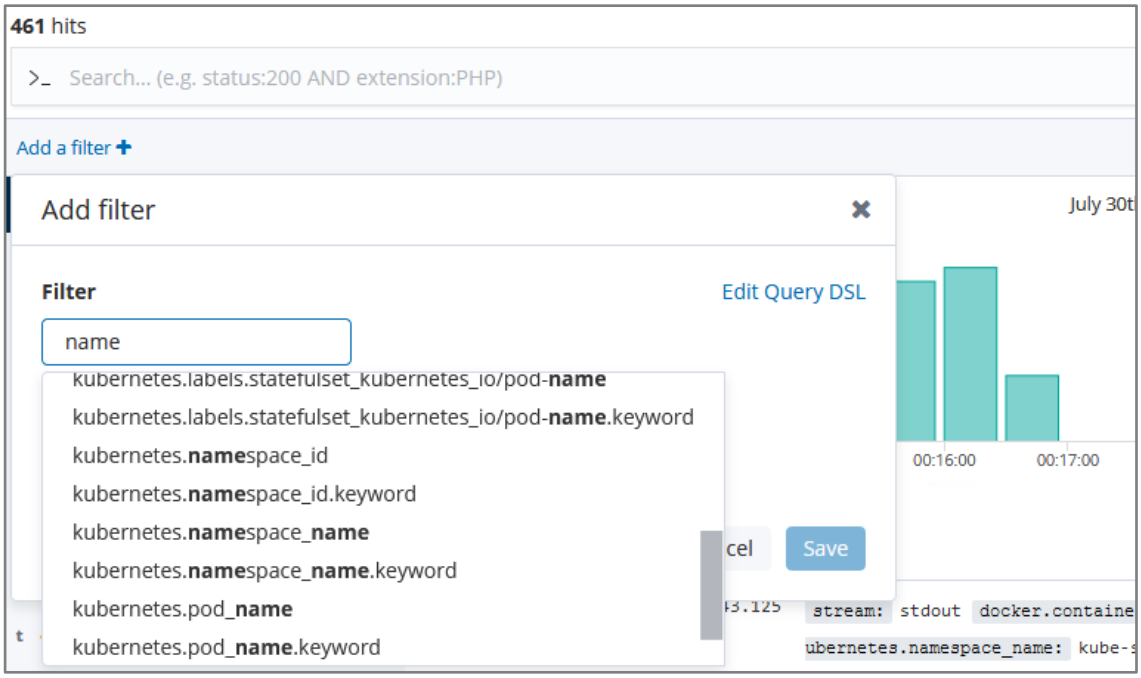
步骤 11 比如要查看过去 15 分钟以内该 Pod（kubernetes-dashboard-f6b846c-2wf89）的日志，先在右上角选择好时间范围。



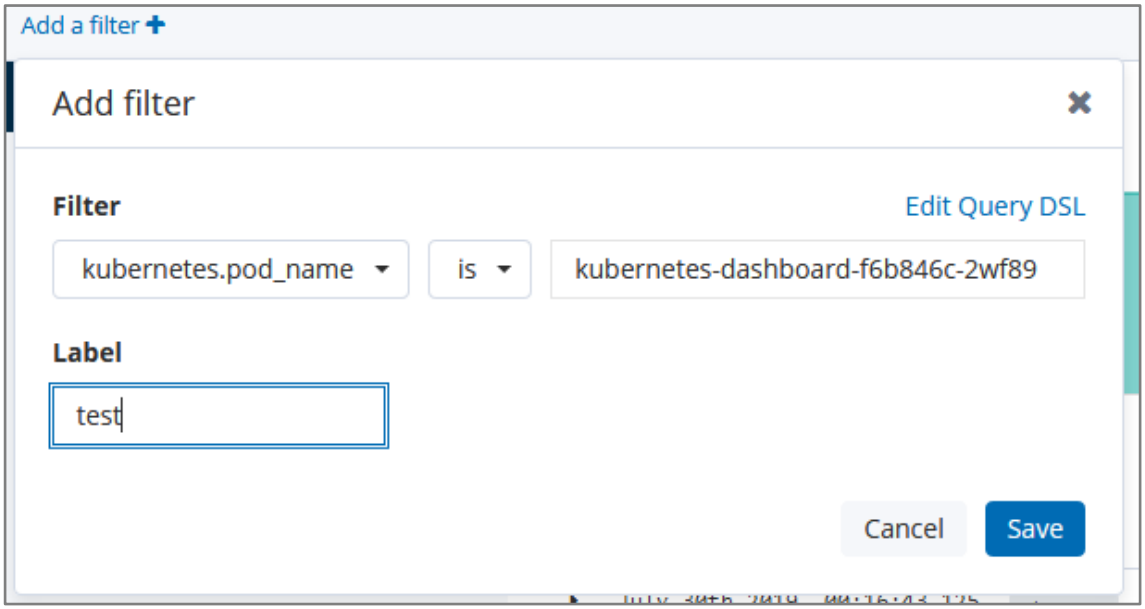
再在页面左上角添加 filter 过滤规则



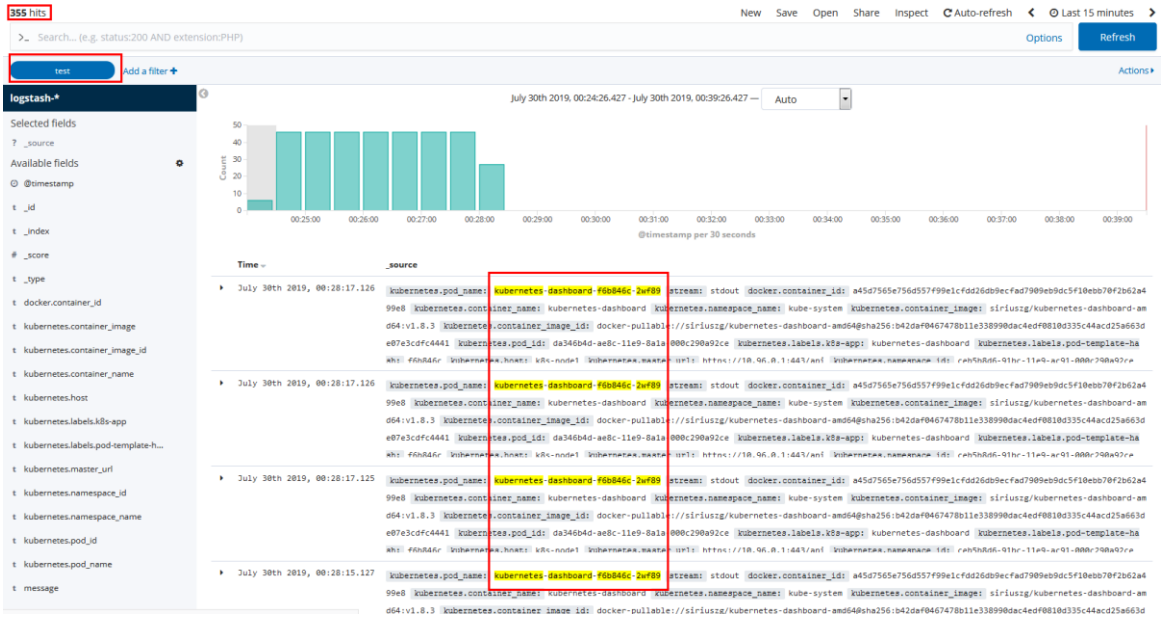
点击 Add filter 后在 Filter 一栏输入关键字 name 后，从弹出的下拉选项中找到 kubernetes.pod_name



步骤 12 填写完毕后见下图，也可以为该 filter 规则创建一个 label 方便以后随时调用



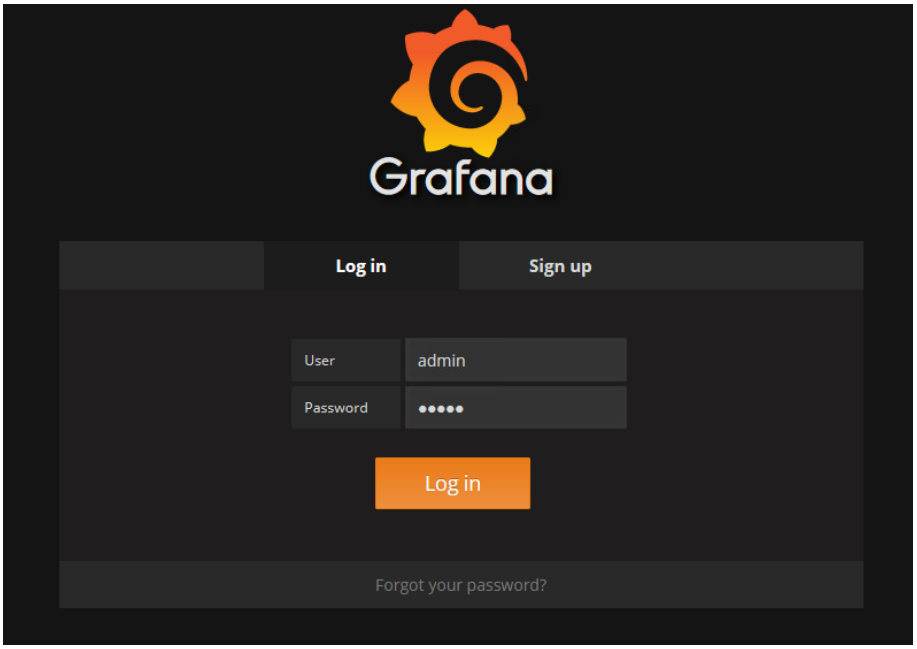
步骤 13 图中为 filter 之后的结果，左上角为命中规则的日志数据条目数（15 分钟内），test 是所创建的 filter 的 label，同时可以看到在结果中 Podname（也就是我们的 filter 条件）被作为关键则标黄。



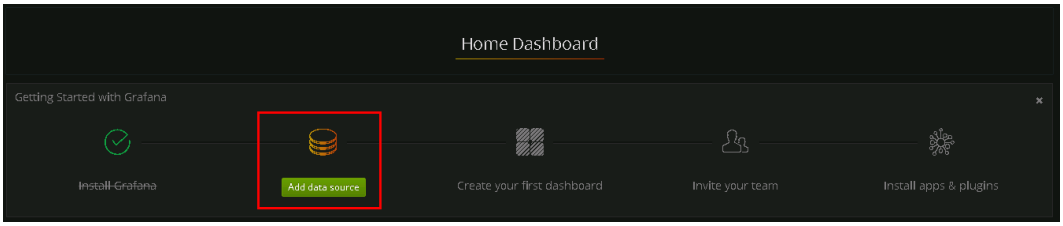
2.19 Prometheus（可选）

2.19.1 Prometheus 的简单使用

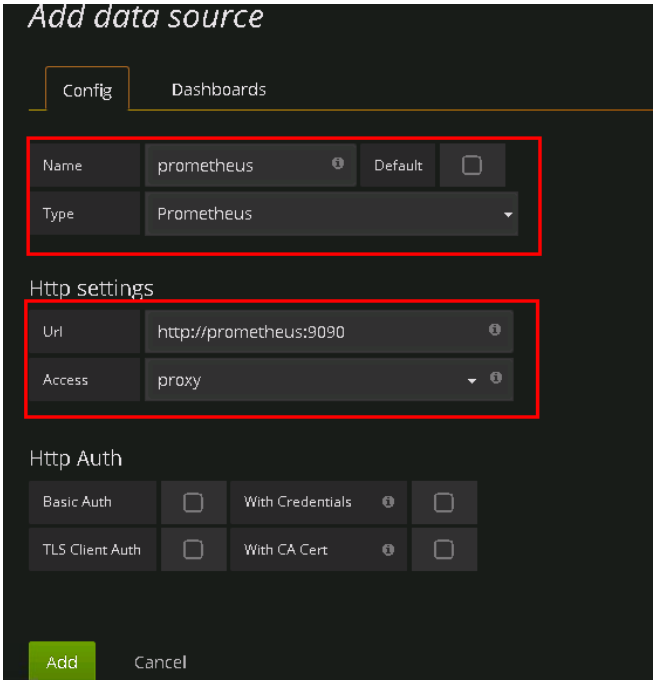
步骤 1 打开 Prometheus 页面 Grafana（页面地址为 NodeIP:svc 端口号，默认账号密码 admin/admin）



步骤 2　登录之后，根据指引添加数据源



步骤 3　根据实际的环境填写数据源



注：配置说明

Name：prometheus #别名，自定义即可

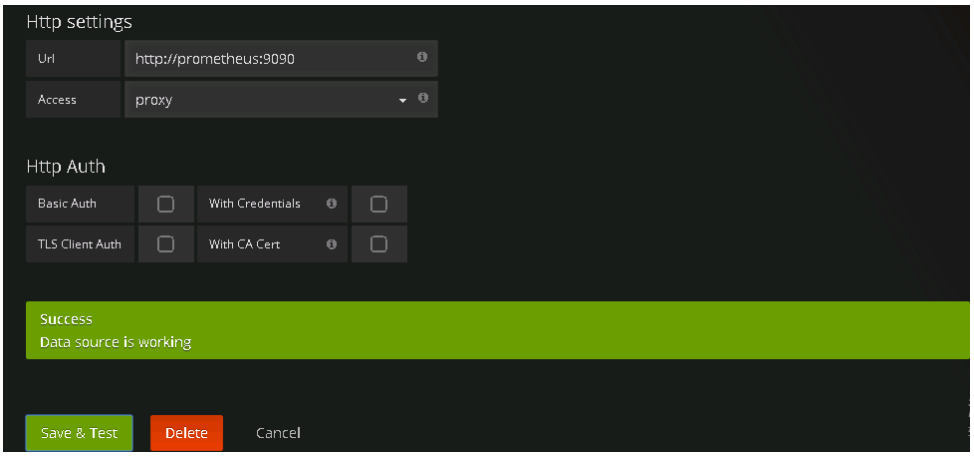
Type：prometheus #数据库类型

Access：proxy #使用代理解析 URL，即 kubernetes 的 coredns

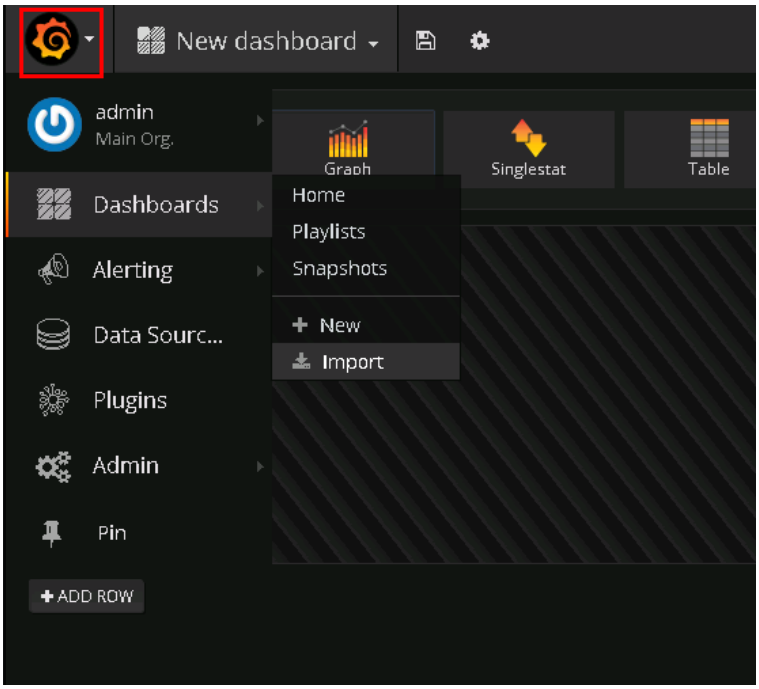
Url：http://prometheus:9090 #当 Access 使用 proxy 的时候，这里 URL 填写 http://“servicename” name:端口号，查询方式见下图

```
[root@k8s-master prometheus]# kubectl get svc -n monitor
NAME            TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
grafana         NodePort    10.111.204.58   <none>        3000:31375/TCP   5d22h
node-exporter   ClusterIP   10.96.162.237   <none>        9100/TCP         5d22h
prometheus      NodePort    10.97.145.93    <none>        9090:32720/TCP   5d22h
```

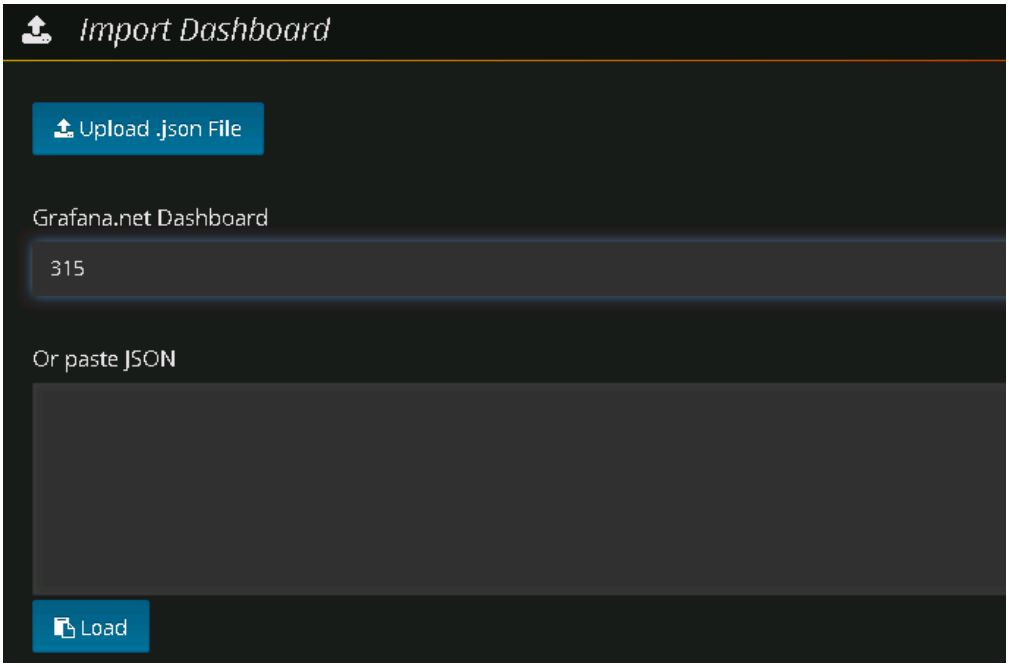
步骤 4 配置结束点击 Add，如果对接成功则在页面下方显示 success



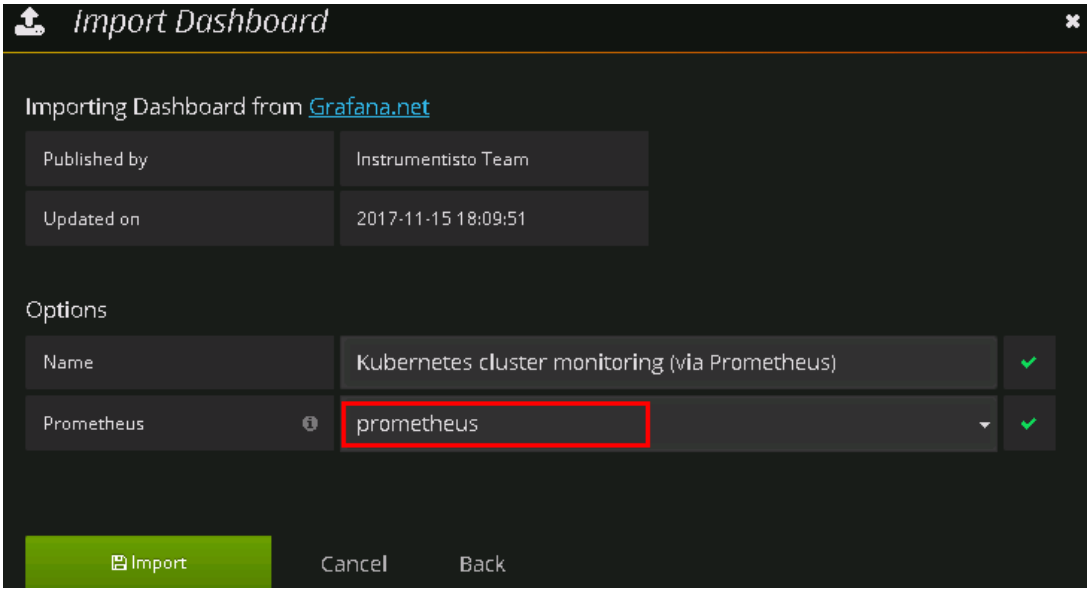
步骤 5 配置页面显示模板，点击图中红色图标后选择弹出菜单的 Dashboards，点击 Import



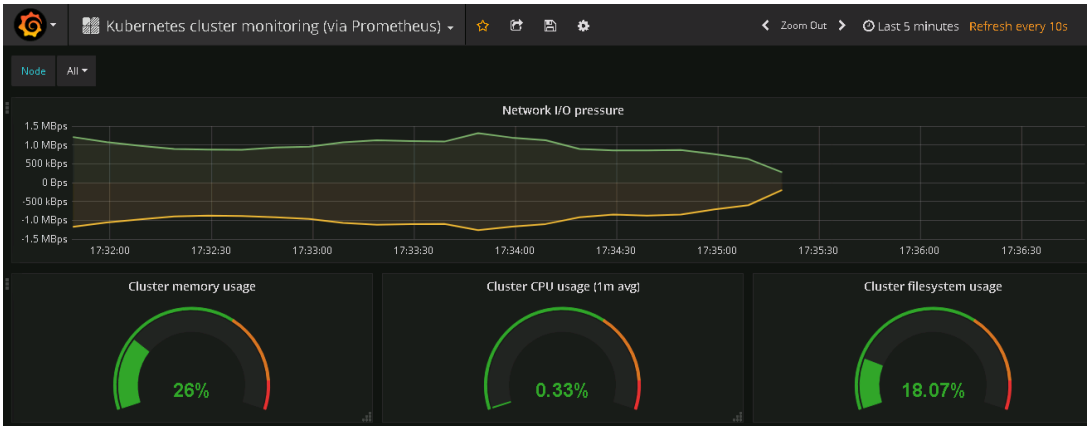
步骤 6 Grafana 为监控数据提供了很多模板，这里使用代码为 315 的模板（注意这里需要连接 internet 网络），点击 Load



步骤 7 选择 prometheus 的 datasource



步骤 8 即可看到对应的监控画面


Kubernetes cluster monitoring (via Prometheus)
Zoom Out
Last 5 minutes
Refresh every 10s
Node
All
Network I/O pressure
1.5 MBps
1.0 MBps
500 kBps
0 Bps
-500 kBps
-1.0 MBps
-1.5 MBps
17:32:00
17:32:30
17:33:00
17:33:30
17:34:00
17:34:30
17:35:00
17:35:30
17:36:00
17:36:30
Cluster memory usage
26%
Cluster CPU usage (1m avg)
0.33%
Cluster filesystem usage
18.07%